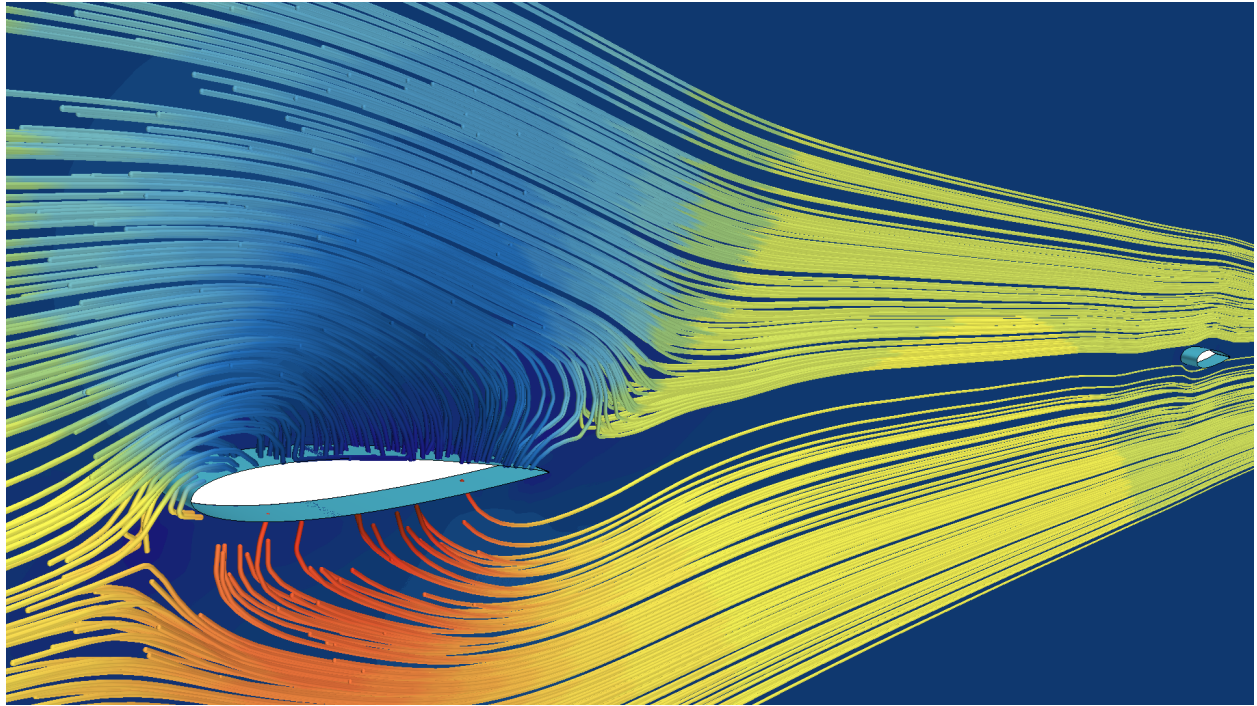




CHALMERS



TECHNISCHE
UNIVERSITÄT
DRESDEN



On the Hydrodynamics of Foiling Craft

Development of Control System for Controlling Hydrofoil using PID Controller

Master's thesis in Computational Modeling and Simulation, Computational Engineering

Martin Mathew

DEPARTMENT OF MECHANICS AND MARITIME SCIENCES

CHALMERS UNIVERSITY OF TECHNOLOGY

Gothenburg, Sweden 2025

www.chalmers.se

MASTER'S THESIS 2025

On the Hydrodynamics of Foiling Craft

Development of Control System for Controlling Hydrofoil using
PID Controller

Martin Mathew



Department of Mechanics and Maritime Sciences
Division of Marine Technology
CHALMERS UNIVERSITY OF TECHNOLOGY
Gothenburg, Sweden 2025

On the Hydrodynamics of Foiling Craft
Development of Control System for Controlling Hydrofoil using PID Controller
Martin Mathew

© Martin Mathew, 2025

Supervisor: Arash Eslamdoost, Associate Professor,
Chalmers University of Technology
Examiner 1: Dr.-Ing. Burkhard Hensel, Technische Universität Dresden
Examiner 2: Arash Eslamdoost, Associate Professor,
Chalmers University of Technology

Master's Thesis 2025
Department of Mechanics and Maritime Sciences
Division of Marine Technology
Chalmers University of Technology
SE-412 96 Gothenburg
Telephone +46 31 772 3684

Cover: Streamline visualization showing two Hydrofoils in action

Typeset in L^AT_EX
Printed by Chalmers Reproservice
Gothenburg, Sweden 2025

On the Hydrodynamics of Foiling Craft
Development of Control System for Controlling Hydrofoil using PID Controller
Martin Mathew
Department of Mechanics and Maritime Sciences
Chalmers University of Technology

Abstract

Hydrofoils play a crucial role in enhancing the efficiency of marine transport by reducing hydrodynamic drag. However, maintaining stability under varying operational conditions remains a significant challenge. This thesis presents a numerical study focused on developing a control system for hydrofoil craft using a Proportional-Integral-Derivative (PID) controller. The control system regulates the hydrofoil's angle of attack (AoA) to achieve various control objectives, ensuring stable ride height and trim. A coupled Computational Fluid Dynamics (CFD)-PID control framework is implemented, where real-time hydrodynamic responses from CFD simulations guide the dynamic adjustment of hydrofoil angles. The control system is integrated into STAR-CCM+ using a Java Macro script, allowing seamless interaction between the CFD solver and the control algorithm.

The study investigates hydrofoil performance under steady and unsteady flow conditions, evaluating the influence of environmental disturbances such as wave-induced oscillations and fluctuating flow velocities. Simulink-based modeling is used to approximate PID tuning and control behavior before full CFD integration. The simulation results demonstrate that the PID controller effectively mitigates fluctuations, leading to improved lift regulation and vessel stability. However, challenges such as aggressive angle-of-attack adjustments and reduced lift efficiency at deeper submergence levels highlight the need for further optimization.

This research contributes to the advancement of hydrofoil control strategies by providing insights into PID tuning and system adaptability. The findings emphasize the importance of integrating CFD-based hydrodynamic analysis with control algorithms for improved operational efficiency. Future research could focus on advanced control strategies, such as Model Predictive Control (MPC), adaptive control, and machine learning-based approaches, to enhance hydrofoil performance in dynamic marine environments.

Keywords: Hydrofoil, CFD, PID Controller, Ride Height Control, Stability, Hydrodynamics, AoA

Acknowledgements

I would like to express my deepest gratitude to my supervisor and examiner, Associate Professor Arash Eslaamdoost, for his invaluable guidance, continuous support, and insightful discussions throughout this research. I also extend my sincere appreciation to my second advisor and examiner, Dr.-Ing. Burkhard Hensel from Technische Universität Dresden, for his constructive feedback and expert advice, which have significantly contributed to the quality of this work.

This research was conducted as part of my Erasmus exchange thesis program between my home university, Technische Universität Dresden, and my host university, Chalmers University of Technology. I am grateful to both institutions for providing me with the opportunity to engage in this collaborative research and for their academic and technical support.

I would also like to extend my heartfelt thanks to the Erasmus coordinators from both universities for their efforts in facilitating this exchange program. Their support and dedication ensured a smooth and enriching experience throughout my time at Chalmers. Additionally, I sincerely appreciate the financial support provided by the European Union through the Erasmus+ program, which helped fund my stipend during this research period.

Furthermore, I am deeply thankful to my colleagues and friends for their encouragement, shared discussions, and invaluable feedback, which have helped refine my ideas. Their support has made this journey both productive and enjoyable. Lastly, I would like to thank the CMS exam office for their assistance in ensuring a successful registration process.

Finally, I acknowledge the Tetralith cluster for providing access to high performance computing (HPC) resources, which were essential for the simulations performed in this thesis.

Martin Mathew, Gothenburg, March 2025

Declaration

I declare that the work is entirely my own and was produced with no assistance from third parties. I certify that the work has not been submitted in the same or any similar form for assessment to any other examining body and all references, direct and indirect, are indicated as such and have been cited accordingly.

Martin Mathew, Gothenburg, March 2025

List of Acronyms

Below is the list of acronyms that have been used throughout this thesis listed in alphabetical order:

API	Application Programming Interface
AoA	Angle of Attack
CAD	Computer-Aided Design
CFD	Computational Fluid Dynamics
DFBI	Dynamic Fluid-Body Interaction
DOF	Degrees of Freedom
DNS	Direct Numerical Simulation
EMA	Exponential Moving Average
IAE	Integral of Absolute Error
ISE	Integral of Squared Error
ITAE	Integral of Time-weighted Absolute Error
LES	Large Eddy Simulation
MPC	Model Predictive Control
MPI	Message Passing Interface
NS	Navier-Stokes
PID	Proportional-Integral-Derivative
PV	Process Variable
RANS	Reynolds-Averaged Navier-Stokes equations
RMSE	Root Mean Square Error
SIMPLE	Semi-Implicit Method for Pressure Linked Equations
SST	Shear Stress Transport
SP	Setpoint
VOF	Volume of Fluid

Nomenclature

Below is the nomenclature of parameters, and variables that have been used throughout this thesis.

Parameters

Target Ride Height	0.1 meters (desired height of the vessel)
Target Trim Angle	0.0 degrees (desired level of the vessel).
Sampling Interval	0.0001 seconds – Initial sampling interval.
Max Integral Sum	10.0 – Anti-windup limit for the integral term in PID.
AoA_{MIN}	-8 degrees – Minimum angle of attack (AoA).
AoA_{MAX}	10.0 degrees – Maximum angle of attack (AoA).
$Max_{\Delta AoA}$	0.05 degrees – Maximum change in AoA per step (rate limit for AoA).
Implicit Unsteady Time-Step	0.0001 s (Time step for unsteady simulation)
System Body Mass	10 kg (Mass of the system body)
Inner Iterations	25 (Number of inner iterations per time step)
System Release Time	1 s (Time after which DFBI equations are applied)
System Ramp Time	3 s (Time duration for smooth motion application in DFBI)

Variables

AoA	Angle of Attack of the hydrofoil
k_p	Proportional gain of the PID controller
k_i	Integral gain of the PID controller
k_d	Derivative gain of the PID controller

Previous Error	Error value from the last time step (used for derivative calculation)
Integral Sum	Accumulated integral error (used in the integral term of PID)
Last Recorded Time	Timestamp of the last recorded PID update

Contents

List of Acronyms	xi
Nomenclature	xii
List of Figures	xix
List of Tables	xxi
1 Introduction	1
1.1 Problem Statement	1
1.2 Research Questions	2
1.3 Organization of the Report	2
2 Background	5
2.1 Hydrofoils	5
2.1.1 Role of Hydrofoils in Marine Craft	6
2.1.2 Foil Configuration	6
2.2 Literature Review	8
2.3 Principles of Hydrofoil Hydrodynamics	11
2.3.1 Lift and Drag Forces	11
2.3.2 Flow Regimes and Boundary Layers	12
2.4 Dynamics and Unsteady Hydrodynamic Phenomena in Hydrofoil Vessels	13
2.4.1 Stability and Control Challenges	13
2.4.2 Unsteady Hydrodynamic Phenomena	14
2.4.2.1 Lift Coefficient Loop (C_l vs. AoA)	14
2.4.2.2 Impact of Unsteady Inlet Conditions	15
2.5 Introduction to Computational Fluid Dynamics	16
2.5.1 Governing Equations: Navier-Stokes Equations	17
2.5.1.1 Mathematical Formulation	17
2.5.1.2 Solution of NS equation	19
2.5.2 Steps involved in CFD Simulation	19
2.5.3 Relevance of CFD in Hydrofoil Analysis	20
2.5.4 Multiphase Flow and its Simulation in CFD	21
2.5.4.1 Importance of Multiphase Flow in Hydrofoil Sim- ulations	21
2.5.4.2 Relevance to Hydrofoil Simulation	22

2.5.4.3	Simulation Approach	22
2.5.4.4	Challenges in Multiphase Flow Simulations	24
2.5.5	High-Performance Computing (HPC) in CFD	25
2.5.5.1	Role of HPC in CFD Simulations	25
2.5.5.2	Parallel Computing Techniques in CFD	25
2.5.5.3	Workflow of CFD Simulation Using HPC	25
2.6	Control Algorithms for Hydrofoil Stability and Performance	26
2.6.1	Introduction to Control Systems	26
2.6.1.1	Key Components of a Control System	27
2.6.1.2	Types of Control Systems	27
2.6.2	PID Controller and its Role in Hydrofoil Applications	28
2.6.2.1	Tuning of PID Controller	29
2.6.2.2	Limitations of PID Controllers	30
2.6.3	Control System Performance Analysis and Evaluation	31
2.6.3.1	Performance Metrics of Control Systems	31
2.6.3.2	Evaluation Metrics of Control Systems	32
3	Methodology	35
3.1	CFD Simulation Framework	35
3.1.1	Geometry and Mesh Generation	35
3.1.1.1	Hydrofoil Geometry	36
3.1.1.2	Computational Domain	36
3.1.1.3	Meshing	36
3.1.1.4	Boundary Conditions	38
3.1.2	CFD Simulation Settings	39
3.1.2.1	Solver Settings	39
3.1.2.2	Initial Testing: 2D Setup	40
3.1.2.3	Transition to Unsteady Inlet Velocity Conditions	40
3.1.2.4	Multiphase Flow Setup in a Semi-2D Domain	40
3.1.2.5	Flow Conditions and Wave Modeling	41
3.1.2.6	Introduction to DFBI Motion	42
3.1.2.7	Implementation of DFBI in the Semi-2D Setup	42
3.1.2.8	Measurement of Ride Height and Pitch Angle in STAR-CCM+	43
3.2	Control System Framework	44
3.2.1	PID Control System Implementation	44
3.2.1.1	PID Controller in Java Macros	45
3.2.1.2	Coupling between CFD and PID Control System	47
3.2.1.3	PID Parameters Selection and Tuning	47
3.2.1.4	Discrete PID Control for Improved Stability and Noise Reduction	48
3.2.1.5	Challenges in Integration	49
3.2.2	Simulink-Based Control System Analysis	50
3.2.2.1	Assumptions in the Simulink model	51
3.2.2.2	Simulink Modeling for Hydrofoil Control System in Steady-Flow Conditions	51

3.2.2.3	Simulink Modeling for Hydrofoil Control System in Unsteady-Flow Conditions	52
3.2.2.4	Simulink Modeling for Hydrofoil Control System with Noise in Velocity Input	52
4	Results	55
4.1	Simulink-Based Simulation Results	55
4.1.1	Steady-Flow Response for Target Lift of 5000N	55
4.1.2	Unsteady-Flow Response with Sinusoidal Velocity Inlet	56
4.1.3	Response Under Additional Noise in Velocity Input	59
4.2	CFD-Based Simulation Results	62
4.2.1	Steady-Flow 2D Setup	62
4.2.1.1	Key Observations	64
4.2.2	Unsteady-Flow 2D Setup	65
4.2.2.1	Key Observations	65
4.2.3	Semi-2D Setup with Multiphase Flow Integration	68
4.2.3.1	Filter Selection for Lift Signal	69
4.2.3.2	Optimization of Filter Parameters	72
4.2.3.3	Hydrofoil Performance at Different Depths	74
4.2.3.4	Ride Height and Trim Angle Control	80
5	Conclusion	93
5.1	Future Scope of Research	95
	Bibliography	97
A	Appendix 1	I
A.1	Lift Force and AoA Calculation Function	I
A.2	Calculation of Moments of Inertia for DFBI model	III
A.3	Code for visualizing First-order and Fifth-order wave	IV
A.4	Code for generating velocity scene animation	V
A.5	Java Macro for Steady-Flow 2D Setup	VII
A.6	Java Macro for Lift Signal Filtering Comparison	XIV
A.7	Java Macro for Optimization of Filter Parameters	XXII
A.8	Java Macro for Hydrofoil Performance at Different Depths	XXVIII
A.9	Java Macro for Hydrofoil Control of Ride Height and Trim Angle	XXXVI
A.9.1	Functionality Overview	XXXVI
A.9.2	Function Descriptions	XXXVII
B	Appendix 2	XLIX
B.1	Steady-State 2D Setup Raw Lift Force Plot	XLIX
B.2	Unsteady-State 2D Setup Raw Lift Force Plot	L
B.3	Raw Lift Plot for Dual Hydrofoil Control	LI
B.4	Plots for Hydrofoil Depth 0.5CL 4.2.3.3	LII
B.5	Plots for Hydrofoil Depth 1.5CL 4.2.3.3	LV
B.6	Plots for Hydrofoil Depth 3.0CL 4.2.3.3	LVIII

C Project Timeline

LXI

List of Figures

2.1	Foiling Craft	6
2.2	Hydrofoil Types	7
2.3	Angle of Attack (AoA)	8
2.4	Flow Regime and Boundary Layer	13
2.5	C_l vs AoA	15
2.6	Wave Orbital Velocities	16
2.7	Multiphase: Dispersed and Separated Flow	22
2.8	Volume of Fluid (VOF)	23
2.9	Block diagram of a Closed Control System	28
2.10	Block diagram of a PID Controller	29
2.11	Performance Metrics of Control Systems	32
3.1	Hydrofoil Geometry - NACA 0012	36
3.2	Computational domain with boundary conditions	37
3.3	Example of polygonal mesh	38
3.4	Example of trimmed cell mesh	38
3.5	Comparison of first-order and fifth-order stokes waves	41
3.6	DFBI model	42
3.7	DFBI motion selection	43
3.8	Hydrofoil Vessel Motion Measurements	44
3.9	Control flowchart	45
3.10	Simulink model for steady-flow conditions	52
3.11	Simulink model for unsteady-flow conditions	53
3.12	Simulink model with noise addition in velocity input	53
3.13	Velocity subsystem	54
4.1	Response curve for steady-flow model	56
4.2	Response curve for AoA and sinusoidal inlet velocity	57
4.3	Response curve for unsteady-flow model	58
4.4	Response curve for unsteady-flow model (enlarged)	58
4.5	Noise model for velocity	60
4.6	Response curve for AoA and sinusoidal inlet velocity with noise	60
4.7	Response curve for unsteady-flow model with noise	61
4.8	Response curve for unsteady-flow model with noise (enlarged)	61
4.9	Lift Force vs Time	63
4.10	AoA vs Time	63
4.11	Steady flow—final velocity scene	64

List of Figures

4.12	Lift Force vs Time	66
4.13	AoA vs Time	66
4.14	Unsteady flow—final velocity scene	67
4.15	Lift Force vs Time	70
4.16	AoA vs Time	70
4.17	Mean Lift vs Time	71
4.18	Smoothed Lift vs Time	74
4.19	Hydrofoil at varying depths from the free surface in Density Scene	75
	(a) Depth = 0.5CL	75
	(b) Depth = 1.5CL	75
	(c) Depth = 3.0CL	75
4.20	Mean Lift Data vs Time for Depth 0.5CL	77
4.21	AoA vs Time for Depth 0.5CL	77
4.22	Mean Lift Data vs Time for Depth 1.5CL	78
4.23	AoA vs Time for Depth 1.5CL	78
4.24	Mean Lift Data vs Time for Depth 3.0CL	79
4.25	AoA vs Time for Depth 3.0CL	79
4.26	Ride Height vs Time for <i>RideHeight</i> = 0.3m	84
4.27	Ride Height vs Time for <i>RideHeight</i> = −0.2m	84
4.28	Trim Angle vs Time for <i>RideHeight</i> = 0.3m	85
4.29	Trim Angle vs Time for <i>RideHeight</i> = −0.2m	86
4.30	AoA vs Time for <i>RideHeight</i> = 0.3m	88
4.31	AoA vs Time for <i>RideHeight</i> = −0.2m	88
4.32	Mean Lift vs Ride Height for Front Hydrofoil	89
4.33	Mean Lift vs Ride Height for Rear Hydrofoil	90
B.1	Raw Lift Force vs Time for Steady-State 2D Setup [??]	XLIX
B.2	Raw Lift Force vs Time for Unsteady-State 2D Setup [??]	L
B.3	Raw Lift vs Time for Dual Hydrofoil Control [4.2.3.2]	LI
B.4	Raw Lift vs Time	LII
B.5	Smoothed Lift vs Time	LIII
B.6	Combined Lift vs Time	LIV
B.7	Raw Lift vs Time	LV
B.8	Smoothed Lift vs Time	LVI
B.9	Combined Lift vs Time	LVII
B.10	Raw Lift vs Time	LVIII
B.11	Smoothed Lift vs Time	LIX
B.12	Combined Lift vs Time	LX

List of Tables

2.1	Evaluation Metrics	33
2.2	Differences Between Performance Metrics and Evaluation Metrics .	34
4.1	PID Gains for the Control System	55
4.2	Unsteady Flow Velocity Block Parameters	56
4.3	Noise model parameters for velocity input consisting of three blocks: base signal, high-frequency superimposed signal, and band-limited white noise.	59
4.4	Performance Metrics for Different Filters	71
4.5	IAE Values for Different Alpha Values (Smoothed Data)	73
4.6	Data Summary for Hydrofoil Ride Height Control System	82
4.7	Links for animations of ride height simulations, visualizing the density field from STAR-CCM+	91

1

Introduction

Hydrofoils are specialized structures that lift a vessel/boat above the water's surface, significantly reducing drag and enhancing speed capability and efficiency. The focus of this thesis is on the development of a control system that controls the angle of attack of the hydrofoil using a Proportional-Integral-Derivative (PID) controller. This control mechanism aims to optimize the hydrofoil's performance by maintaining stability and improving responsiveness under varying operational conditions. This chapter provides an overview of the fundamental concepts and significance of hydrofoil craft in modern marine engineering. Additionally, this chapter includes a clear problem statement, the purpose of the study, and the research questions guiding this investigation.

By integrating advanced computational modeling and simulation techniques, this thesis aims to contribute valuable methods to the design and control of hydrofoil craft, leading the way for more efficient and effective maritime applications.

1.1 Problem Statement

The hydrofoil concept is a crucial area of research for boatbuilders focused on sustainable marine transport. Despite advancements in control systems for hydrofoil craft, existing methodologies often struggle to maintain robust performance under dynamic and unpredictable conditions, such as varying wave heights and frequencies. A notable gap in the research pertains to the control strategies associated with PID controllers. While PID controllers are favored for their simplicity and ease of implementation, tuning these controllers poses significant challenges in practice. The dynamics of the marine environment can vary considerably, leading to fluctuations in hydrofoil performance that directly affect the control performance of PID systems. Consequently, the PID parameters must be continuously retuned to adapt to changing flow conditions and environmental disturbances.

Furthermore, many existing control strategies are either overly simplistic or fail to adequately account for the complex flow phenomena that significantly influence vessel performance. This thesis intends to prevail over these limitations by developing a more robust control system capable of effectively managing the dynamic behaviors of hydrofoil craft across a range of operational scenarios. By improving the responsiveness and precision of control systems, the proposed method seeks to enhance the overall efficiency and performance of hydrofoil vessels.

The scope of this study is centered around the development and analysis of control systems for hydrofoil vessels, focusing primarily on enhancing their stability

and performance under varying operational conditions. The research incorporates both theoretical and computational approaches to investigate the hydrodynamic behavior of hydrofoils and the effectiveness of control algorithms.

The study also evaluates unsteady hydrodynamic phenomena, the impact of multiphase flow, and the challenges associated with integrating control systems into marine applications. A combination of Computational Fluid Dynamics (CFD) tools and Simulink-based models is used to simulate various scenarios, enabling a comprehensive analysis of the control systems under both steady and unsteady conditions. The research addresses critical aspects, such as noise reduction, parameter tuning, and the coupling between CFD simulations and control algorithms, to optimize system performance.

Additionally, the study emphasizes the importance of High-Performance Computing (HPC) in conducting large-scale CFD simulations to ensure accurate and efficient analysis. This scope also extends to investigating potential improvements in control strategies and evaluating their feasibility for physical applications.

The objective of this study is to design and implement a robust control system for hydrofoil craft capable of efficiently regulating the angle of attack and maintaining target lift, while offering adaptable control actions such as ride height regulation. The system should be able to respond effectively to varying environmental conditions, such as changes in velocity and the influence of waves. By incorporating advanced control strategies, this research aims to optimize the operational efficiency, stability, and dynamic response of hydrofoil vessels.

1.2 Research Questions

The study seeks to address the following key research questions:

- How can a control system be designed to maintain the optimal angle of attack for hydrofoil craft under varying operational conditions?
- What advanced control strategies can be implemented to improve the stability and adaptability of hydrofoil vessels, especially in response to wave and velocity fluctuations?

1.3 Organization of the Report

The report is structured to ensure a clear and logical progression of ideas, from understanding the problem to presenting the results and suggesting future research directions.

Chapter 2 begins with an extensive review of relevant literature, providing a solid theoretical foundation for understanding the dynamics of hydrofoil systems and the design of associated control mechanisms. The chapter initially presents a detailed examination of the hydrofoil vessel's mechanical principles, followed by an exploration of the integration and application of control systems within marine environments. Furthermore, the chapter introduces key concepts in control

theory, with a particular emphasis on PID controllers, which form the core of the control strategy employed in this research.

The methodology chapter provides a detailed framework for the research, comprising both CFD simulations and control system implementation. The CFD simulation framework includes the generation of hydrofoil geometry, computational domain setup, meshing techniques, boundary condition specifications, and solver configurations. Various simulation scenarios, such as steady-flow, unsteady-flow, and multiphase conditions, are explored to replicate realistic operating environments.

In parallel, the control system framework focuses on the development and integration of a PID controller. This includes implementation through Java macros, parameter tuning, and coupling with the CFD environment. A complementary analysis using Simulink models evaluates the control system's performance under steady-state, unsteady, and noisy conditions.

Chapter 4 presents the results obtained from the simulations and provides an analysis of the findings. The focus is on evaluating the performance of the control mechanism, with an emphasis on performance metrics and how well the system achieved the desired outcomes. The results are compared to the expectations set at the start of the study, and the chapter offers insights into the behavior of the control system, discussing any unexpected observations or implications for future research.

Chapter 5 summarizes the key findings of the study and discusses its limitations. It also offers suggestions for future research.

The appendices contain supplementary information that supports the main content of the report. This includes a detailed description of the simulation setup, the code used for macro programming, and additional plots and visualizations that offer further insight into the study's findings. These materials are provided to ensure transparency and help readers understand the full scope of the work.

2

Background

This chapter begins with a comprehensive literature review and subsequently introduces the fundamental concepts supporting the research on hydrofoil craft, focusing primarily on two essential domains: CFD and control algorithms. Hydrofoil craft represents a significant advancement in marine transportation, as they reduce drag and increase speed by lifting the vessel's hull above the water, resulting in more efficient use of resources and providing smoother rides. These characteristics make hydrofoil vessels a desirable option for both commercial and recreational use. However, the operation of hydrofoils involves complex fluid-structure interactions and dynamic behaviors, requiring sophisticated computational tools and control systems to ensure optimal performance.

CFD plays a critical role in simulating the complex fluid-structure interactions that arise from hydrofoils operating in various environmental conditions. It allows for a better understanding of key performance metrics such as lift, drag, and stability, which are essential for the vessel's performance. By modeling how the hydrofoil interacts with the surrounding fluid under different conditions, CFD helps optimize design and predict behavior, ensuring maximum efficiency and stability.

On the other hand, hydrofoil vessels depend on control algorithms to manage their dynamic behavior in real-time. Controllers play a crucial role in regulating the angle of attack or maintaining the desired lift, ensuring stable ride height under changing conditions. While various control approaches can be used, PID controllers are favored for their simplicity and long-standing effectiveness in practical applications. Properly tuned PID controllers allow hydrofoil vessels to respond to varying environmental conditions, such as waves or velocity changes, ensuring stability and performance. Thus, both CFD and control algorithms are critical to the development of high-performing hydrofoil vessels, with CFD guiding the design and control systems managing operational dynamics.

2.1 Hydrofoils

A hydrofoil is a specialized lifting surface designed to operate underwater, similar to the way an airfoil functions in air. It is typically a wing-like structure that generates lift when a craft moves through water, enabling it to lift the hull above the water's surface at higher speeds. Hydrofoils are employed in various watercraft, which are often referred to as "hydrofoils" due to the unique technology they use.

2. Background

As a hydrofoil-equipped vessel accelerates, the hydrofoils produce lift that gradually raises the hull above the water. This reduction in hull contact with the water decreases hydrodynamic drag, which is a major resistance force acting on the vessel. As a result, the hydrofoil enables the craft to achieve higher speeds with improved fuel efficiency compared to traditional hull designs. By reducing drag, hydrofoils make it possible for boats to glide over the surface as seen in Figure 2.1 [23], similar to how aircraft soar through the air, thus allowing vessels to reach greater speeds and operate more efficiently, particularly in rough sea conditions. The hydrofoil's ability to minimize drag and improve performance makes it an essential feature in high-speed maritime transportation and recreational boating.

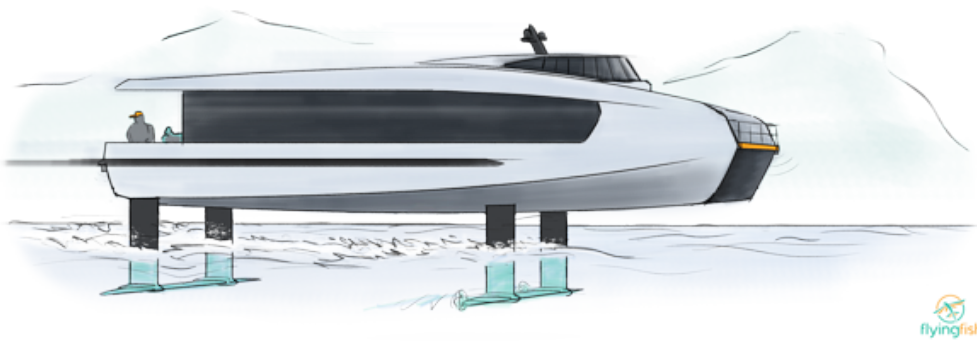


Figure 2.1: Foiling Craft

2.1.1 Role of Hydrofoils in Marine Craft

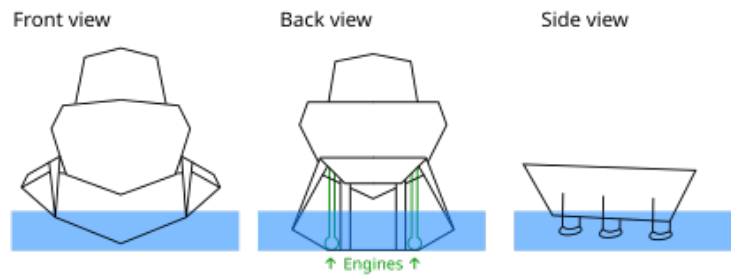
Hydrofoils serve as a critical component in marine vessels, enabling the hull to rise above the water at sufficient speeds. By reducing the hull's contact with water, hydrofoils significantly decrease drag forces and improve fuel efficiency. This enables the vessel to achieve higher speeds with reduced power requirements. Furthermore, hydrofoils enhance the seakeeping ability of vessels by minimizing wave resistance and mitigating the impact of rough seas. The integration of hydrofoils into marine craft is a revolutionary step, providing smoother, faster, and more efficient waterborne transport.

2.1.2 Foil Configuration

Early hydrofoils typically featured V-shaped designs, often referred to as "surface-piercing" foils. In these configurations, parts of the hydrofoil's V-shape extend above the water's surface when the vessel is in action [13] as depicted in Figure 2.2. This design allows the hydrofoil to generate lift as it moves through the water, but it is also subject to wave action, which can cause instability and discomfort for the vessel's occupants.

Hydrofoil types

Surface-piercing (SP) hydrofoils (usually U-shaped)



Fully submerged (FS) hydrofoils (usually shaped as inverted T or as V)

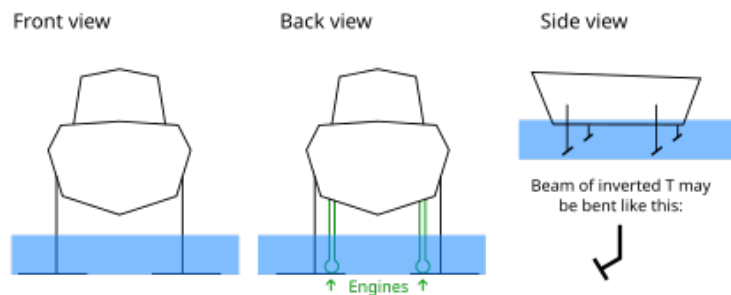


Figure 2.2: Hydrofoil Types

In contrast, modern hydrofoils often use fully submerged inverted T-shaped foils. These hydrofoils are entirely beneath the water's surface, making them less susceptible to the effects of waves. As a result, they provide greater stability in rough seas, contributing to a smoother and more comfortable experience for both crew and passengers. However, this submerged design does come with its own set of challenges. Unlike surface-piercing foils, fully submerged hydrofoils are not self-stabilizing. To maintain optimal performance and prevent the hydrofoil from losing lift or stability, the angle of attack of the foil must be continuously adjusted in response to changing conditions. The angle of attack (AoA) is defined as the angle between the *chord line* of an airfoil (such as a wing or hydrofoil) and the *oncoming flow velocity* (freestream velocity) as shown in Figure 2.3 [35].

The standard sign convention for the angle of attack follows these principles. A positive angle of attack ($\text{AoA} > 0$) occurs when the leading edge of the foil is tilted upward relative to the incoming flow, typically generating a positive lift. In contrast, a negative angle of attack ($\text{AoA} < 0$) means the leading edge is tilted downward, often producing negative lift or downforce. When the angle of attack is zero ($\text{AoA} = 0$), the chord line of the foil is exactly aligned with the freestream velocity, resulting in no lift, except in cambered foils where the lift can still be generated due to asymmetry.

In our scope, we assume the use of a fully submerged hydrofoil design. However, as the project progresses, we plan to explore variations in the distance between the hydrofoil and the free surface, particularly in relation to wave conditions. This aspect will be examined in greater detail during the later stages of the project,

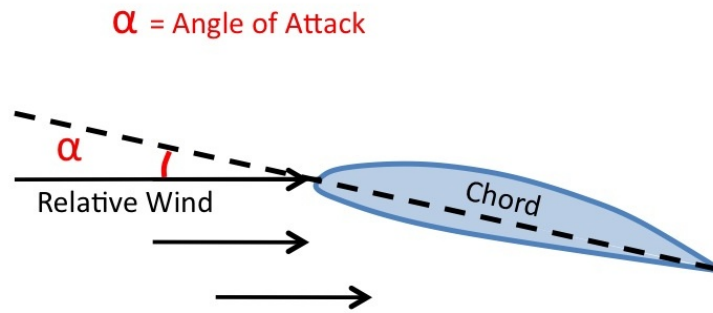


Figure 2.3: Angle of Attack (AoA)

as the dynamic interaction between the hydrofoil and the free surface becomes more significant.

2.2 Literature Review

The literature on hydrofoil craft highlights significant advancements in understanding their hydrodynamics, control systems, and performance optimization. Hydrofoil vessels achieve efficiency by reducing drag through the lift generated by submerged foils, which minimizes the wetted surface area. This lift mechanism enables the vessel to "fly" above the water's surface, reducing drag and increasing speed. However, ensuring stability and precise control, especially under variable sea conditions, remains a critical challenge. The dynamic behavior of hydrofoils is influenced by numerous factors such as wave height, current velocity, and environmental conditions, which complicates the task of maintaining the desired ride height, pitch, and overall vessel stability. This has driven the development of advanced control strategies, such as PID controllers, adaptive control systems, and the integration of CFD simulations. Hydrofoil boats, which are equipped with submerged foils that elevate the hull above the water surface, have been extensively studied due to their ability to reduce hydrodynamic drag and enhance operational efficiency [10]. These vessels achieve superior performance by minimizing the wetted surface area, thus reducing resistance and increasing speed. However, maintaining stability and control during operation, especially in dynamic sea conditions, remains a significant challenge. This requires sophisticated control systems, such as Nonlinear Model Predictive Control (NMPC) controllers, and advanced computational modeling techniques.

Early research into hydrofoil dynamics, as described by Grogono et al. [2], emphasized the importance of optimizing the lift-to-drag ratio for effective performance. Subsequent studies, such as those by Welaya and Abdulmotaleb [1] and Heppel [22], further expanded these principles, demonstrating that higher cruising altitudes can reduce hydrodynamic resistance but also introduce instability due to increased sensitivity to wave impacts and environmental variations. To counteract these effects, the development of active control systems became nec-

essary to dynamically adjust the vessel's lift and pitch, ensuring stable operation. CFD has become a key tool in analyzing hydrofoil performance and refining control strategies. Filippas and Belibassakis [9] studied the oscillatory behavior of hydrofoils under wave influences, highlighting the importance of incorporating hydrodynamic interactions into control system design. The coupling of CFD with control systems has been further advanced in studies such as that of Sun et al. [43], which introduced a CFD-based virtual test method for ventilation systems. Although their work focused on mechanical ventilation, it provided valuable insights into integrating CFD simulations with control components like PID controllers, actuators, and Variable Air Volume (VAV) dampers. The use of virtual sensors to address non-uniform stratifications and enhance control reliability has also influenced hydrofoil vessel applications, where accurate adjustments are needed to compensate for dynamic water conditions.

Bai and Kim [7] made significant contributions by investigating the control of hydrofoil vertical motion, incorporating variations in lift forces caused by orbital fluid particle motion as a time-varying disturbance in coupled heave and pitch equations. Their implementation of control strategies to reduce vertical oscillations proved essential for maintaining a stable lift-to-drag ratio. This work underscored the importance of adaptive and robust control systems capable of responding to changing conditions such as wave impacts and fluctuating speeds, laying the groundwork for more advanced active control methods.

Choi et al. [12] further advanced this field with their study of a closed-loop adaptive flow control system for managing flow separation over an NACA 64A210 airfoil. Their system, which employed piezoelectrically driven synthetic jet actuators and pressure sensors, coupled an adaptive inverse control system with extremum-seeking algorithms to regulate the pressure gradient. Compared to traditional PID systems, this adaptive method showed superior performance in tracking pressure gradients and improving aerodynamic characteristics such as lift-to-drag ratios, indicating the potential for advanced control systems in hydrofoil vessel applications.

Strömqvist [40] addressed the challenges of scaling control systems from model-scale to full-scale hydrofoil vessels. This work emphasized the importance of understanding scaling effects to ensure that control systems developed for model-scale testing can be effectively applied to larger vessels. The integration of computational and experimental methodologies is essential to bridging this gap and advancing the practical application of hydrofoil control systems.

The integration of CFD simulations with self-propulsion and maneuvering simulations has become a prominent research topic in naval architecture and marine engineering. Researchers such as Kwon et al. [26] have utilized CFD to investigate self-propulsion points, underscoring the importance of accurate control systems for replicating real-world conditions. These studies focus on estimating propulsion characteristics using robust models that incorporate parameters such as initial propeller speeds and trial-and-error methods for tuning PID controllers. Notably, this research employed a Ziegler-Nichols method for determining PID constants, showing consistency across different tuning methods.

A significant gap in the literature is the lack of uniform guidelines for establish-

ing control parameters in CFD-based self-propulsion simulations under variable conditions. Previous works highlighted the computational efficiency required for real-time simulation integration, especially with high-fidelity CFD models that demand time-step optimization, as demonstrated in the time-step sensitivity observed in turning simulations. Although progress has been made with dynamic simulations for turning behavior in various marine environments, these techniques are not yet fully operationalized for commercial use.

Future advancements in CFD applications for self-propulsion could benefit from refining time discretization methods and improving real-time integration with PID controllers to simulate complex maneuvering scenarios, such as turning in turbulent conditions. Enhancing simulation accuracy and computational efficiency is crucial for developing practical guidelines that can be applied across various marine crafts and scales.

The coupling of CFD with control strategies offers promising solutions for addressing the dynamic challenges of hydrofoil operations. By combining real-time simulations with robust control algorithms, researchers can develop systems that optimize lift, regulate ride height, and stabilize pitch, even under fluctuating sea conditions. CFD has emerged as a cornerstone for analyzing hydrofoil performance, providing critical insights into the hydrodynamic forces acting on the vessel, and enabling the refinement of control strategies for better performance. These simulations can account for complex phenomena like wave-induced oscillations, turbulence, and interactions between multiple appendages. Researchers have emphasized the importance of incorporating these environmental factors into CFD models to ensure realistic and accurate predictions, which, when coupled with control systems like PID controllers, allow for testing and optimizing performance in controlled settings before practical implementation.

Despite these advancements, several challenges remain. Current models often fail to fully account for real-time environmental variability, such as wave interference, turbulence, and changing water conditions, which limits their practical applicability in real-world scenarios. Furthermore, the high computational demands of high-fidelity CFD simulations pose challenges for their real-time integration with control systems, preventing widespread operational use. Another limitation is the limited understanding of multi-appendage dynamics. Many studies focus on simplified hydrofoil configurations with a single foil or a small set of appendages, but fail to address the complexities arising from interactions between multiple lifting surfaces under dynamic conditions.

Future research in hydrofoil control should prioritize improving the adaptability of control systems to respond to real-time environmental changes. Incorporating machine learning and adaptive algorithms into control systems could enhance predictive adjustments and real-time optimization, making them more capable of adjusting to sudden fluctuations in sea conditions. The computational efficiency of CFD simulations should also be improved, possibly through the use of lumped parameter models or more efficient numerical methods, to facilitate their integration into real-time control applications. Additionally, more in-depth studies on multi-appendage coordination and the interactions between multiple foils, wings, or other appendages should be conducted. Such studies would contribute

to a more accurate understanding of the hydrodynamic forces involved, and help develop robust and scalable control strategies for complex hydrofoil systems.

2.3 Principles of Hydrofoil Hydrodynamics

Hydrofoils and airfoils generate lift based on similar fluid dynamics principles, despite differences in the properties of the fluids involved, such as viscosity, density, and compressibility. A hydrofoil operates by smoothly moving through water, redirecting the flow downward. This deflection of flow results in an upward force on the foil, as described by Euler's equations. The pressure difference between the upper and lower surfaces of the foil arises from this deflection, with lower pressure on the top surface and higher pressure on the bottom. This pressure differential is accompanied by a velocity difference, as explained by Bernoulli's principle, leading to a higher velocity on the bottom surface compared to the top surface.

In the case of a hydrofoil boat, the lift produced by the foil serves to elevate the vessel's hull above the water. By reducing the contact area of the hull with the water, drag is minimized, which in turn allows the vessel to achieve higher speeds. As the boat accelerates, the lift force increases and eventually balances the weight of the craft, achieving equilibrium. Once the hull is raised above the water, the resistance from waves and other drag forces acting on the hull are significantly reduced. As a result, turbulence and drag are concentrated on the much smaller surface area of the hydrofoil, leading to a substantial increase in speed and efficiency.

2.3.1 Lift and Drag Forces

The primary forces acting on a hydrofoil are lift and drag.

Lift: Lift is the upward force generated by the hydrofoil as it moves through the water. This force arises due to the pressure difference between the upper and lower surfaces of the foil. As the hydrofoil deflects the water flow downward, according to Bernoulli's principle, the velocity of the flow increases above the foil and decreases below it. This velocity difference creates a pressure difference, with higher pressure on the bottom surface and lower pressure on the top, resulting in an upward force. The lift generated depends on factors like the shape of the foil, its angle of attack, and the speed of the fluid (water) passing over it.

The lift force L is given by the equation (2.1) [4]:

$$L = \frac{1}{2} C_L \rho v^2 A \quad (2.1)$$

Drag: Drag is the resisting force experienced by the hydrofoil as it moves through the water. It arises due to the frictional forces between the water and the surface of the hydrofoil, as well as the pressure drag caused by the changes in the flow velocity around the foil. Drag is a critical factor for hydrofoil efficiency, as it opposes the motion of the craft. Minimizing drag is essential for improving speed and fuel efficiency in hydrofoil-based vessels.

The drag force D is given by the equation (2.2) [4]:

$$D = \frac{1}{2}C_D\rho v^2A \quad (2.2)$$

Where:

- L = Lift force (N)
- D = Drag force (N)
- C_L = Lift coefficient (dimensionless)
- C_D = Drag coefficient (dimensionless)
- ρ = Density of the fluid (kg/m³)
- v = Velocity of the fluid relative to the hydrofoil (m/s)
- A = Exposed area of the hydrofoil (m²)

Lift-to-Drag Ratio: The lift-to-drag ratio is a performance indicator of how efficient a hydrofoil is:

$$\frac{L}{D} = \frac{C_L}{C_D} \quad (2.3)$$

This ratio indicates how much lift is generated for a given amount of drag. A higher ratio is more desirable for better efficiency.

2.3.2 Flow Regimes and Boundary Layers

The flow around a hydrofoil can be categorized into different regimes based on factors such as Reynolds number, angle of attack and velocity. These regimes influence how the fluid behaves around the foil and affect the generation of lift and drag as shown in Figure 2.4 [9].

Flow Regimes:

- **Laminar Flow:** In laminar flow, the water moves smoothly over the surface of the hydrofoil with minimal turbulence. This type of flow occurs at lower Reynolds numbers, where viscous forces dominate. Laminar flow is desirable at the leading edge of the hydrofoil as it helps minimize drag.
- **Turbulent Flow:** At higher velocities or angles of attack, the flow around the hydrofoil may transition to turbulent flow. In this regime, the flow becomes chaotic and irregular, resulting in higher drag but more efficient lift generation. Turbulent flow is common in the wake and near the trailing edge of the foil.
- **Transitional Flow:** Transitional flow takes place in the intermediate stage between laminar and turbulent flow, where the flow fluctuates between smooth and chaotic patterns. The transition between these flow regimes is crucial in hydrofoil design, as it can affect performance.

Boundary Layers: The boundary layer is a thin layer of water that develops along the surface of a hydrofoil, where the flow velocity transitions from zero at the surface (due to the no-slip condition) to the free-stream velocity. The behaviour of the boundary layer plays a significant role in both lift and drag production.

- **Laminar Boundary Layer:** In this region, the flow is smooth, and the velocity gradient is small. While Laminar boundary layers produce minimal

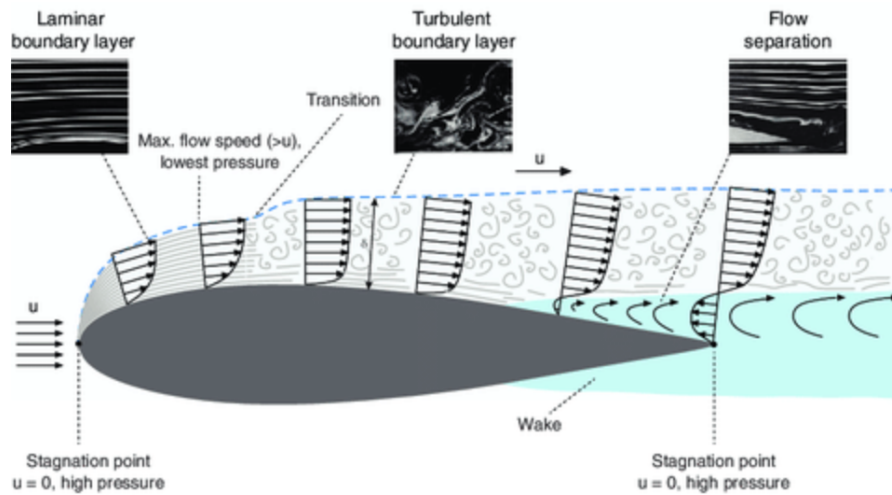


Figure 2.4: Flow Regime and Boundary Layer

skin friction, they are prone to separation, which can reduce lift and lead to turbulent wake formation.

- **Turbulent Boundary Layer:** As the flow becomes turbulent, the boundary layer thickens. However, the turbulent flow's chaotic eddies assist in keeping it adhered to the hydrofoil's surface, thereby delaying separation. Although turbulent boundary layers contribute to increased drag, they improve the overall stability and lift generation.

2.4 Dynamics and Unsteady Hydrodynamic Phenomena in Hydrofoil Vessels

Hydrofoil vessels rely on submerged foils to generate lift. While this innovative design offers significant advantages, it also introduces several challenges related to stability, control, and interaction with the surrounding fluid. The dynamic behavior of hydrofoils is shaped by complex fluid-structure interactions and unsteady flow phenomena. For example, the relationship between angle of attack and lift forms a characteristic loop pattern that reveals the unsteady aerodynamic behavior of the foil. Additionally, variable inlet flow conditions, such as wave-induced oscillations, further complicate the control and performance of these vessels. This section explores the critical dynamics of hydrofoil vessels, emphasizing their role in marine applications, stability and control challenges, and the influence of unsteady hydrodynamic phenomena on their efficiency and operational reliability.

2.4.1 Stability and Control Challenges

While hydrofoils offer numerous advantages, their operation presents unique challenges, particularly in maintaining stability and control.

Surface-piercing hydrofoils, characterized by their V-shaped design, provide inherent self-stabilization. As portions of the foil rise above the water, they naturally adjust their lift, reducing the need for active control. However, this configuration is susceptible to wave effects, which can compromise stability and passenger comfort. In contrast, fully submerged inverted T-shaped hydrofoils, widely adopted in modern designs, are less influenced by wave action and offer superior stability. However, these hydrofoils are not inherently self-stabilizing and require active control mechanisms to maintain their angle of attack (AoA) and performance. Advanced sensors, computer algorithms, and movable surfaces are employed to dynamically adjust the hydrofoil's AoA, ensuring optimal lift and stability under varying conditions.

The inclusion of unsteadiness introduces an additional layer of complexity to the stability and control of hydrofoils. Wave-induced variations in the free surface, along with the orbital velocities they generate toward the depth, create fluctuating flow conditions that challenge the hydrofoil's ability to maintain consistent lift and vessel stability. While fully submerged hydrofoils are less impacted by direct wave effects, these dynamic conditions necessitate robust and adaptive control strategies to ensure optimal performance.

For the scope of this project, fully submerged hydrofoils are considered the baseline configuration. Their interaction with the free surface under wave-induced variations in height and frequency will be analyzed in later stages to assess their influence on stability and control, focusing on how these complexities can be addressed through improved hydrodynamic understanding and control system design.

2.4.2 Unsteady Hydrodynamic Phenomena

The hydrodynamic behavior of hydrofoils under unsteady conditions plays an important role in their performance and operational reliability. Unsteady phenomena such as oscillations in lift and drag, as well as flow separation and reattachment, directly affect the vessel's stability and maneuverability.

2.4.2.1 Lift Coefficient Loop (C_l vs. AoA)

One of the critical unsteady phenomena observed in hydrofoil dynamics is the hysteresis effect in the relationship between the lift coefficient (C_l) and the AoA. This effect occurs when the AoA oscillates, leading to two distinct paths in the C_l vs. AoA curve: one during the increase of AoA and another when it decreases. This elliptical or loop-like behavior is inherently nonlinear, as it cannot be represented by a single, straightforward function $C_l = f(\text{AoA})$. In our simulations, we conducted a steady-state case using a sine function for the AoA. The oscillation started from 6 degrees, increased to 12 degrees, decreased to 0 degrees, and then returned to 6 degrees. The simulation showed the expected elliptical behavior seen in Figure 2.5, confirming the presence of hysteresis in the system. The loop observed in the C_l vs. AoA curve reflects the underlying physical processes such as flow separation and reattachment, which cause the lift to behave differently during increasing and decreasing AoA.

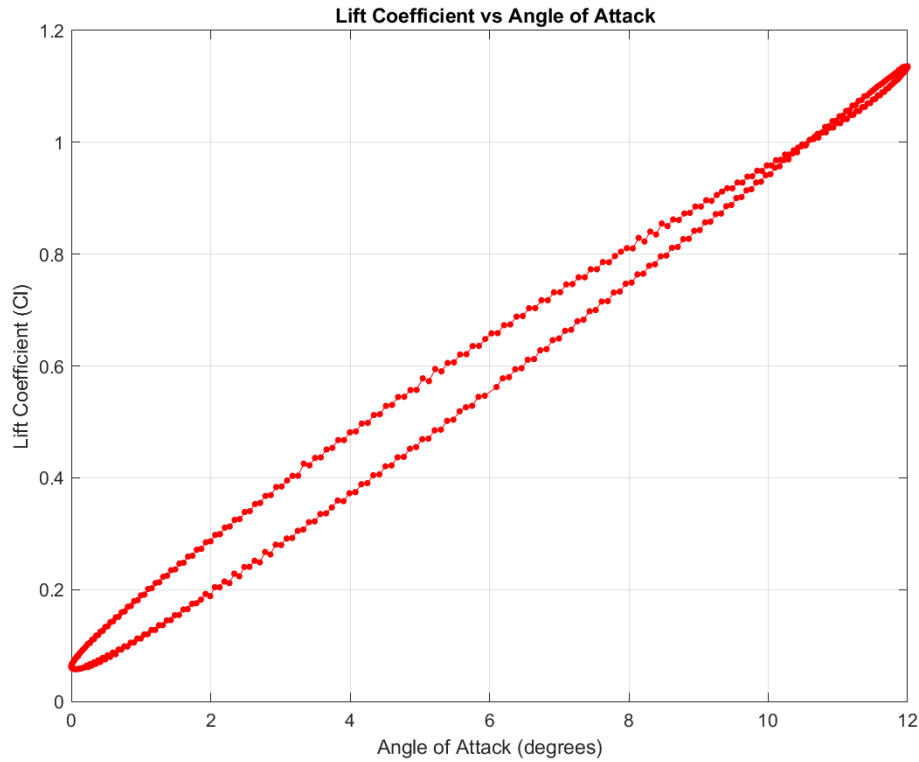


Figure 2.5: C_l vs AoA

However, the implementation of hysteresis is not the responsibility of the PID controller; instead, it should be incorporated into the process model. The PID controller, which assumes a linear relationship between output and input, struggles to perform effectively in the presence of hysteresis. Therefore, in this thesis, we simplify the system by assuming a linear relationship between the lift coefficient (C_l) and the angle of attack (AoA). This assumption allows the use of a PID controller for lift control, despite the inherent non-linearities of the system, including hysteresis.

To address the limitations of PID control, more advanced control techniques may be considered in future work, but for the purposes of this project, a linear approximation is employed to simplify the control process and ensure manageable system dynamics.

2.4.2.2 Impact of Unsteady Inlet Conditions

Hydrofoils often operate in environments with unsteady inlet conditions, such as waves or variable currents. These conditions introduce fluctuations in flow velocity and direction, which impact the forces acting on the foil and pose challenges to maintaining stable performance.

In this study, unsteady inlet conditions are initially simulated in a 2D single-phase setup using a sinusoidal velocity profile to replicate wave-like behavior. This simplified approach allows for a focused analysis of the hydrofoil's response to fluctuating velocities, helping refine control strategies for maintaining a constant lift force. These preliminary simulations serve as a stepping stone for transitioning

to more complex multiphase and full 3D setups.

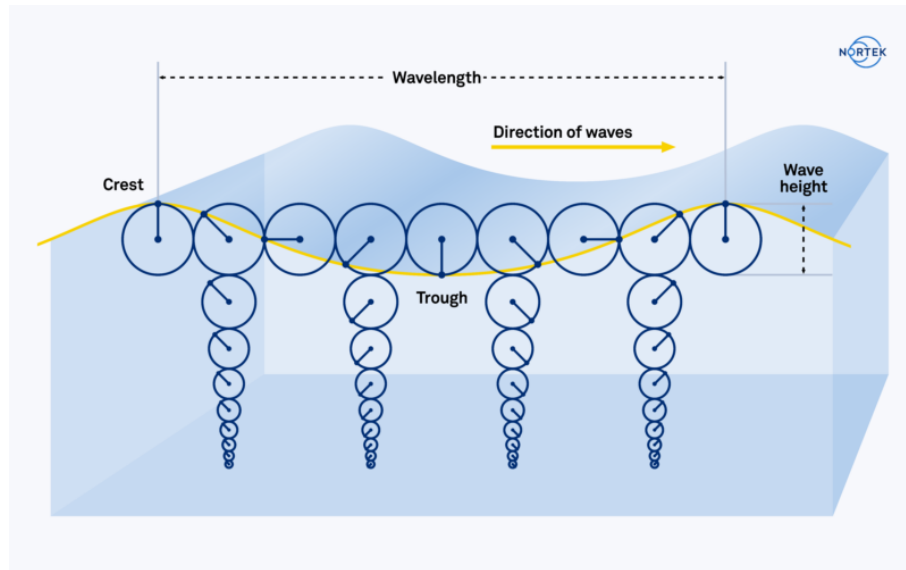


Figure 2.6: Wave Orbital Velocities

Wave patterns play a significant role in unsteady hydrodynamics, as they induce orbital velocities in the fluid as shown in Figure 2.6 [28]. These orbital velocities, which decrease with depth, create a dynamic flow field around the hydrofoil. Near the surface, these oscillatory motions are more pronounced, exerting fluctuating forces on the foil. By simulating such wave-induced velocity profiles, this study seeks to capture the essential dynamics of wave-foil interactions and assess their impact on lift generation and vessel stability.

Through these simulations, insights are gained into the influence of unsteady conditions, informing the design of robust control systems capable of mitigating the effects of wave-induced instabilities.

2.5 Introduction to Computational Fluid Dynamics

Computational Fluid Dynamics (CFD) is a specialized field within fluid mechanics that employs numerical techniques and algorithms to solve and analyze fluid flow problems. It provides the capability to simulate the behavior of liquids and gases interacting with surfaces under different boundary conditions, offering valuable insights into flow characteristics without relying on extensive physical testing.

The core equations governing CFD are based on the Navier-Stokes equations, which describe the conservation of mass, momentum, and energy within a fluid. These equations, when combined with turbulence models, boundary conditions, and numerical methods for discretization, facilitate the solution of complex fluid dynamics problems.

2.5.1 Governing Equations: Navier-Stokes Equations

Navier-Stokes equations (NS equations) are the fundamental governing equations for all known fluid flow problems. CFD simulations rely on solving the NS equations numerically to predict fluid flow behavior. CFD models and techniques are used to simulate and study turbulence within the context of the NS equations in simulations. The viscosity and pressure gradient effects are taken into consideration while describing the conservation of momentum for a fluid flow using the NS equations. The complicated and chaotic nature of fluid motion is reflected in turbulence, which the NS equations attempt to capture.

The NS equations control the flow dynamics in areas where turbulence occurs by taking into account the pressure, viscosity, and acceleration forces acting on the fluid.

Definition: The motion of fluid substances is described by a system of partial differential equations known as the Navier-Stokes equations (NS equations) [4]. They are essential to the study of fluid mechanics and are used in a variety of scientific and engineering contexts to forecast and comprehend fluid flow patterns. The NS equations are derived from the fundamental principles of conservation of mass, momentum, and energy in fluid flow.

- **Conservation of Mass:** According to the principle of conservation of mass, the net mass flux entering or exiting a control volume equals the rate of change in mass within the volume.
- **Conservation of Momentum:** According to the principle of conservation of momentum, the pressure, viscous, and external body forces acting on a control volume add up to the time rate of change of momentum within the volume.
- **Conservation of Energy:** To account for thermal energy transfer and work produced by outside forces, the principle of energy conservation stipulates that the rate of change of total energy within a control volume is equal to the net energy flux entering or leaving the volume.

Oftentimes, the energy equation is not explicitly solved in fluid simulations because the momentum and continuity equations are more important in many fluid flow scenarios than the thermal energy equation, particularly when incompressible flows or flows with small temperature gradients are involved. Without explicitly calculating the energy equation, simplified assumptions like constant temperature or adiabatic conditions are frequently adequate to describe the key fluid flow behavior.

Fluid simulations become more computationally complicated and demand more time and resources when the energy equation is solved explicitly. The additional computing expense of calculating the energy equation might not be warranted for many engineering applications, particularly if the main goal is to comprehend fluid flow dynamics rather than heat transfer.

2.5.1.1 Mathematical Formulation

Mathematically, the NS equations are expressed in vector form for each component of velocity (u , v , w) in a three-dimensional Cartesian coordinate system.

2. Background

They are composed of the continuity equation, which controls mass conservation, and three momentum equations, one for each direction.

The pressure gradient, viscous forces (diffusion), external body forces, and the time rate of change of velocity (convective acceleration) are all described by the momentum equations [4].

The Navier-Stokes equations [4] in vector form for each component of velocity (u, v, w) in a three-dimensional Cartesian coordinate system are:

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} + v \frac{\partial u}{\partial y} + w \frac{\partial u}{\partial z} = -\frac{1}{\rho} \frac{\partial p}{\partial x} + \nu \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2} \right) \quad (2.4)$$

$$\frac{\partial v}{\partial t} + u \frac{\partial v}{\partial x} + v \frac{\partial v}{\partial y} + w \frac{\partial v}{\partial z} = -\frac{1}{\rho} \frac{\partial p}{\partial y} + \nu \left(\frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 v}{\partial y^2} + \frac{\partial^2 v}{\partial z^2} \right) \quad (2.5)$$

$$\frac{\partial w}{\partial t} + u \frac{\partial w}{\partial x} + v \frac{\partial w}{\partial y} + w \frac{\partial w}{\partial z} = -\frac{1}{\rho} \frac{\partial p}{\partial z} + \nu \left(\frac{\partial^2 w}{\partial x^2} + \frac{\partial^2 w}{\partial y^2} + \frac{\partial^2 w}{\partial z^2} \right) \quad (2.6)$$

It can be simplified to the following:

$$\frac{\partial u}{\partial t} + \nabla \cdot (u\vec{u}) = -\frac{1}{\rho} \frac{\partial p}{\partial x} + \nu \nabla^2 u \quad (2.7)$$

$$\frac{\partial v}{\partial t} + \nabla \cdot (v\vec{u}) = -\frac{1}{\rho} \frac{\partial p}{\partial y} + \nu \nabla^2 v \quad (2.8)$$

$$\frac{\partial w}{\partial t} + \nabla \cdot (w\vec{u}) = -\frac{1}{\rho} \frac{\partial p}{\partial z} + \nu \nabla^2 w \quad (2.9)$$

where:

- $\frac{\partial u}{\partial t}, \frac{\partial v}{\partial t}, \frac{\partial w}{\partial t}$: Time rate of change of velocity components u, v , and w respectively.
- $\nabla \cdot (u\vec{u}), \nabla \cdot (v\vec{u}), \nabla \cdot (w\vec{u})$: Convective acceleration terms representing the divergence of the velocity vector field $\vec{u}$ multiplied by the velocity components u, v , and w respectively.
- $\frac{1}{\rho} \frac{\partial p}{\partial x}, \frac{1}{\rho} \frac{\partial p}{\partial y}, \frac{1}{\rho} \frac{\partial p}{\partial z}$: Pressure gradient terms representing the force exerted on the fluid in the x, y , and z directions respectively.
- $\nu \nabla^2 u, \nu \nabla^2 v, \nu \nabla^2 w$: Viscous diffusion terms representing the Laplacian of velocity components u, v , and w , respectively, scaled by the kinematic viscosity ν .

The Reynolds Transport Theorem, which connects the fluxes entering or exiting a control volume to the time rate of change of a fluid property (such as mass, momentum, or energy), is used to derive the NS equations. The theorem takes into consideration both the flow-induced advection (transport) of fluid characteristics and the diffusion (dispersion) caused by molecular processes like viscosity.

The momentum equations are coupled with the continuity equation in a three-dimensional Cartesian coordinate system, and is given by:

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \vec{u}) = 0 \quad (2.10)$$

Assuming the flow to be incompressible:

$$\nabla \cdot \vec{u} = 0 \quad (2.11)$$

where:

- $\nabla \cdot \vec{u}$: Divergence of the velocity vector field $\vec{u}$.

The continuity equation intended for incompressible flow states that the divergence of the velocity field is zero, implying that the flow is divergence-free within the domain.

2.5.1.2 Solution of NS equation

Analytical solutions of the NS equations are frequently difficult, particularly for intricate flow geometries and boundary conditions. NS equations can be discretized and numerically solved using many numerical methods, including finite volume, finite element, and finite difference methods. Numerical solutions of the NS equations are used in CFD simulations to forecast fluid flow patterns and provide technical insights.

Additionally, boundary conditions determine how the flow behaves at the edges of the computational domain; they are crucial for solving the NS equations. In addition to conditions for free surfaces and interfaces, boundary conditions also include velocity, pressure, and other flow parameters stated at the inlet, outlet, and solid boundaries.

2.5.2 Steps involved in CFD Simulation

The CFD process involves several steps [39] [11], including:

1. **Problem Definition:** Clearly state the goals and parameters of the CFD analysis. Determine the geometry, fluid characteristics, and boundary conditions of the fluid flow problem that needs to be simulated. In order to assess the outcomes of the simulation, specify the desired outputs and performance criteria.
2. **Geometry Creation and Optimization:** Using CAD software, create or import the fluid domain's geometry. Reduce complexity and ready the geometry for meshing by clearing out extraneous details and resolving intricate features. Make sure there are no mistakes or holes in the geometry.
3. **Mesh Generation:** Create a discrete fluid domain by generating a computational mesh with small control volumes or elements. Select the right kind of mesh (hybrid, unstructured, or structured) depending on the shape and

flow properties. To effectively capture significant flow features, fine-tune the mesh in the areas of interest. Verify the authenticity and convergence of the mesh by performing quality checks.

4. **Boundary Conditions Setup:** Define boundary conditions at the computational domain's walls, inlet, outlet, and other boundaries. Describe the temperature, pressure, and velocity of the flow at the inlet boundary. Establish boundary conditions for solid walls, such as suitable thermal boundary conditions for heat transfer simulations and no-slip criteria for viscous flows.
5. **Physics Modeling:** Determine which turbulence modeling technique—such as RANS, LES, or DNS—is best for your application's needs and flow regime. Choose turbulence models ($k-\epsilon$, $k-\omega$, etc) and provide model coefficients or constants. Depending on the application, include extra physics models for combustion, heat transport, multi-phase flow, and other pertinent phenomena.
6. **Solver Selection and Configuration:** Based on the flow regime, geometry complexity, and required solution accuracy, select an appropriate solver algorithm. Set the convergence criteria, time-stepping parameters, and solution algorithms for the solver. For discretizing the governing equations, use suitable numerical techniques (e.g., finite volume, finite element, finite difference).
7. **Simulation Execution:** Set the solver parameters and initial conditions for the simulation. Run the simulation for the allotted time or until the convergence requirements are satisfied. Track the development of the solution, taking note of the numerical solution's stability and convergence tendencies. To cut down on simulation time, parallelize the computation if needed.
8. **Post-Processing and Analysis:** Utilizing post-processing tools, extract and display the findings of the simulation. Examine the temperature, pressure, velocity, and turbulence parameters of the flow field. Calculate performance measures and contrast the outcomes of the simulation with analytical or experimental data. To understand and present the results of simulations, create graphs, contours, vectors, streamlines, and other visualizations.
9. **Validation and Verification:** Verify the simulation results by contrasting them with data from experiments or reference solutions. To evaluate numerical accuracy and resilience, conduct sensitivity analysis and grid convergence experiments. Make sure the solution configurations and simulation setup appropriately reflect the physical problem by verifying them.

2.5.3 Relevance of CFD in Hydrofoil Analysis

Understanding the hydrodynamic performance of hydrofoils is essential to optimizing their design and ensuring operational stability. CFD provides a powerful tool to study these aspects in detail, offering several advantages such as:

- **Lift and Drag Prediction:** CFD enables accurate prediction of lift and drag forces across a range of operating conditions. This is particularly useful for optimizing the hydrofoil shape for performance efficiency.
- **Visualization of Flow Fields:** It allows visualization of flow separation, boundary layers, and pressure distributions around hydrofoils, helping in design improvements.
- **Unsteady Phenomena Analysis:** CFD helps in studying time-dependent phenomena such as oscillatory motion, cavitation, and unsteady inlet flows, which are critical in real-world applications.
- **Multiphase Flow Simulation:** Advanced CFD models can simulate air-water interactions, accounting for free-surface effects and cavitation, enhancing the accuracy of hydrodynamic predictions.

2.5.4 Multiphase Flow and its Simulation in CFD

In fluid mechanics, multiphase flow refers to the simultaneous movement of substances in two or more distinct thermodynamic phases [45]. This type of flow is integral to a wide range of engineering and industrial processes, such as cavitating pumps, turbines, paper production, and plastic manufacturing. Multiphase flow also plays a significant role in various natural phenomena.

The phases in a multiphase flow system can either consist of a single substance (e.g., water and its vapor) or involve multiple components (e.g., oil and water). A phase is considered continuous when it occupies a fully connected space, whereas a dispersed phase exists in discrete, unconnected regions. The continuous phase is typically a gas or liquid, while the dispersed phase may consist of solids, liquids, or gases.

Two primary flow structures are observed in multiphase systems: dispersed and separated flows as shown in Figure 2.7 [30]. Dispersed flows are characterized by finite particles, droplets, or bubbles dispersed within a continuous medium. In contrast, separated flows involve two or more continuous fluid streams divided by distinct interfaces.

Understanding and modeling multiphase flow dynamics are crucial for optimizing engineering systems and addressing challenges in various applications, from industrial machinery to environmental systems.

2.5.4.1 Importance of Multiphase Flow in Hydrofoil Simulations

Multiphase flow simulations play a critical role in accurately predicting and optimizing the performance of hydrofoil simulations. Multiphase flow is relevant for hydrofoil simulations because there is a significant difference between the behavior of the hydrofoil when it is submerged in water versus when it is above the water. In this work, the focus is solely on the hydrofoil itself, and the vessel is considered out of scope. Therefore, the importance of multiphase flow is primarily centered around the hydrofoil's interaction with the water surface. The following key aspects highlight their importance:

2. Background

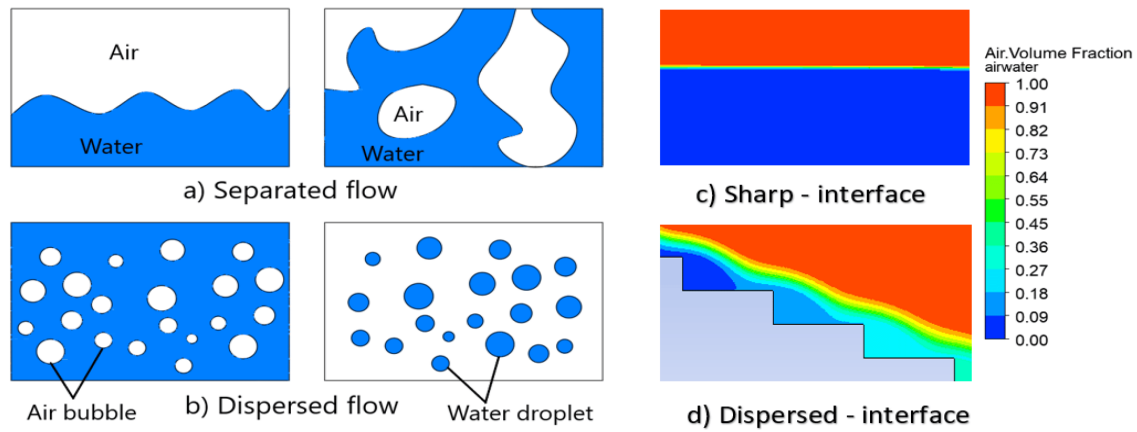


Figure 2.7: Multiphase: Dispersed and Separated Flow

- **Wave Effects:** Multiphase simulations enable the detailed study of interactions between fluids at their interfaces, such as the behavior of free surfaces under wave action. This includes capturing wave-induced orbital velocities that propagate below the surface, influencing the forces acting on submerged objects. These insights are crucial for accurately modeling fluid dynamics in environments influenced by wave motion.
- **Cavitation Modeling:** The ability to simulate cavitation—where vapor cavities form in low-pressure regions—plays a significant role in understanding fluid-structure interactions. Cavitation can alter performance characteristics and contribute to structural degradation, making its accurate representation vital in engineering applications involving high-speed or low-pressure flows.
- **Spray and Splashing Phenomena:** Multiphase simulations also account for spray and splashing phenomena, which occur when liquids interact dynamically with solid surfaces or other fluid phases. These effects are critical to understanding energy dissipation, momentum transfer, and fluid distribution in systems subjected to high velocities or turbulent conditions.

2.5.4.2 Relevance to Hydrofoil Simulation

Incorporating wave effects, cavitation, and spray dynamics into simulations provides a detailed understanding of how hydrofoils interact with variable flow conditions, including waves and turbulent water surfaces. By evaluating the impact of cavitation and spray on hydrofoil efficiency and durability, simulations help identify potential performance limitations and material challenges. This ensures that hydrofoil designs are not only optimized for performance but also capable of withstanding demanding marine environments.

2.5.4.3 Simulation Approach

Multiphase flow is typically modeled using the Navier-Stokes equations combined with additional equations for phase interactions. The Volume of Fluid

(VOF) method is commonly used to track the interface between phases.

Volume of Fluid (VOF) Technique: This technique is a widely used numerical method in CFD to simulate and analyze multiphase flows, where two or more immiscible fluids occupy the same computational domain. It is particularly effective for tracking and capturing sharp interfaces between different fluid phases, such as the air-water interface in free surface flows. This method was introduced by Hirt and Nichols in 1981 and has since become a standard approach for solving problems involving phase interactions, such as those found in hydrodynamics, cavitation, and wave dynamics.

The VOF model belongs to the family of interface-capturing methods, which predict the distribution and movement of the interface between immiscible phases. The key idea behind the VOF technique is that the computational grid must be sufficiently fine to resolve the interface between phases, allowing for an accurate representation of phase distribution and the dynamics of the interface as seen in Figure 2.8 [21]. In the VOF model, the distribution of phases and the position of

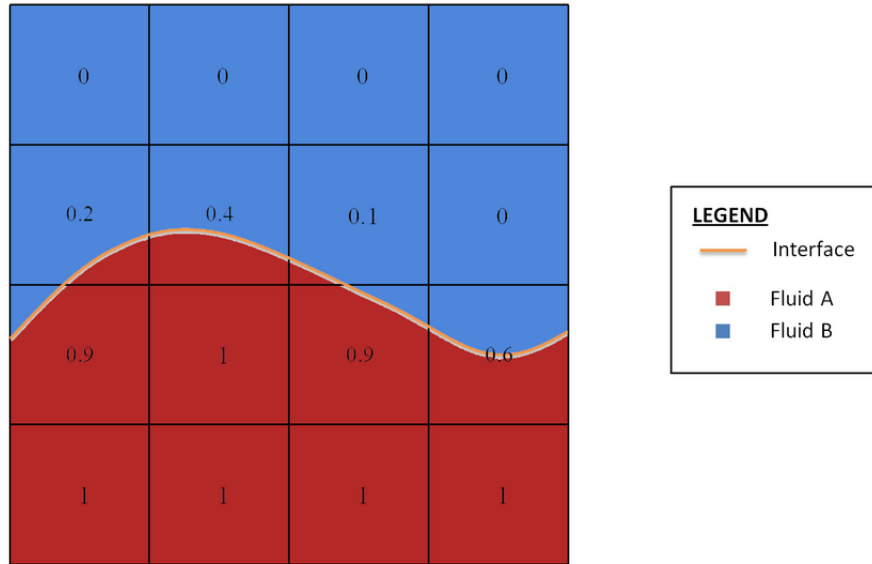


Figure 2.8: Volume of Fluid (VOF)

the interface are described by the fields of phase volume fraction α_i . The volume fraction of phase i is defined as:

$$\alpha_i = \frac{V_i}{V}$$

where V_i is the volume of phase i in the cell and V is the volume of the cell. The volume fractions of all phases in a cell must sum up to one:

$$\sum_{i=1}^N \alpha_i = 1$$

where N is the total number of phases.

Depending on the value of the volume fraction, the presence of different phases or fluids in a cell can be distinguished:

- $\alpha_i = 0$ — the cell is completely void of phase i .
- $\alpha_i = 1$ — the cell is completely filled with phase i .
- $0 < \alpha_i < 1$ — values between the two limits indicate the presence of an interface between phases.

2.5.4.4 Challenges in Multiphase Flow Simulations

Multiphase flow simulations involve significant complexities that make accurate modeling challenging. Below are the key difficulties encountered in such simulations [41]:

- **High Computational Demand:** Resolving interactions between multiple phases requires high spatial and temporal resolution. Capturing fine details, such as phase interfaces and small-scale phenomena like cavitation bubbles, significantly increases computational costs, including memory usage, processing power, and simulation time.
- **Numerical Instabilities:** Discontinuities in fluid properties, such as density and viscosity, at the phase interface can cause numerical instabilities. These may manifest as oscillations or errors in mass conservation. Ensuring stability requires careful selection of numerical schemes, adaptive time-stepping, and robust interface-capturing techniques like the Volume of Fluid (VOF) method.
- **Accurate Representation of Wave Dynamics:** Surface waves introduce orbital velocities beneath the water surface, which decay with depth but still influence phase interactions. High-resolution spatial grids and precise time integration are essential to capture realistic wave behaviors and avoid inaccuracies in force predictions and interaction patterns.
- **Cavitation Phenomena:** Cavitation involves the formation and collapse of vapor cavities in low-pressure regions, generating steep pressure gradients and shock waves. Accurately modeling these effects, including phase changes and thermal influences, is crucial to predict cavitation's impact on performance and structural integrity.
- **Coupling of Physical Phenomena:** Multiphase flows involve interactions among turbulence, surface tension, and heat or mass transfer. The challenge is capturing these coupled effects across different spatial and temporal scales. Specialized approaches and empirical adjustments are often required to model these interactions effectively.
- **Scalability of Simulations:** Ensuring scalability on high-performance computing platforms is critical as the complexity of multiphase simulations increases. Efficient parallelization and optimization of algorithms are necessary for large-scale industrial and environmental applications.

By addressing these challenges, multiphase flow simulations can achieve higher accuracy and reliability and tackle complex problems in various fields, including marine applications, energy systems, and fluid-structure interactions.

2.5.5 High-Performance Computing (HPC) in CFD

HPC refers to the use of supercomputers and parallel processing techniques to perform complex calculations at high speeds [38] [15]. Unlike traditional computing systems, HPC clusters consist of interconnected nodes that work together to solve computationally intensive tasks, significantly reducing simulation time.

2.5.5.1 Role of HPC in CFD Simulations

HPC is integral to CFD simulations [18], especially for scenarios involving:

- **Large Computational Domains:** Simulations of industrial-scale or environmental fluid flows, such as hydrofoils in open seas, require significant computational resources to resolve detailed phenomena across expansive domains.
- **High-Resolution Grids:** HPC allows for finer mesh resolutions, enabling accurate modeling of small-scale features like turbulent eddies, boundary layers, and phase interfaces in multiphase flows.
- **Transient Simulations:** Time-dependent problems, including wave interactions and vortex shedding, demand extensive computational effort, which HPC makes feasible.

2.5.5.2 Parallel Computing Techniques in CFD

High-Performance Computing (HPC) [24] utilizes parallel computing techniques to handle the computational demands of large-scale CFD problems, optimizing resource utilization and accelerating simulations. One key approach is domain decomposition, where the computational domain is divided into smaller subdomains, each processed independently by different processors, reducing individual workload and enabling faster computation. Communication between subdomains ensures boundary consistency, though data exchange at subdomain boundaries can be challenging. Task parallelism divides computational tasks, such as solving momentum equations or turbulence models, across processors to improve throughput and resource allocation. However, task dependencies can cause delays and require precise scheduling. Load balancing ensures even distribution of computational work among processors, minimizing idle time and optimizing performance, particularly for adaptive meshes. Dynamic load balancing, though effective, adds computational overhead. Lastly, communication strategies, including Message Passing Interface (MPI) and shared memory, enable efficient data exchange between processors, essential for maintaining consistency and accuracy in large simulations, especially in multiphase or turbulent flow scenarios.

2.5.5.3 Workflow of CFD Simulation Using HPC

The integration of HPC into CFD simulations follows a structured workflow to optimize computational resources. The process begins with problem definition

and preprocessing, where the geometry is created using CAD tools, and the domain is discretized into a mesh, ensuring adequate resolution in critical regions. The physics, governing equations, and boundary conditions are set up. Next, HPC resource allocation involves selecting an appropriate cluster, assigning processors, and determining the parallelization strategy, such as domain decomposition or task parallelism. During solver execution, the mesh, physics setup, and initial conditions are loaded into the CFD solver, and the simulation is run in parallel, with monitoring of convergence and key quantities. In data management and output generation, transient data is stored, and the HPC system's scalability is assessed. Post-processing and visualization involves using tools to analyze flow fields and validate results against experimental data, providing insights for design optimization. Finally, iteration and optimization refine the design based on results, with adjustments to solver settings and HPC resource allocation to enhance efficiency in future simulations.

2.6 Control Algorithms for Hydrofoil Stability and Performance

In this section, the role of control algorithms in maintaining hydrofoil stability and optimizing performance is discussed. These algorithms manage critical parameters such as angle of attack and ride height, adapting dynamically to environmental changes like waves and currents.

2.6.1 Introduction to Control Systems

A control system is a mechanism that manages, directs, or regulates the behavior of devices or processes through control loops [42]. These systems can vary in complexity, from a simple thermostat regulating a home heating system to sophisticated industrial systems that oversee large-scale machinery and processes. The design and development of control systems are achieved through the principles and methodologies of control engineering.

In systems requiring continuous regulation, feedback controllers are employed to automate the control of processes or operations. A feedback control system functions by comparing the current value of a process variable (PV), which represents the system's actual state, with a predefined desired value known as the setpoint (SP). The system calculates the difference between the PV and SP and generates a control signal based on this discrepancy. The process variable is adjusted by applying the control signal to the system, ensuring it aligns with the desired set point.

In hydrofoil systems, effective control of various parameters is crucial for ensuring stability, efficiency, and optimal performance under a wide range of operating conditions. One of the most important aspects is the control of the angle of attack (AoA), which directly affects the lift generated by the hydrofoil. Precise adjustments are necessary to avoid excessive drag or stalling. Maintaining a consistent

ride height, or the depth at which the hydrofoil operates, is essential for stability and to prevent cavitation, which can compromise efficiency and lead to structural issues. Stabilization of pitch and roll is another critical factor, as it enhances safety and comfort by countering destabilizing forces. Hydrodynamic drag must also be minimized through real-time optimization of the AoA and foil configuration to improve energy efficiency.

Cavitation, a phenomenon that can affect performance and structural integrity, is mitigated by operating within suitable conditions. Other key aspects include yaw control to maintain the desired course, as well as dynamic responses to wave-induced disturbances to ensure vessel stability. Additionally, features like retractable foils allow for safe operation in shallow waters or during docking. Speed regulation, integration with propulsion systems, and structural load management further enhance the hydrofoil's overall functionality, preventing excessive forces and ensuring durability. Collectively, these control strategies enable hydrofoil systems to adapt to challenging marine environments while maintaining performance and efficiency.

2.6.1.1 Key Components of a Control System

Key components of a control system include:

- **Sensors:** These devices measure various system parameters such as lift, ride height or trim angle. They provide real-time data, which is essential for accurate and responsive control. Reliable sensors are crucial to ensure the system can react promptly to changes.
- **Controller:** The controller processes the data collected by the sensors and determines the necessary corrective actions. It uses algorithms, such as PID controllers or more advanced methods like Model Predictive Control (MPC), to minimize deviations from the desired state of the system.
- **Actuators:** Actuators execute the corrective actions decided by the controller. In hydrofoil systems, this typically involves adjusting the angle of attack or altering the hydrofoil's orientation through movable surfaces to maintain optimal performance.
- **Plant:** The plant refers to the physical system being controlled. In the context of hydrofoils, the plant is the hydrofoil structure itself, which responds to external forces such as water flow and adjusts according to the commands from the controller.
- **Feedback Loop:** The feedback loop continuously monitors the system's current state and provides this information to the controller. This closed-loop structure allows the system to make real-time adjustments, ensuring stable and adaptive behavior under varying conditions.

2.6.1.2 Types of Control Systems

Control systems can be categorized into two types:

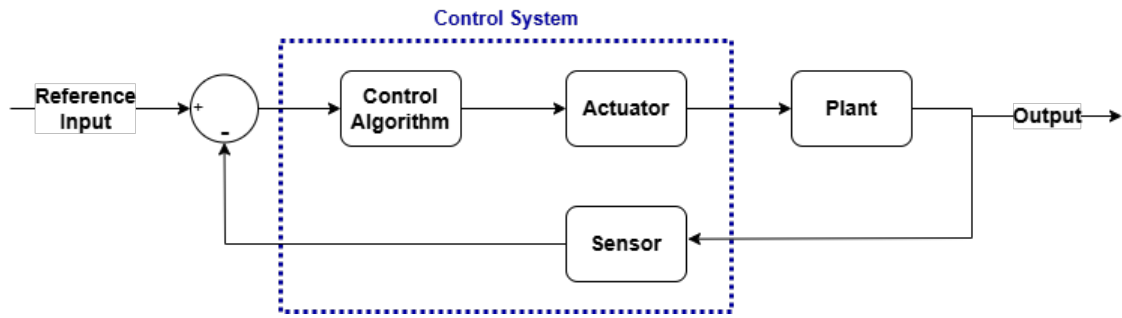


Figure 2.9: Block diagram of a Closed Control System

- **Open-Loop Control:** This type of control system operates without feedback, executing predefined actions regardless of the system's actual state. While open-loop systems are simple and cost-effective, they lack adaptability and are unsuitable for dynamic or unpredictable environments.
- **Closed-Loop Control:** Unlike open-loop systems, closed-loop control incorporates feedback to continuously monitor and adjust the system's state. This enables greater accuracy and robustness, making closed-loop systems ideal for hydrofoil applications, where environmental conditions can fluctuate and the system must adapt accordingly.

In Figure 2.9, a block diagram of a plant and controller is presented, illustrating a closed-loop control system and its interaction with the plant. Each block in the diagram corresponds to a specific component of the system. The solid lines with arrows depict the flow of signals between these components, demonstrating how information is transferred through the control loop, enabling continuous feedback and adjustments to maintain the desired system behavior.

2.6.2 PID Controller and its Role in Hydrofoil Applications

The PID controller is a fundamental control algorithm widely employed in various engineering and industrial applications [25]. It provides a simple yet effective method for achieving desired system behavior by continuously adjusting the control input based on the error between a setpoint and the system's current state. The PID controller operates through three distinct components: proportional, integral, and derivative, each addressing specific aspects of the system's performance.

The control law for a PID controller can be expressed as:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (2.12)$$

where:

- $u(t)$ is the control output at time t .
- $e(t)$ is the error, defined as $e(t) = r(t) - y(t)$, where $r(t)$ is the desired setpoint and $y(t)$ is the measured system output.

- K_p is the proportional gain, which determines the contribution of the proportional term.
- K_i is the integral gain, which scales the contribution of the integral term.
- K_d is the derivative gain, which scales the contribution of the derivative term.

Each term in the control law has a distinct role:

- **Proportional Term:** The proportional term, $K_p e(t)$, produces an output that is directly proportional to the current error, providing immediate corrective action.
- **Integral Term:** The integral term, $K_i \int_0^t e(\tau) d\tau$, accumulates the error over time, helping to eliminate steady-state errors by addressing cumulative discrepancies.
- **Derivative Term:** The derivative term, $K_d \frac{de(t)}{dt}$, predicts future error trends by considering the rate of change of the error, thereby introducing damping to reduce overshooting and oscillations.

The combined effect of these components enables the PID controller to provide robust and adaptive control, making it suitable for a wide range of dynamic systems. A depiction of PID controller is shown in Figure 2.10 [14]

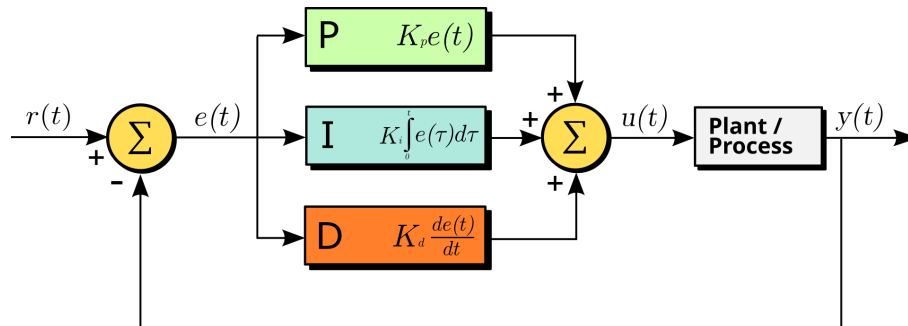


Figure 2.10: Block diagram of a PID Controller

2.6.2.1 Tuning of PID Controller

PID tuning [32] refers to the process of adjusting the proportional (K_p), integral (K_i), and derivative (K_d) gains of a PID controller to optimize its performance in a given system. These three parameters govern the controller's response to the error between the desired setpoint and the actual process variable. The proportional term addresses the current error, the integral term accounts for accumulated past errors, and the derivative term anticipates future errors based on their rate of change. Proper tuning of these gains is critical for ensuring system stability, minimizing errors, and achieving desired performance, particularly in dynamic systems where disturbances or changes in conditions may occur. Without adequate tuning, the controller may result in instability, overshooting, or sluggish response, negatively affecting the system's efficiency and safety. Additionally, correctly tuned PID controllers help minimize steady-state errors, reduce

oscillations, and enhance the system's ability to adapt to varying inputs, thus optimizing control performance and extending the lifespan of system components. Therefore, PID tuning is essential for balancing the trade-offs between response speed, stability, and accuracy, ensuring the system operates effectively across a wide range of conditions. Common methods include:

- **Manual Tuning:** The controller gains are adjusted manually through trial and error based on system response, often starting with the proportional gain and gradually adding the integral and derivative components.
- **Ziegler-Nichols Method:** Another empirical method where the system is placed in proportional control only, and the critical gain (where the system oscillates) is used to determine appropriate PID parameters.
- **Cohen-Coon Method:** A more systematic method used for processes with time delays, which provides formulas to calculate PID parameters based on the system's open-loop response.
- **Optimization Techniques:** Techniques like genetic algorithms, particle swarm optimization, or other heuristic methods are used to automatically tune PID parameters for complex systems.
- **Software Tools:** Many modern control systems offer built-in PID tuning tools that automatically calculate optimal parameters based on the system's dynamics.

2.6.2.2 Limitations of PID Controllers

PID controllers are widely used in engineering applications due to their simplicity and effectiveness. However, they are not universally applicable and exhibit several limitations, which are outlined below:

- **Inability to Handle Nonlinear Systems:** PID controllers are inherently linear control algorithms, making them unsuitable for highly nonlinear processes. Real-world systems with complex or varying dynamics often require advanced control methods to achieve optimal performance.
- **Lack of Adaptability:** PID controllers rely on fixed gains for proportional, integral, and derivative terms, determined through tuning. Changes in system dynamics over time, due to factors like environmental variations or component aging, necessitate frequent retuning or the addition of adaptive control mechanisms.
- **Sensitivity to Noise:** The derivative term amplifies rapid changes in the error signal, making the controller highly sensitive to measurement noise. This can result in erratic control actions and degraded performance in noisy environments.
- **Challenges with Time-Delay Systems:** PID controllers are not well-suited for systems with significant time delays between inputs and outputs. Without predictive capabilities, the controller may introduce oscillations or instability in such systems.

- **Performance in Multivariable Systems:** Processes involving multiple interacting variables cannot be effectively controlled with a single PID controller. Multivariable control systems often require advanced strategies, such as Model Predictive Control (MPC), to address interdependencies.
- **Difficulty in Achieving Optimal Tuning:** Proper tuning of PID parameters is crucial for effective performance. However, the tuning process can be complex and time-intensive, particularly for systems with intricate dynamics. Poorly tuned parameters may result in instability or unsatisfactory responses.

While PID controllers are effective for many applications, these limitations highlight the need to evaluate their suitability for specific systems. For more complex or dynamic scenarios, alternative or supplementary control strategies may be necessary to achieve desired performance.

2.6.3 Control System Performance Analysis and Evaluation

2.6.3.1 Performance Metrics of Control Systems

In relation to the control systems, performance metrics are crucial to evaluate how effectively a system achieves its desired objectives under varying operating conditions. For control applications, these metrics provide quantitative measures of stability, accuracy, and responsiveness, which are critical for ensuring optimal operation. The key performance metrics are described below, and their relationships are illustrated in Figure 2.11.

Response Time:

- Response time refers to how quickly a system reacts to changes, disturbances, or a shift in the desired setpoint.
- It includes key time-based indicators:
 - *Rise Time:* The time taken for the system to go to 90% from 10% of the desired setpoint.
 - *Settling Time:* The time taken for the system to stabilize within a specified tolerance around the setpoint.
- Faster response times are preferred; however, excessively fast responses may lead to overshoot or instability.

Steady-State Error:

- This metric quantifies the difference between the system's steady-state output and the desired setpoint.
- A low steady-state error indicates high precision and accuracy in achieving the control objective.
- For hydrofoil systems, maintaining minimal steady-state error is essential for achieving a consistent ride height or lift force.

Overshoot:

2. Background

- Overshoot represents the extent to which the system output exceeds the desired setpoint during transient conditions.
- High overshoot can result in undesirable behavior, such as structural stress.
- Minimizing overshoot improves system reliability and prevents energy inefficiency or damage to components.

Stability:

- Stability indicates the system's ability to maintain controlled behavior over time, even when subjected to environmental disturbances or parameter variations.
- A stable control system effectively prevents oscillations or divergence from the desired operating point.

These performance metrics also depicted in Figure 2.11 [17] collectively provide a comprehensive framework for assessing the effectiveness of control algorithms. In hydrofoil systems, achieving a balance among these metrics is critical to ensure operational reliability, energy efficiency, and overall system stability.

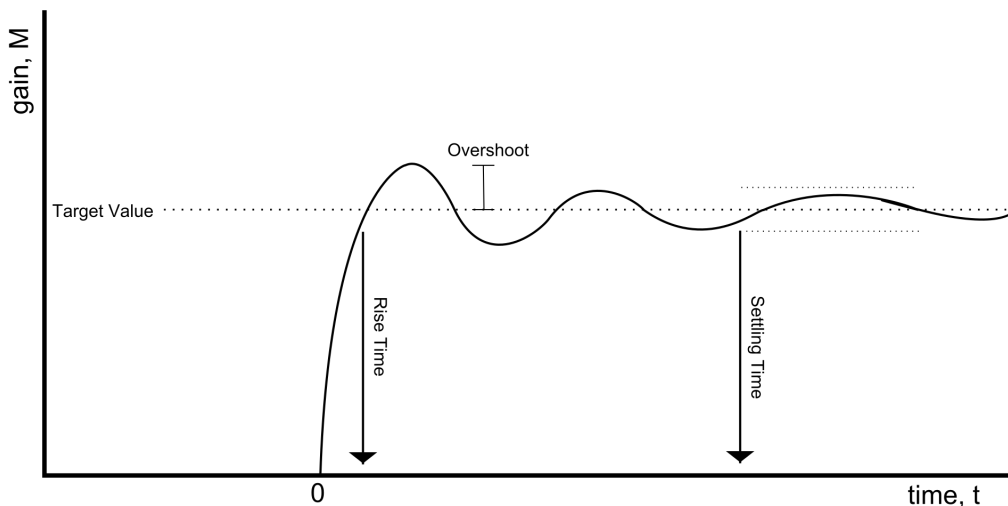


Figure 2.11: Performance Metrics of Control Systems

2.6.3.2 Evaluation Metrics of Control Systems

Evaluation metrics are essential for quantitatively assessing the performance and reliability of control systems in meeting their design objectives. In hydrofoil applications, these metrics provide critical insights into the effectiveness of control algorithms in maintaining stability, achieving the desired performance, and adapting to varying environmental conditions. The purpose and significance of each metric are summarized in Table 2.1, with the mathematical definitions described as follows:

Integral of Absolute Error (IAE): The Integral of Absolute Error (IAE) accumu-

lates the absolute values of the error over time:

$$\text{IAE} = \int_0^T |e(t)| dt \quad (2.13)$$

where $e(t)$ is the error at time t and T is the total time.

Integral of Squared Error (ISE): The Integral of Squared Error (ISE) computes the squared error over time:

$$\text{ISE} = \int_0^T e(t)^2 dt \quad (2.14)$$

Integral of Time-Weighted Absolute Error (ITAE): The Integral of Time-Weighted Absolute Error (ITAE) incorporates a time-weighting factor:

$$\text{ITAE} = \int_0^T t|e(t)| dt \quad (2.15)$$

Root Mean Square Error (RMSE): The Root Mean Square Error (RMSE) is a measure of the average magnitude of the error:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N e_i^2} \quad (2.16)$$

where N = the total number of samples or observations in the dataset

Table 2.1: Evaluation Metrics

Metric	Purpose	Significance
IAE	Evaluates the magnitude of error over time.	Smaller IAE values indicate better tracking of the desired setpoint.
ISE	Penalizes larger errors more due to squaring.	Useful when minimizing significant deviations is critical.
ITAE	Emphasizes errors occurring later in the process.	Ensures long-term stability and smooth system response.
RMSE	Quantifies the overall error magnitude.	Reflects the control system's accuracy over time.

Evaluation metrics gives a structured way for understanding and improving control system performance. By analyzing these metrics, engineers can fine-tune control algorithms, ensure reliable operation. These metrics are integral to iterative design and testing, enabling continuous optimization of the control system.

Performance metrics are distinct to the evaluation metrics but complementary aspects of assessing a control system's effectiveness. The following Table 2.2 [33] outlines the differences between these two categories of metrics:

Table 2.2: Differences Between Performance Metrics and Evaluation Metrics

Aspect	Performance Metrics	Evaluation Metrics
Focus	Measures specific system behavior under ideal or controlled conditions.	Assesses overall robustness and adaptability in real-world scenarios.
Examples	Settling time, overshoot, steady-state error.	Integral of Absolute Error (IAE), control effort, disturbance rejection.
Application Phase	Used primarily during the design and tuning phases to optimize system response.	Used to validate system performance under varying operating conditions.
Purpose	Ensures the system meets design objectives and achieves desired response characteristics.	Evaluates reliability, efficiency, and adaptability in dynamic environments.

3

Methodology

This chapter presents the systematic methodology adopted to achieve the project objectives, focusing on the control of the angle of attack of the hydrofoil to maintain the desired objective. The study employed STAR-CCM+, a powerful CFD software, to simulate the hydrodynamic behavior of the hydrofoil under progressively complex flow conditions. Initial simulations were performed under steady-flow inlet conditions to establish a baseline, followed by transient inlet conditions that introduced wave-like disturbances. To further enhance realism, a multiphase flow setup was implemented to replicate the interaction of the hydrofoil with water and air phases.

A PID controller was integrated into the CFD simulations using a custom Java Macro script in STAR-CCM+. The PID controller dynamically adjusted the hydrofoil's angle of attack based on real-time feedback either from the lift force generated or current ride height or angles, ensuring stability and performance in varying flow scenarios. Additionally, Simulink was employed in parallel to analyze the behavior of the PID controller in a controlled environment. The Simulink model was instrumental in simulating noise and disturbances, such as those induced by turbulence or wave motion.

This integrated approach allowed for the iterative tuning of PID parameters to optimize system performance, balancing stability, responsiveness, and noise mitigation. The combination of CFD simulations and Simulink-based control system analysis provided a holistic understanding of the hydrofoil's behavior and the effectiveness of the control strategy under both idealized and realistic operating conditions. This methodology lays a solid foundation for developing a robust and adaptive control system for hydrofoil applications.

3.1 CFD Simulation Framework

This section introduces the simulation setup used to model the hydrofoil's hydrodynamics in different flow conditions, starting with the steady state and progressing to more complex unsteady and multiphase flow conditions.

3.1.1 Geometry and Mesh Generation

The computational domain for the present study is designed to simulate the flow around a hydrofoil, ensuring an accurate representation of the fluid dynamics

while minimizing the influence of boundary conditions. The domain parameters and hydrofoil configuration are described as follows.

3.1.1.1 Hydrofoil Geometry

The hydrofoil employed is based on the NACA 0012 profile as depicted in Figure 3.1, which is a symmetric airfoil commonly utilized in hydrodynamic and aerodynamic studies due to its predictable and well-documented performance characteristics. The chord length of the hydrofoil is set to 0.2 m, enabling detailed analysis in a computationally efficient 2D setup.

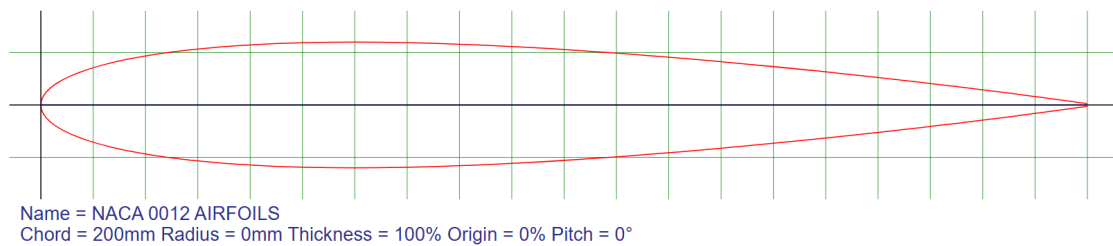


Figure 3.1: Hydrofoil Geometry - NACA 0012

3.1.1.2 Computational Domain

The computational domain is defined as a rectangular region measuring 6 m × 6 m, which ensures sufficient space for the flow to develop upstream and downstream of the hydrofoil. The hydrofoil is positioned 1 m from the inlet boundary and is centrally aligned in the vertical direction. This placement minimizes the influence of inlet conditions on the flow field around the hydrofoil and allows for the detailed capture of wake structures in the downstream region as shown in Figure 3.2. In the extended semi-2D computational domain, the outlet boundary was positioned further downstream to minimize the influence of flow reversal near the hydrofoil.

3.1.1.3 Meshing

The computational mesh is a vital component of the simulation setup, as it directly influences the accuracy and convergence of the numerical solution. During the initial stages of testing, a polygonal meshing as illustrated in Figure 3.3 approach was employed due to its inherent flexibility and ability to handle complex geometries efficiently.

As the project progressed, a more refined meshing strategy was adopted using the Trimmed Cell Mesher as shown in Figure 3.4 available in Simcenter STAR-CCM+ [36]. This meshing technique combines robustness, accuracy, and efficiency, making it well-suited for both simple and complex geometries. The key features of the Trimmed Cell Mesher include:

- **Predominantly Hexahedral Mesh:** The trimmed cell mesh consists of structured, hexahedral-dominated cells, which reduce numerical diffusion and

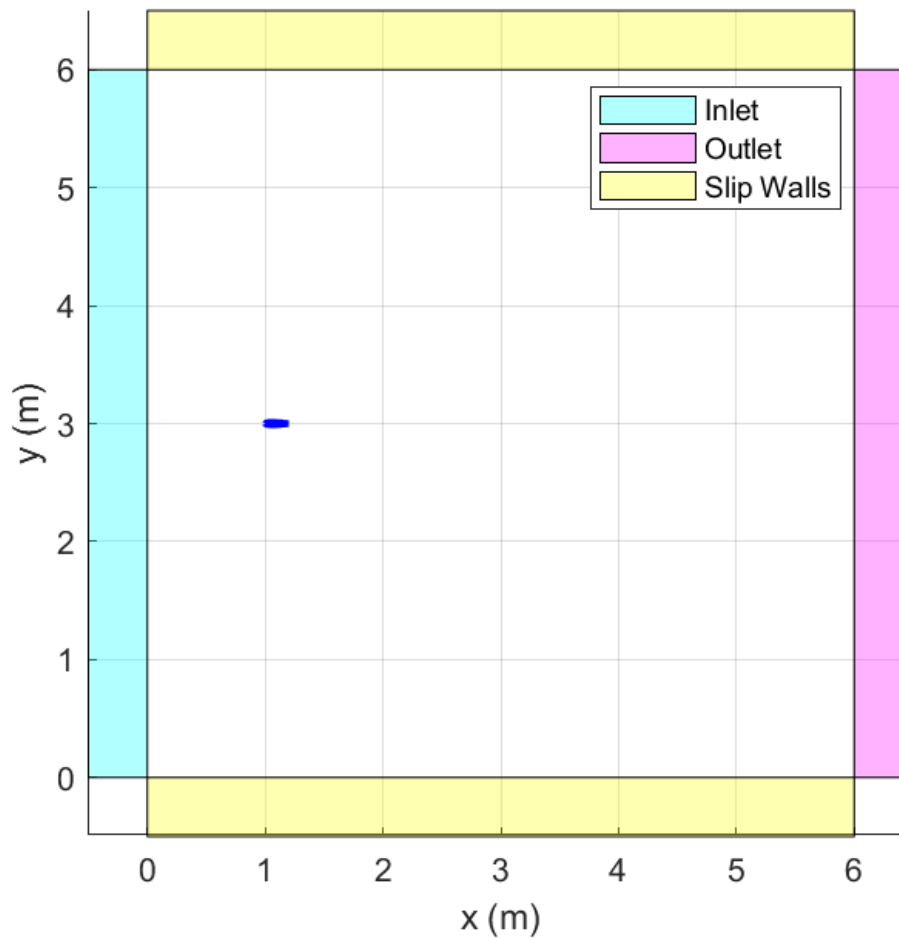


Figure 3.2: Computational domain with boundary conditions

ensure minimal cell skewness, thereby improving solution stability and accuracy.

- **User-Defined Refinement:** Mesh refinement is controlled based on the surface mesh size and user-defined refinement criteria, allowing for localized refinement in regions of interest, such as the hydrofoil's leading and trailing edges.
- **Surface Quality Independence:** The mesh quality is independent of the surface geometry's complexity, ensuring consistent cell quality even for intricate hydrofoil contours.
- **Directional Alignment:** The mesh aligns with a user-specified coordinate system, providing directional accuracy that enhances the resolution of flow features, such as boundary layers and wakes.

For the hydrofoil simulations, a base mesh size of 0.2 m was selected to have a balance between computational efficiency and the required accuracy. This mesh size provided adequate resolution for the flow around the hydrofoil while minimizing the computational resources required. To enhance accuracy in critical regions, the mesh was locally refined around areas such as the hydrofoil's leading and trailing edges, where rapid changes in velocity and pressure occur.

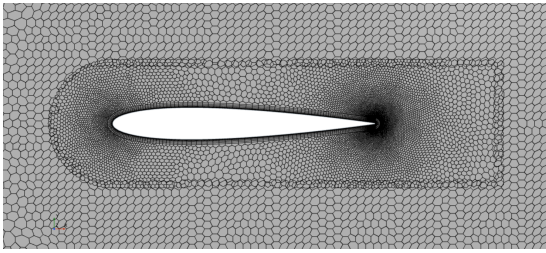


Figure 3.3: Example of polygonal mesh

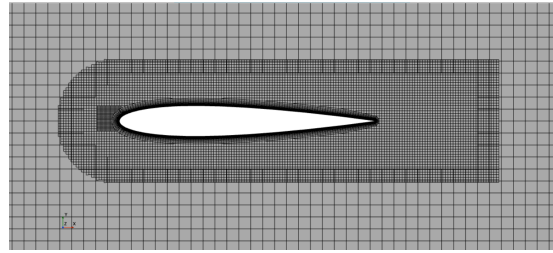


Figure 3.4: Example of trimmed cell mesh

Additionally, overset meshing techniques were employed to model the hydrofoil's motion through the fluid. This approach enabled high-resolution meshing in key areas around the hydrofoil while maintaining computational efficiency. In simulations involving waves or multiphase flow, the mesh was further refined in the free surface region. This refinement was essential to accurately capture the complex fluid dynamics at the interface between the fluid phases, particularly in the presence of wave interactions.

Downstream of the hydrofoil, the wake region is designed to extend sufficiently to allow for the capture of wake structures and turbulent flow features. This facilitates detailed analysis of the hydrodynamic forces and flow behavior influenced by the hydrofoil.

3.1.1.4 Boundary Conditions

The boundary conditions were specified to represent a realistic and physically accurate flow environment, as depicted in Figure 3.2.

- **Inlet:** Defined as a velocity inlet, allowing for the imposition of steady-flow or sinusoidal velocity profiles, depending on the simulation scenario.
- **Outlet:** Configured as a pressure outlet to facilitate smooth exit of the flow without any backflow effects.
- **Hydrofoil Surface:** Modeled as a no-slip wall, where the velocity at the wall is zero relative to the surface. This boundary condition captures the viscous effects and flow separation critical to the hydrofoil's hydrodynamic performance.
- **Top and Bottom Boundaries:** Defined as slip walls to approximate an infinite flow domain. This boundary condition permits tangential velocity components while constraining normal velocity to zero.
- **Front and Back Surfaces (in semi-2D simulations):** Treated as symmetry planes to represent an infinitely repeating spanwise flow, reducing computational demand while preserving the physical characteristics of the flow.

3.1.2 CFD Simulation Settings

In this section, the simulation setup is discussed, including solver configurations, temporal and spatial discretization, and turbulence modeling. These settings were systematically tailored for each stage of the study, beginning with a 2D steady-flow configuration, progressing to an unsteady single-phase flow setup, and then a semi-2D multiphase simulation. The goal is to ensure numerical stability, computational efficiency, and reliable results that accurately capture the hydrofoil's performance across diverse flow scenarios.

3.1.2.1 Solver Settings

The solver settings for the CFD simulations are carefully chosen to ensure numerical accuracy, stability, and computational efficiency for various flow conditions. The details are as follows:

- **Governing Equations and Turbulence Model:** The simulations are based on the incompressible Navier-Stokes equations, which govern the fluid flow. For turbulence modeling, the $k - \omega$ SST (Shear Stress Transport) model is employed. This model is selected for its capability to accurately predict flow separation and boundary layer behavior while effectively accounting for adverse pressure gradients, which are critical for analyzing hydrofoil performance.
- **Discretization Schemes:** Spatial discretization is performed using a second-order upwind scheme to minimize numerical diffusion and enhance solution accuracy, particularly in regions with steep gradients, such as the hydrofoil's leading edge.

Temporal discretization employs a first-order implicit scheme for all simulations. While this method provides lower accuracy compared to higher-order schemes, it ensures numerical stability when dealing with complex flow phenomena and multiphase interactions.

- **Time Step and Resolution:** The time step size is varied between 10^{-3} and 10^{-4} seconds, depending on the simulation setup:
 - For 2D simulations, larger time steps (10^{-3}) are used due to relatively lower computational demands.
 - For semi-2D and 3D setups, smaller time steps (10^{-4}) are required to resolve finer spatial and temporal details accurately.

This variation balances computational cost and solution accuracy.

- **Solver and Numerical Algorithm:** A segregated flow solver with an implicit time integration scheme is used for all simulations. This approach solves the momentum and pressure equations sequentially, ensuring computational efficiency and stability, particularly for incompressible flow problems.

Using the SIMPLE (Semi-Implicit Method for Pressure-Linked Equations) algorithm pressure-velocity coupling can be achieved. This iterative method

adjusts pressure and velocity fields to satisfy mass conservation, providing robust convergence in steady-flow and transient simulations.

3.1.2.2 Initial Testing: 2D Setup

Steady-Flow Simulation: Initial simulations are conducted in a 2D domain to verify the correctness of the hydrofoil geometry, boundary conditions, and solver settings. A constant inlet velocity of 10 m/s (approximately 19.4 knots) is applied to represent steady-flow conditions, allowing for a basic evaluation of the hydrofoil's lift and drag characteristics. This phase is crucial for identifying and addressing potential errors before introducing more complex flow scenarios.

3.1.2.3 Transition to Unsteady Inlet Velocity Conditions

Sinusoidal Inlet Velocity Profile: The next stage involves incorporating a boundary normal sinusoidal inlet velocity profile to mimic the unsteadiness induced by wave-like conditions. The velocity at the inlet is expressed as:

$$U_{\text{inlet}} = U_0 + A \sin(\omega t) \quad (3.1)$$

where $U_0 = 10$ m/s is the mean velocity, A is the amplitude of the velocity oscillation, and ω is the angular frequency. This setup captures the dynamic behavior of the hydrofoil under varying flow conditions, simulating a single-phase fluid environment.

The sinusoidal velocity function is implemented through a user-defined field function in STAR-CCM+ and assigned to the inlet boundary, allowing control of the oscillation parameters. These simulations test the hydrofoil's ability to maintain stability and performance in unsteady-flows.

3.1.2.4 Multiphase Flow Setup in a Semi-2D Domain

Transition to Multiphase Simulation: To model free surface interactions and wave effects, the setup transitions to a multiphase flow simulation. A semi-2D domain with a one-cell thickness is used to maintain computational feasibility while approximating 3D multiphase behavior. The Volume of Fluid (VOF) method is employed to capture the interface between water and air accurately. This one-cell thickness implementation enhances computational efficiency by significantly reducing memory usage and simulation time while preserving essential 3D flow characteristics. It also simplifies mesh generation, improves numerical stability, and allows for better resolution of the free surface, making it a practical approach for studying wave interactions.

Mesh Refinement in Critical Regions: Mesh refinement is applied near the free surface and the hydrofoil's vicinity to resolve complex interactions between phases. The domain dimensions and boundary conditions are adjusted to simulate realistic wave conditions, such as crest and trough interactions with the hydrofoil.

3.1.2.5 Flow Conditions and Wave Modeling

Before delving into the PID control system implementation, it is essential to first discuss the flow conditions and wave modeling used in the simulations. After introducing multiphase flow conditions to simulate more realistic operating environments, additional tests were conducted to assess the hydrofoil's performance under varying wave conditions.

Initially, tests were performed under flat wave conditions, similar to the steady-flow setup, where the flow is considered uniform, and there is no significant fluctuation in the inlet velocity. This setup served as a baseline for comparison with the more complex wave models.

Next, first-order waves were introduced to mimic simple sinusoidal wave patterns, characterized by a smooth oscillation. The velocity at the inlet was modeled as a sinusoidal function, representing periodic fluctuations in the flow, which are typical of low-amplitude waves that the hydrofoil may encounter during operation.

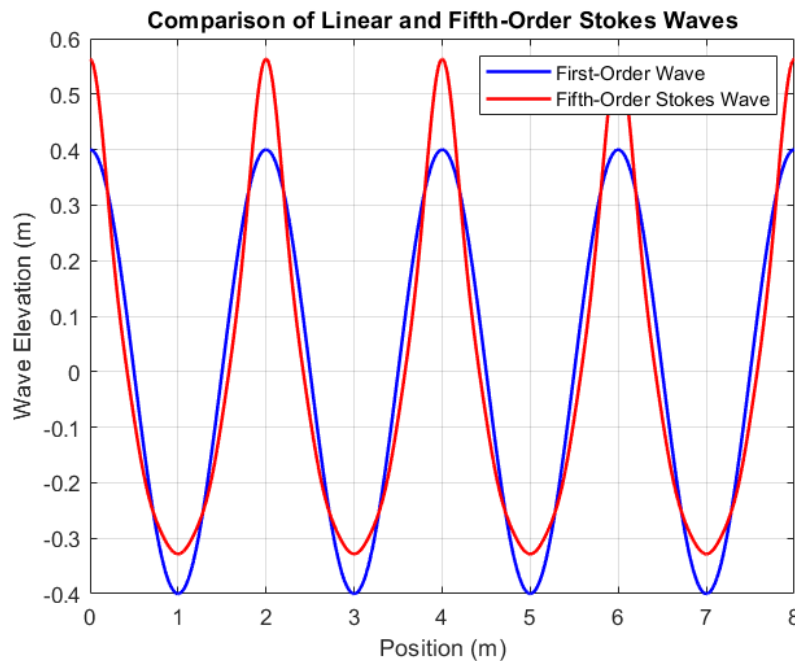


Figure 3.5: Comparison of first-order and fifth-order stokes waves

To further enhance the realism of the simulations, fifth-order waves based on Fenton's formulation [19] were incorporated to represent more complex and higher-amplitude wave conditions. These waves provide a more accurate description of the dynamic marine environment in which the hydrofoil operates, capturing the characteristics of turbulent and irregular wave patterns. An example comparison between first-order and fifth-order waves with a wavelength of 2 m and an amplitude of 0.4 m is illustrated in Figure 3.5.

These stages of wave modeling provided a stepwise approach, beginning with simple conditions and gradually increasing the complexity of the wave dynamics.

The following section will now describe the integration of the PID control system within this dynamic simulation environment.

3.1.2.6 Introduction to DFBI Motion

Dynamic Fluid-Body Interaction (DFBI) is a computational technique used to simulate the interaction between a moving body, such as a hydrofoil, and the surrounding fluid. Unlike static simulations, DFBI allows for the dynamic response of the body to hydrodynamic forces and moments, thus providing a more realistic and accurate representation of hydrofoil behavior under operational conditions. The DFBI motion model selection in StarCCM+ is depicted in Figure 3.6.

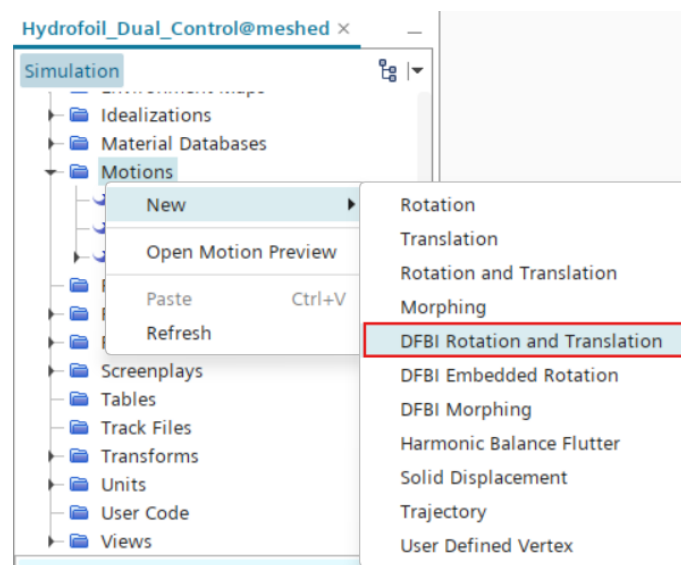


Figure 3.6: DFBI model

3.1.2.7 Implementation of DFBI in the Semi-2D Setup

In the semi-2D setup, the hydrofoil was modeled with a single cell thickness, limiting the degrees of freedom that could be implemented in the simulation. The DFBI model was integrated to allow motion in heave (vertical direction) about the Y-axis and trim (pitch angle) around the Z-axis, but roll angle control could not be implemented due to the restriction of a single cell thickness in the model. This limitation constrained the ability to control roll angles independently, focusing the control strategy on maintaining a steady vertical ride height and trim angle.

The DFBI model was coupled with the fluid dynamics solver, enabling the hydrofoil to respond to hydrodynamic forces while interacting with the surrounding fluid. The primary challenge in this setup was to maintain control over the vertical motion and trim while accounting for wave-induced disturbances, which were incorporated into the simulation via a time-varying velocity inlet.

Incorporating DFBI motion into the semi-2D setup has laid the foundation for more advanced simulations that will provide a deeper understanding of hydro-

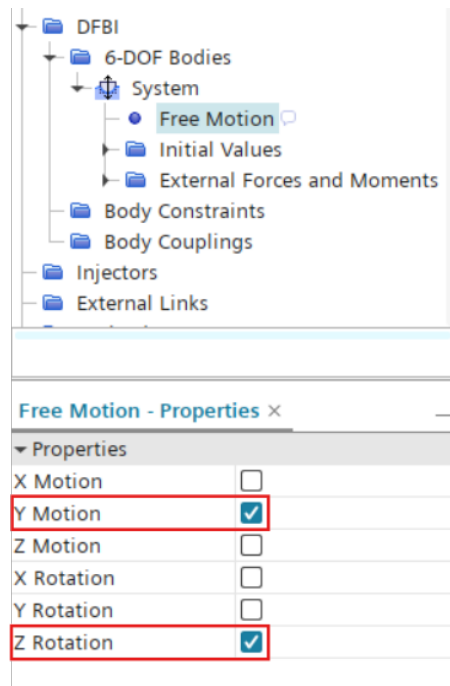


Figure 3.7: DFBI motion selection

foil behavior under realistic conditions. The next steps will involve improving the model's complexity and incorporating more detailed control strategies to enhance the performance of hydrofoils in varying fluid environments.

3.1.2.8 Measurement of Ride Height and Pitch Angle in STAR-CCM+

Precise measurement of ride height and pitch angle is crucial for providing accurate feedback to the PID controller, ensuring effective error correction and stability in dynamic flow conditions. In STAR-CCM+, these parameters are determined using built-in utilities that track the vessel's motion in response to hydrodynamic forces.

Ride Height Measurement

Ride height is defined as the vertical displacement of the vessel's center of gravity (COG) relative to its initial position. This deviation is measured using the *6-DOF Body Translation Report*, a utility in STAR-CCM+ that records translational motion in all three spatial directions. For ride height calculations, only the displacement along the vertical axis (y -axis) is considered. By continuously tracking this displacement over time, the ride height variations can be analyzed to assess the effectiveness of control strategies aimed at maintaining a stable elevation above the free surface.

Pitch Angle Measurement

The pitch angle, also referred to as the trim angle, represents the rotational mo-

tion of the vessel about its lateral axis. This angle is a critical parameter for evaluating the vessel's longitudinal stability and hydrodynamic efficiency. In STAR-CCM+, the pitch angle is measured using the *6-DOF Body Orientation Report*, which records the angular displacement of the vessel along all three rotational axes. The reported pitch angle allows for the identification of excessive trim variations, which could indicate instability or inefficient control performance.

Reporting and Visualization

The measured ride height and pitch angle values are typically recorded and analyzed through time-series plots, enabling a clear visualization of their variations under different operating conditions. These results are crucial for validating the performance of control algorithms, such as PID-based control strategies, in mitigating wave-induced oscillations.

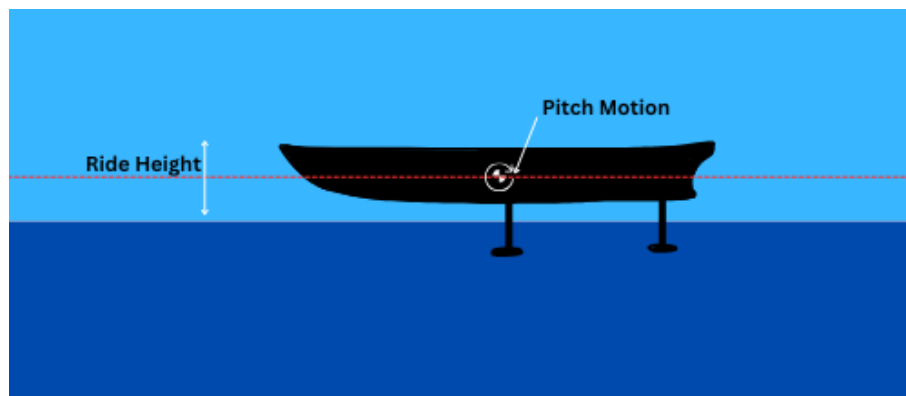


Figure 3.8: Hydrofoil Vessel Motion Measurements

Figures 3.8 illustrate the measurement of ride height and pitch angle, providing a graphical representation of how these parameters are tracked and analyzed in the simulation framework.

3.2 Control System Framework

This section outlines the design and implementation of the control system for regulating the hydrofoil's angle of attack (AoA). The integration of the PID controller with the STAR-CCM+ CFD environment is achieved using Java macros. The control system continuously monitors the lift force and adjusts the AoA to minimize deviations from the set point. The following subsections provide a detailed account of the PID control system implementation and its coupling with the CFD simulations.

3.2.1 PID Control System Implementation

The implementation of the PID controller within the STAR-CCM+ environment is achieved through the use of Java Macros, which enables the dynamic control

of the hydrofoil's angle of attack (AoA) based on the real-time feedback from the CFD simulation. The core function of the PID controller is to adjust the AoA in such a way that the hydrofoil maintains a desired objective, ensuring the control action is achieved. The detailed implementation and interaction of the control system with the CFD solver are described below.

3.2.1.1 PID Controller in Java Macros

The PID controller is implemented through a Java Macro that runs alongside the STAR-CCM+ simulation environment. The macro mainly performs the following tasks to regulate the AoA:

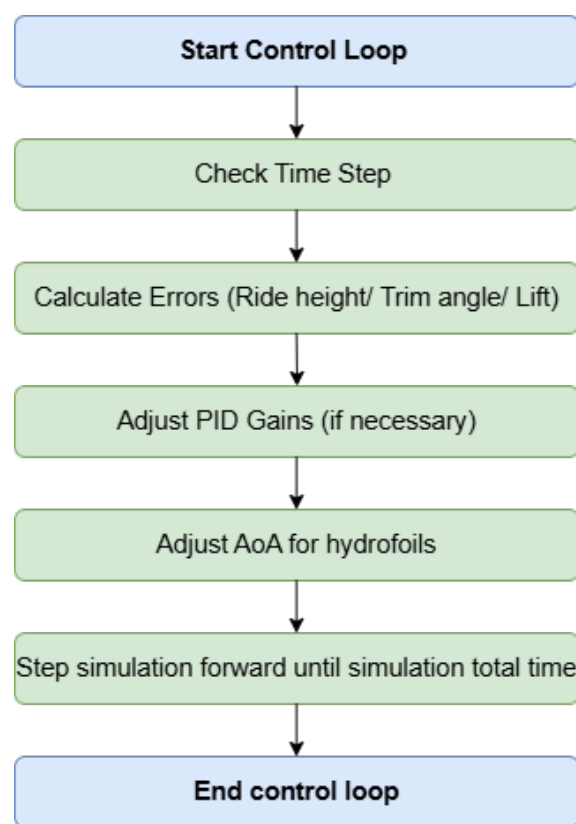


Figure 3.9: Control flowchart

1. **Reading Lift Force Feedback:** The lift force generated by the hydrofoil is continuously monitored within STAR-CCM+. This value is the primary feedback used by the PID controller.
2. **Computing Error:** The error is calculated as the difference between the desired lift force (setpoint) and the current lift force measured during the simulation. This error drives the PID control logic.

Pseudo-code for error calculation:

```
error = targetLift - currentLift;
```

3. **PID Calculation:** The error value is used in the PID control formula to calculate the correction required for the AoA. The controller applies proportional, integral, and derivative terms to compute the correction at each sampling timestep.

The PID control law is expressed as:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{d}{dt} e(t)$$

Pseudo-code for PID calculation:

```
proportional = Kp * error;  
integral = integral + error * deltaTime;  
derivative = (error - previousError) / deltaTime;  
controlSignal = proportional + Ki * integral + Kd *  
    derivative;
```

4. **Adjusting AoA:** The PID output, calculated in the previous step, is used to adjust the AoA of the hydrofoil in STAR-CCM+.

Pseudo-code for adjusting AoA:

```
adjustAoA(controlSignal);
```

The versatility of the PID control system implemented via Java Macros in the STAR-CCM+ environment lies in its adaptability to different control objectives. While the primary focus in the current setup is maintaining a target lift force by dynamically adjusting the AoA, the control framework can be extended to address alternative objectives with minimal modifications to the macro structure. For example, in scenarios where the objective is to maintain a constant ride height or to achieve a specific trim angle that minimizes vessel oscillations in response to wave-induced disturbances, the macro logic remains fundamentally the same.

The key difference in such cases lies in the definition of the error term, which is central to the PID controller. For instance, instead of comparing the measured lift force to a target lift, the error term may represent the deviation from the desired ride height or trim angle. Similarly, in applications involving highly dynamic environments, such as unsteady-flow conditions or wave interactions, the input to the error calculation might involve filtered values of measured parameters. For instance, when lift spikes are observed due to abrupt changes in AoA, applying a filter to the lift signal ensures that the PID controller receives clean and stabilized input, preventing drastic control adjustments.

This adaptability ensures that the PID controller can cater to a wide range of operational goals by tailoring the feedback parameters, error definitions, and filtering techniques. Such flexibility makes the Java Macro-based implementation a robust and scalable solution for controlling various aspects of hydrofoil dynamics in STAR-CCM+.

3.2.1.2 Coupling between CFD and PID Control System

The PID controller operates in a closed-loop system where the feedback from the CFD simulation directly influences the control action. The lift force data is provided by STAR-CCM+ to the Java Macro, which then processes the feedback and adjusts the AoA accordingly.

To facilitate this integration, STAR-CCM+'s Application Programming Interface (API) is used to interface the simulation data with the PID controller. The macro continuously interacts with the simulation to monitor lift forces and update the AoA in real-time.

The coupling process can be visualized in the following simplified flow:

```
while simulationRunning:
    currentLift = simulation.getLiftForce();
    error = targetLift - currentLift;
    controlSignal = Kp * error + Ki * integral + Kd *
        derivative;
    hydrofoil.setAngleOfAttack(controlSignal);
```

In this structure, the simulation continuously provides lift force values, the PID controller adjusts AoA in response, and the feedback loop ensures the hydrofoil achieves intended action.

3.2.1.3 PID Parameters Selection and Tuning

The effectiveness of a PID controller depends largely on the appropriate selection and tuning of its three fundamental parameters: proportional gain (K_p), integral gain (K_i), and derivative gain (K_d). These parameters influence the system's response, determining how well it tracks the desired setpoint and reacts to disturbances. Proper tuning of these parameters is crucial to achieving desired performance in a control system.

Initial Selection and Fixed PID Gains: During the early phases of the simulation, PID parameters were selected based on the results obtained from a Simulink model, which provided a preliminary framework for the control system. In this initial stage, the PID gains were set as fixed values and remained unchanged throughout the simulations. This approach was effective in providing a starting point for the controller's performance, but it had limitations. The fixed parameters lacked adaptability to changing conditions or disturbances in the system, which could negatively impact performance when the hydrofoil's dynamics varied or in the presence of external forces like waves.

The gains used in these early simulations were chosen heuristically, based on the expected behavior of the hydrofoil system, ensuring that the system did not exhibit excessive overshoot or instability. Although this approach provided a reasonable level of control, it was not optimized for dynamic performance across varying conditions. This experience highlighted the need for more advanced tuning techniques as the research progressed.

Adaptive PID Tuning: In adaptive PID tuning, the controller dynamically adjusts its parameters in response to the system's error characteristics over time. Rather than relying on fixed gain values, the adaptive PID approach updates the proportional (K_p), integral (K_i), and derivative (K_d) gains continuously, based on the current state of the error signal.

In this approach, the proportional gain (K_p) is adjusted for large errors, the derivative gain (K_d) helps smooth rapid fluctuations, and the integral gain (K_i) corrects accumulated errors. The details of this adaptive tuning logic are further elaborated in the pseudo code below.

Pseudo-code for adaptive tuning of PID:

```
Initialize PID parameters: Kp, Ki, Kd

While system is running:
    Measure the error:
        error = target_value - current_value

    Adjust Proportional Gain (Kp):
        If error is large:
            Increase Kp to make controller react strongly
        Else:
            Decrease Kp to prevent overshooting

    Adjust Integral Gain (Ki):
        If system is stable (error is small):
            Increase Ki to correct steady-state errors
        Else:
            Decrease Ki to avoid instability

    Adjust Derivative Gain (Kd):
        If error is changing rapidly:
            Increase Kd to smooth fluctuations and anticipate
            future error
        Else:
            Decrease Kd to avoid excessive damping

    Apply the updated PID parameters to the control system
End While
```

3.2.1.4 Discrete PID Control for Improved Stability and Noise Reduction

When analyzing the control of hydrofoils in CFD simulations, a discrete PID controller was implemented instead of a continuous one due to stability issues in the coupled simulation environment. In a fully continuous PID implementation, the control signal is updated in real time based on instantaneous feedback, whereas in a discrete PID setup, updates occur at fixed simulation timesteps. The continuous updates in a CFD simulation led to abrupt control input changes at each timestep, causing large, unanticipated adjustments in the hydrofoil's behav-

ior. This resulted in unwanted oscillations, commonly referred to as 'lift spikes,' which distorted the simulation results and hindered accurate performance assessment.

The lift spikes observed in the simulations can primarily be attributed to the instantaneous changes in the AoA between consecutive timesteps. In a continuous PID framework, these rapid adjustments in AoA lead to sudden shifts in the aerodynamic forces acting on the hydrofoil. These abrupt changes in control inputs cause aggressive reactions, where the system attempts to correct the error too quickly, resulting in an unstable response. The high frequency of such adjustments introduces significant noise into the lift data, generating erratic and unpredictable behavior. This noise complicates the evaluation of system performance and makes it difficult to establish a reliable and consistent response from the control system.

A more gradual adjustment of the AoA, applied over several time steps, can help reduce these spikes. By allowing the AoA to transition more smoothly, the changes in lift would be more continuous and less prone to abrupt fluctuations. Despite the presence of these spikes in the results, they tend to dissipate quickly, and the lift force stabilizes over time. This suggests that while the spikes are transient, they do not significantly impact the system's ability to eventually reach a steady state. Consequently, although these lift spikes appear in the simulations, their transient nature makes them less concerning when considering the overall stability and performance of the system.

This phenomenon highlights the critical importance of selecting an appropriate control strategy. In the case of this simulation, the discrete PID controller, which applies corrections every 0.1 seconds, ensures sufficient time for the system to adapt to error trends without causing rapid, large-scale adjustments that could lead to instability. By reducing the frequency of control corrections, this approach minimizes high-frequency noise, leading to a more stable system response and allowing for more accurate performance evaluation of the hydrofoil.

3.2.1.5 Challenges in Integration

Several challenges were encountered during the integration of the PID controller with STAR-CCM+:

- **Real-time Data Processing:** Ensuring that the PID controller could process real-time lift force data without introducing delays or performance issues was critical for maintaining stable control. Optimizing the Java Macro for speed was essential.
- **Parameter Tuning:** Tuning the PID parameters K_p , K_i , and K_d to ensure optimal system performance was a significant challenge. The controller's response was highly sensitive to these parameters, requiring iterative testing and adjustments.
- **Numerical Stability:** Ensuring numerical stability between the PID control system and CFD solver was crucial. Any instability in the CFD simula-

tion could lead to erratic control responses, requiring continuous monitoring and fine-tuning of both simulation settings and PID gains.

- **Dynamic Simulation Complexity:** Transitioning from steady-flow to dynamic flow conditions, such as sinusoidal and multiphase flows, introduced additional complexity. The PID controller had to adapt to changes in lift force due to fluctuating inlet velocities.

Despite these challenges, the integration of the PID control system with STAR-CCM+ enhanced the hydrofoil's performance by dynamically adjusting the AoA based on real-time feedback. This setup provides a robust framework for simulating and controlling the hydrofoil's behavior across different flow conditions.

3.2.2 Simulink-Based Control System Analysis

This section discusses the use of Simulink for modeling and analyzing the control system of a hydrofoil vessel, highlighting its role in complementing the CFD-PID coupled framework. Simulink, a MATLAB-based simulation platform, offers a robust and flexible environment for modeling, simulating, and evaluating dynamic systems. In this study, Simulink was utilized to replicate the control system of the hydrofoil vessel, enabling a detailed analysis of its performance under various dynamic conditions.

Simulink's block diagram architecture provided an intuitive interface for modeling the system's dynamics, noise effects, and filtering techniques, making it a useful tool for testing and fine-tuning the control system. The primary focus of the Simulink model was to analyze the closed-loop behavior of the control system, integrating the PID controller with the hydrofoil dynamics. By employing Simulink, it enabled to model a simplified version of the hydrofoil system, capturing the key elements of the system's behavior without the complexity of real-world fluid dynamics. This allowed for a more controlled and manageable testing environment, where specific aspects of the control system could be isolated and studied in detail.

It is important to note that the Simulink model was not intended for validation or verification of the control system. Instead, its primary purpose was to understand the dynamics of the closed-loop control system, the integration of the PID controller, and to approximate the optimal PID gains. The model served as an initial testing ground for the control strategy before it was applied to the more computationally intensive CFD environment. The simplified nature of the Simulink model made it an ideal tool for gaining insights into the fundamental behavior of the system, such as control response, system stability, and the tuning of PID gains. These insights were crucial in guiding the subsequent application of the control system in CFD simulations.

However, for the Simulink model to be effective in validation and verification, it would need to replicate the dynamics of the CFD model more closely. In particular, the Simulink model lacks the ability to simulate the complex fluid dynamics present in the CFD model, such as wave interactions, multiphase flow, and depth-dependent effects. These complexities play a critical role in the actual

performance of the hydrofoil vessel, and their absence in the Simulink model limits the ability of the model to fully replicate real-world conditions. For instance, phenomena like wave-induced disturbances, turbulence, and the interactions between the hydrofoils and the surrounding fluid are challenging to simulate in Simulink, as these effects require advanced multiphase and turbulence models that are typically handled in specialized CFD software.

3.2.2.1 Assumptions in the Simulink model

The following assumptions were implemented in the Simulink model:

- **Linearized Dynamics:** The system dynamics are simplified to linear approximations around a steady-flow operating point, neglecting complex nonlinear behaviors.
- **Ideal Actuator Response:** Actuators are assumed to have instantaneous or minimal delays in response to control inputs, ignoring real-world lag effects.
- **Simplified Noise Representation:** Noise disturbances, such as wave-induced or turbulence-related noise, are modeled as basic patterns, often sinusoidal, rather than replicating real-world complexity.
- **Perfect Sensor Feedback:** Sensors are assumed to provide accurate, noise-free, and instantaneous measurements, without accounting for delays or inaccuracies.
- **Constant Environmental Conditions:** Parameters like water density and flow velocity are treated as constant, excluding potential variations.
- **Fixed Sampling Period:** The control updates occur at a consistent interval, which may not fully represent the dynamic variability of the physical system.
- **Isolated System Analysis:** The control system is analyzed independently, without integrating detailed feedback from fluid dynamics as seen in coupled CFD simulations.

3.2.2.2 Simulink Modeling for Hydrofoil Control System in Steady-Flow Conditions

The Simulink model was developed to analyze the behavior of the hydrofoil control system under steady-flow conditions. In the steady-flow setup as shown in Figure 3.10, the flow velocity, water density, and other environmental parameters are assumed to remain constant. This enables a preliminary examination of the controller's proportional, integral, and derivative components to achieve precise adjustments to the AoA. The model incorporates simplified representations of system dynamics.

The primary objective of this Simulink study is to predict the behavior of the control system under idealized conditions, facilitating the identification of optimal control gains and initial validation of the system's response.

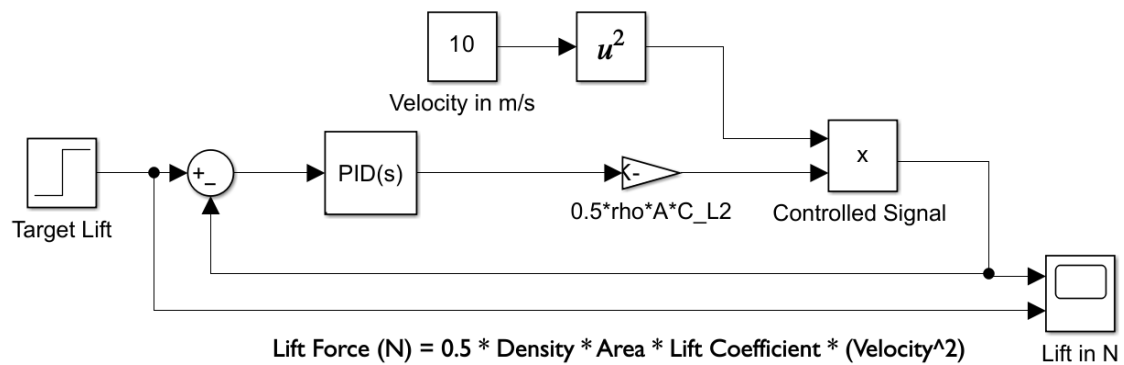


Figure 3.10: Simulink model for steady-flow conditions

3.2.2.3 Simulink Modeling for Hydrofoil Control System in Unsteady-Flow Conditions

The Simulink model was further extended to analyze the behavior of the hydrofoil control system under unsteady-flow conditions. In this setup, the flow velocity was varied sinusoidally to replicate the fluctuating wave-induced flow. The sine wave velocity function introduces periodic changes in flow velocity, adding complexity to the system's dynamics.

The model evaluates the PID controller's ability to adapt to continuously varying conditions, ensuring that the desired lift force is maintained despite changes in flow velocity. The unsteady-flow scenario requires the controller to dynamically adjust the AoA in response to the periodic velocity variations, challenging the stability and responsiveness of the control system.

The primary objective of this unsteady-flow Simulink study is to understand the robustness of the PID controller under unsteady conditions. The Simulink model, depicted in Figure 3.11, illustrates the incorporation of the sine wave velocity function and its integration with the control system to simulate the hydrofoil's behavior under unsteady-flow conditions.

3.2.2.4 Simulink Modeling for Hydrofoil Control System with Noise in Velocity Input

The Simulink model was further enhanced to include noise components in the velocity input, providing a more realistic simulation of dynamic conditions encountered by hydrofoils. Along with the sinusoidal velocity function, noise was implemented to mimic high-frequency disturbances and random fluctuations typically observed in wave-induced and turbulent environments.

The noise model shown in Figure 3.13 consists of a combination of:

- A sinusoidal base velocity function representing periodic changes in flow velocity.
- High-frequency noise signals superimposed on the sinusoidal function to emulate rapid disturbances.

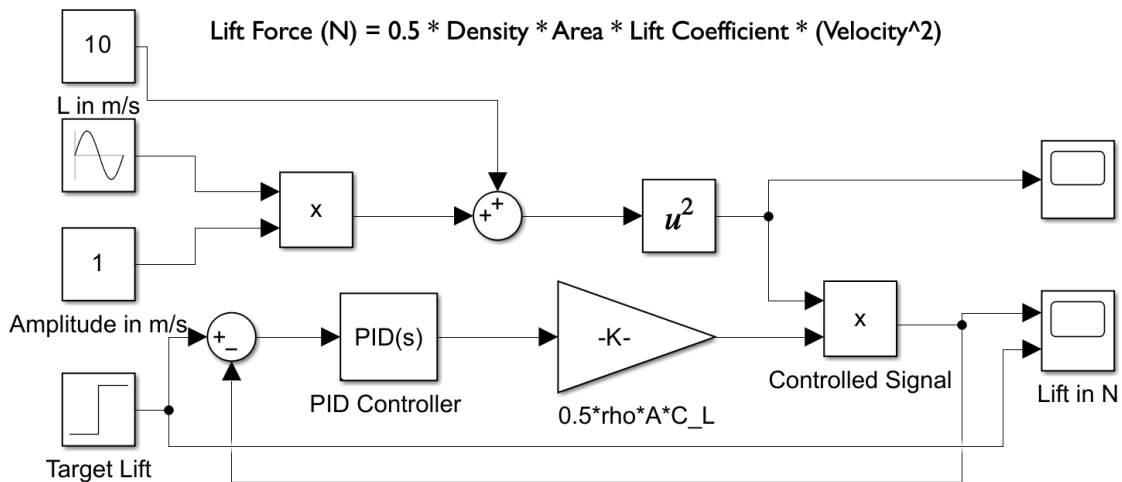


Figure 3.11: Simulink model for unsteady-flow conditions

- Band-limited white noise generated using Simulink's built-in noise generator, introducing stochastic variability.

These components were integrated into a dedicated subsystem, as shown in Figure 3.12, allowing modular incorporation into the control system model. The resulting composite velocity signal provides a challenging input for the PID controller, requiring it to filter out noise while maintaining the desired lift force.

The primary objective of this noise-augmented Simulink study is to evaluate the PID controller's robustness and effectiveness in managing noisy and highly variable input conditions.

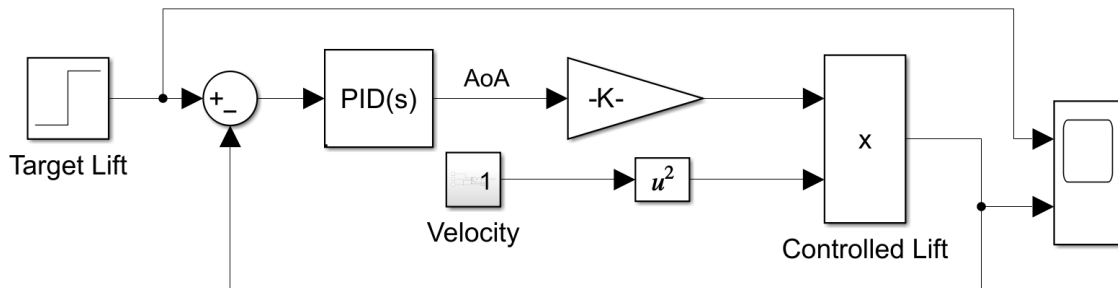


Figure 3.12: Simulink model with noise addition in velocity input

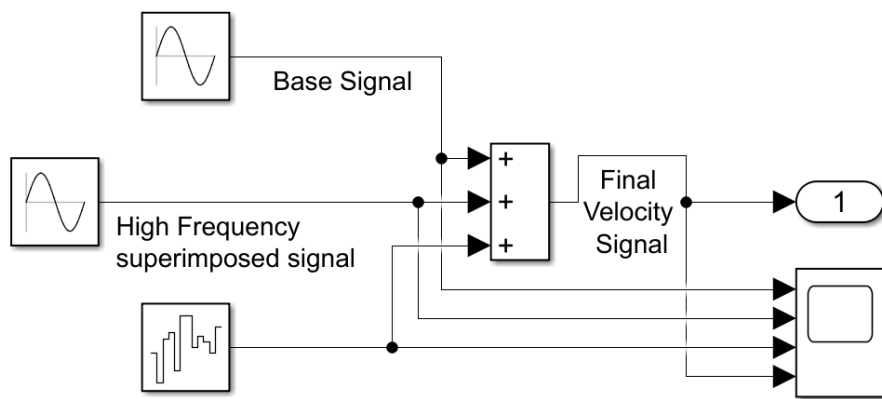


Figure 3.13: Velocity subsystem

4

Results

This chapter presents a detailed analysis of the findings derived from the implemented methodologies, focusing on the performance and behavior of the control system under varying simulation conditions. The results provide a quantitative and qualitative evaluation of the control strategies, highlighting their effectiveness in achieving the defined objectives.

The progression of results begins with fundamental simulations, including steady-flow analyses, and advances to scenarios involving unsteady-flow conditions with sinusoidal velocity inlets. Additionally, the outcomes of more complex configurations, such as dual-hydrofoil setups, are systematically included. Each section emphasizes the impact of control system adjustments, including noise mitigation techniques and dynamic parameter tuning, on the stability and efficiency of the hydrofoil system.

4.1 Simulink-Based Simulation Results

This section presents the results obtained from the Simulink model simulations for hydrofoil lift control. The analysis is structured into three subsections: (1) steady-flow performance, (2) unsteady-flow response under sinusoidal velocity input, and (3) performance under additional noise perturbations. The PID gains used in all simulations are the same, as detailed in Table 4.1.

PID Controller Parameter	Value
Proportional Gain (K_p)	10
Integral Gain (K_i)	0.5
Derivative Gain (K_d)	1

Table 4.1: PID Gains for the Control System

4.1.1 Steady-Flow Response for Target Lift of 5000N

In the first simulation, the hydrofoil control system was tested under steady-flow conditions with a constant velocity inlet to achieve a target lift force of 5000N. The PID controller adjusted the angle of attack (AoA) to match the desired lift force.

The steady-flow results depicted in Figure 4.1 establish a baseline for assessing the system's performance under more dynamic and disturbed conditions, which are explored in the following subsections.

The results indicated that the PID-controlled system successfully reached the target lift with minimal error after a transient phase. The AoA adjustments stabilized over time, showing only negligible fluctuations.

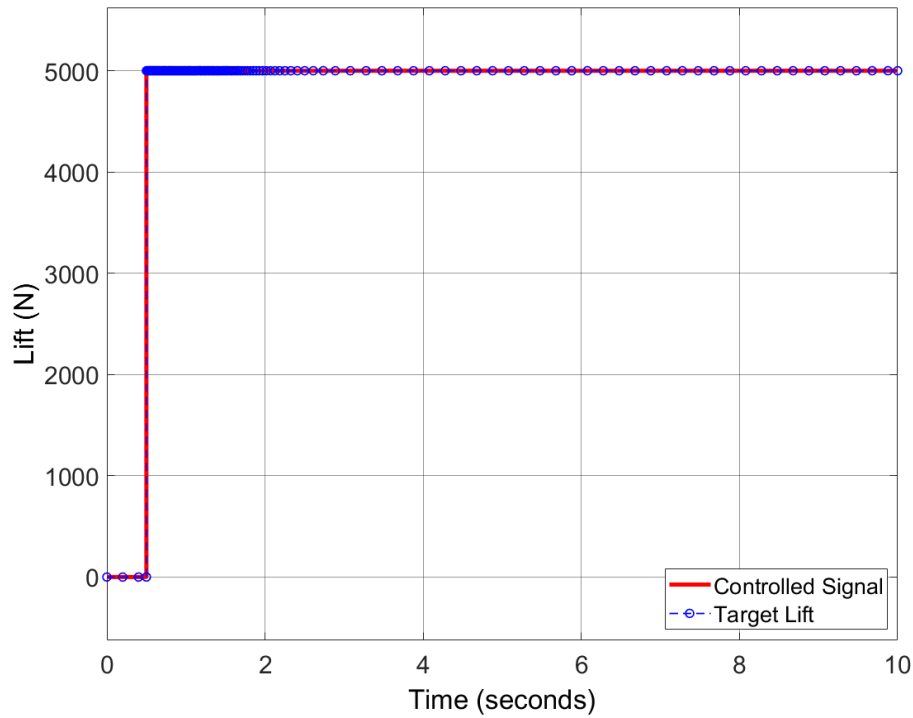


Figure 4.1: Response curve for steady-flow model

4.1.2 Unsteady-Flow Response with Sinusoidal Velocity Inlet

To simulate real-world operating conditions, the second simulation based on 3.2.2.3 introduced a time-varying velocity inlet defined by a sinusoidal function. This variation emulated wave-induced velocity fluctuations, providing a more challenging test for the control system. The sinusoidal velocity input applied in the simulation is depicted in Figure 4.2, illustrating the periodic variations that influence the hydrofoil's lift generation.

Unsteady Flow Velocity Parameter	Value
Bias	10 m/s
Amplitude	1 m/s
Frequency	3.141 rad/sec

Table 4.2: Unsteady Flow Velocity Block Parameters

The results in Figure 4.4 show that the lift force fluctuates in response to the velocity changes, requiring continuous AoA adjustments by the PID controller. The corresponding AoA corrections are presented in Figure 4.2, which highlight the dynamic response of the control system. It can be observed that whenever the velocity reaches a trough, the AoA must increase (crest) to compensate for the reduction in lift. This behavior demonstrates how the control system dynamically adjusts the hydrofoil's angle to maintain the desired lift despite velocity variations.

Although the controller managed to track the target lift over time, the oscillations in AoA were significantly more pronounced compared to the steady-flow case. This insight is valuable for application in CFD models, as it emphasizes the necessity of robust control strategies that can handle wave-induced velocity fluctuations. The correlation between velocity troughs and AoA crests serves as a fundamental principle in designing adaptive control strategies for hydrofoils operating in unsteady-flow conditions.

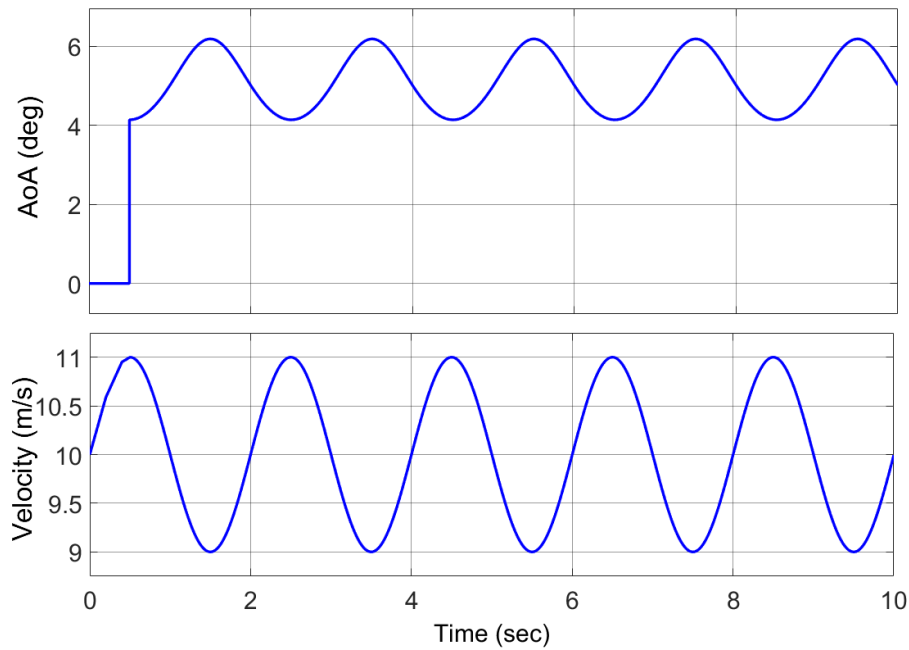


Figure 4.2: Response curve for AoA and sinusoidal inlet velocity

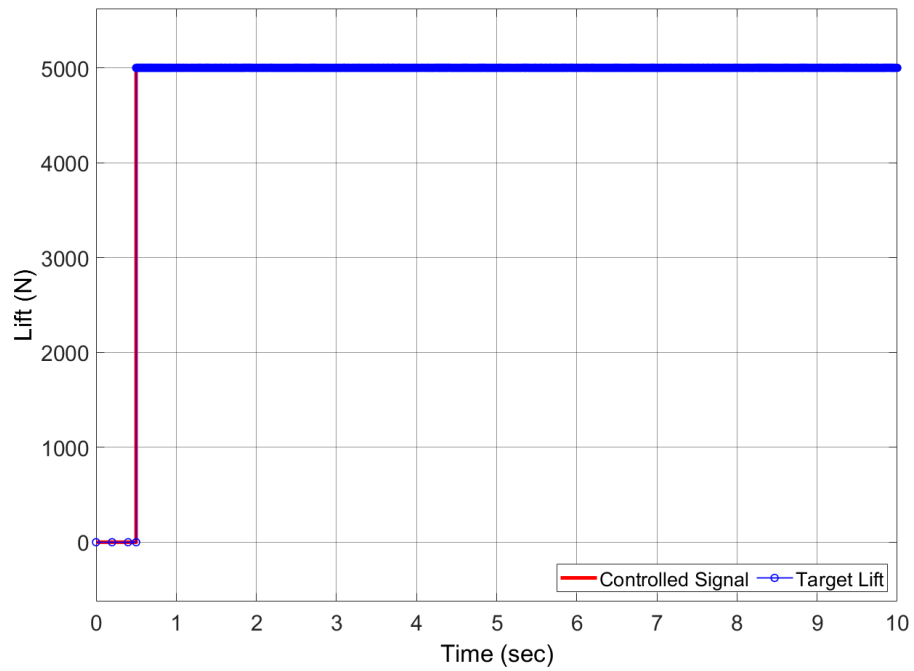


Figure 4.3: Response curve for unsteady-flow model

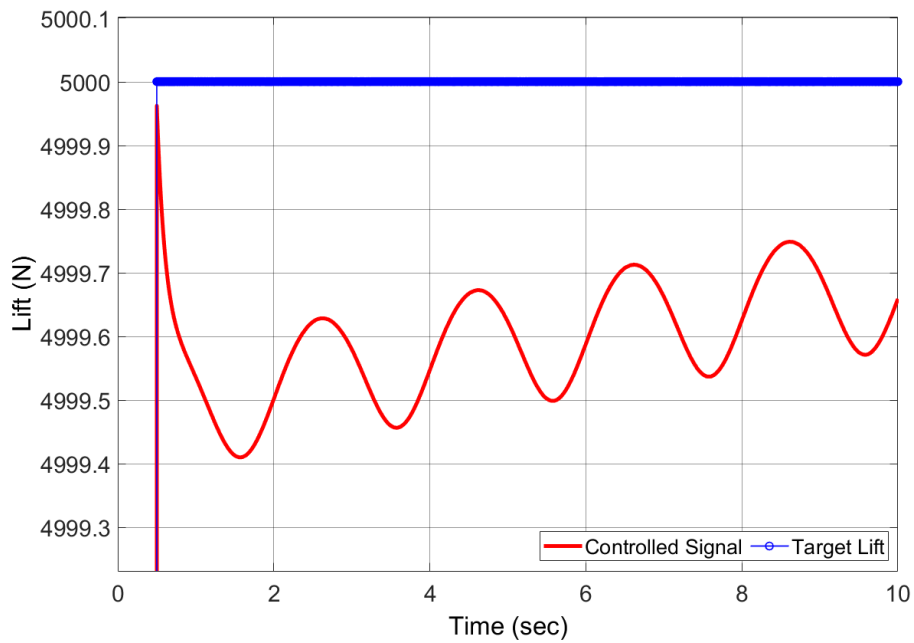


Figure 4.4: Response curve for unsteady-flow model (enlarged)

4.1.3 Response Under Additional Noise in Velocity Input

In the third simulation, additional noise was introduced, as described in Section 3.2.2.4, into the velocity input to simulate realistic environmental disturbances such as turbulence or sensor inaccuracies. The noise was randomly distributed, adding unpredictability to the velocity fluctuations as mentioned in 3.2.2.4 and further testing the robustness of the control system.

Noise Block	Parameter	Value
Base Signal	Amplitude	2 m/s
	Bias	10 m/s
	Frequency	2×6.28 rad/sec
	Sample Time	0.0 s
High-Frequency Superimposed Signal	Amplitude	0.2 m/s
	Bias	0.0 m/s
	Frequency	100 rad/sec
	Sample Time	0.0 s
Band-Limited White Noise	Noise Power	0.1
	Sample Time	0.1 s
	Seed	23341

Table 4.3: Noise model parameters for velocity input consisting of three blocks: base signal, high-frequency superimposed signal, and band-limited white noise.

The results indicated that the overall control response remained similar to the previous unsteady-flow case, with the PID controller continuously adjusting the AoA to counteract velocity variations. However, the presence of noise introduced additional perturbations, leading to higher-frequency oscillations in the AoA corrections seen in Figure 4.6 and increased deviations in the lift response. Furthermore, a noticeable steady-state error was observed as seen in Figure 4.7 and 4.8, as the lift force no longer consistently reached the target value. This suggests that the controller struggled to fully compensate for the noise-induced fluctuations, resulting in a persistent offset from the desired lift. The inability to perfectly track the target lift highlights the limitations of the PID controller in handling high-frequency disturbances without additional filtering or adaptive tuning strategies.

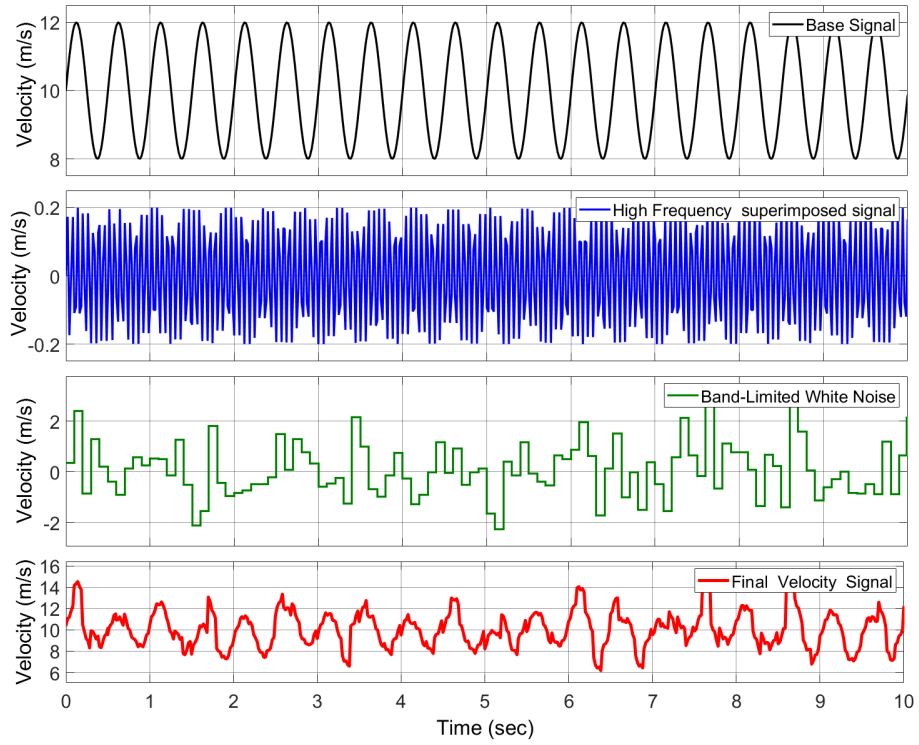


Figure 4.5: Noise model for velocity

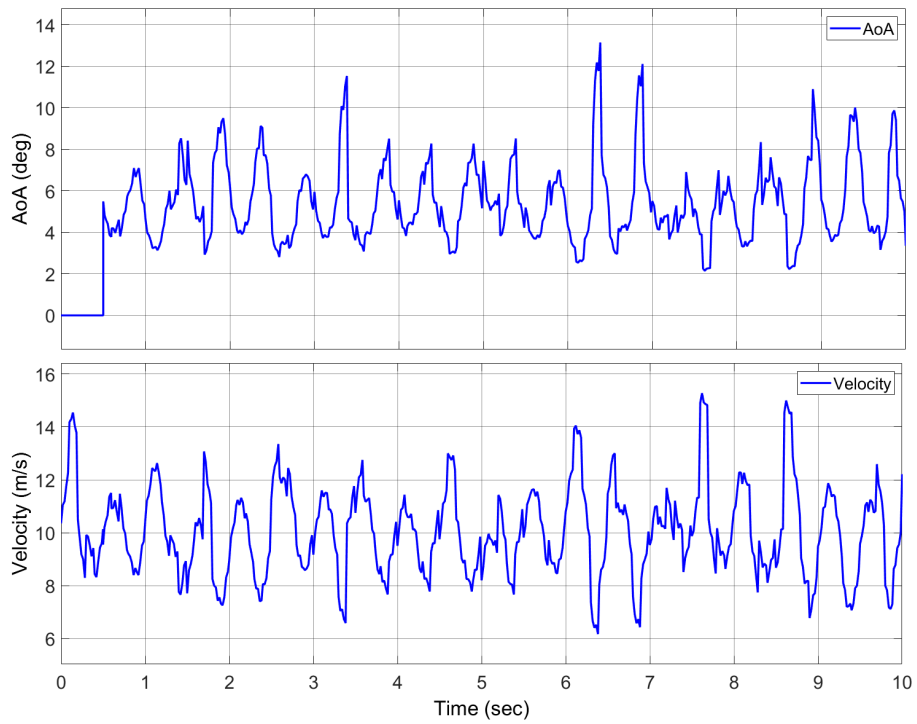


Figure 4.6: Response curve for AoA and sinusoidal inlet velocity with noise

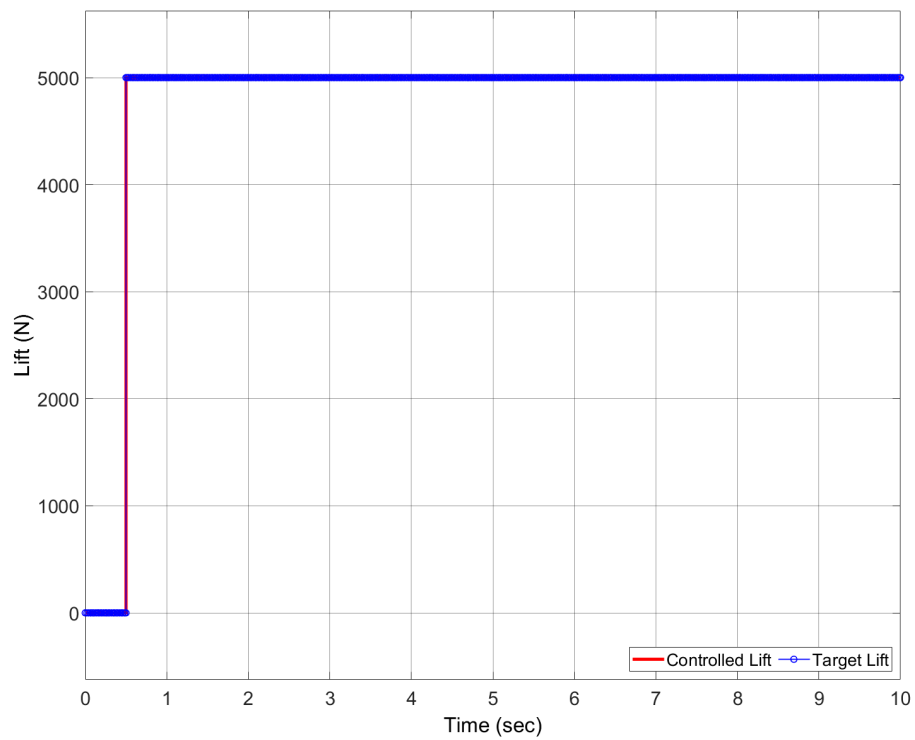


Figure 4.7: Response curve for unsteady-flow model with noise

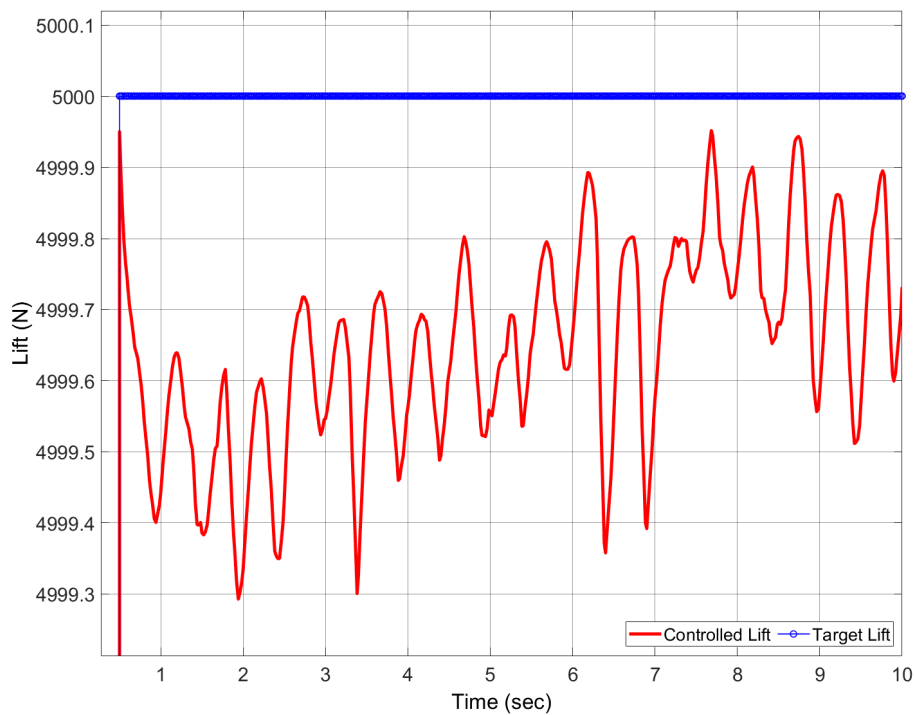


Figure 4.8: Response curve for unsteady-flow model with noise (enlarged)

4.2 CFD-Based Simulation Results

This section presents the results obtained from the CFD simulations conducted to evaluate the performance of the hydrofoil control system.

4.2.1 Steady-Flow 2D Setup

Under steady-flow conditions, the performance of a simple PID control system is evaluated using a 2D CFD framework. This approach provides a controlled environment to assess the controller's capability to regulate the hydrofoil's AoA and achieve a specified target lift force. The 2D setup simplifies the hydrofoil problem by removing the spanwise direction, effectively modeling the hydrofoil as an infinitely long body. In practical terms, the CFD solver assumes a span of 1 meter to calculate aerodynamic forces. The resulting planform area is computed based on the chord length of the hydrofoil and the assumed span:

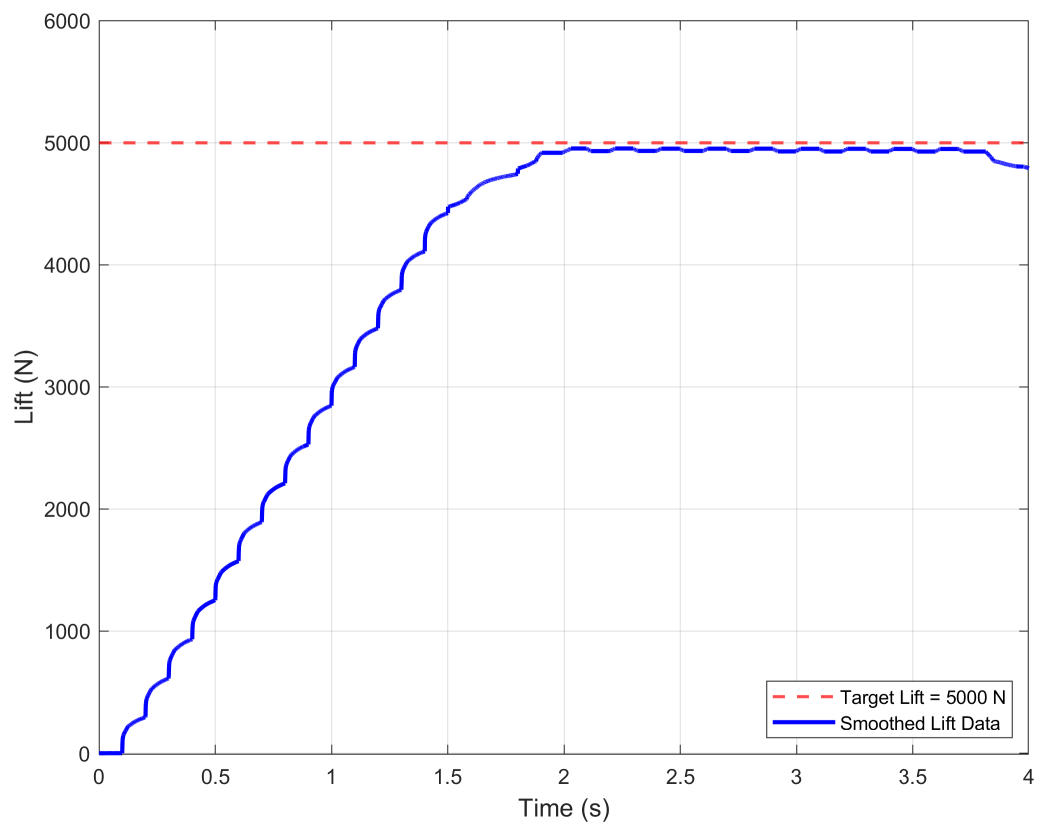
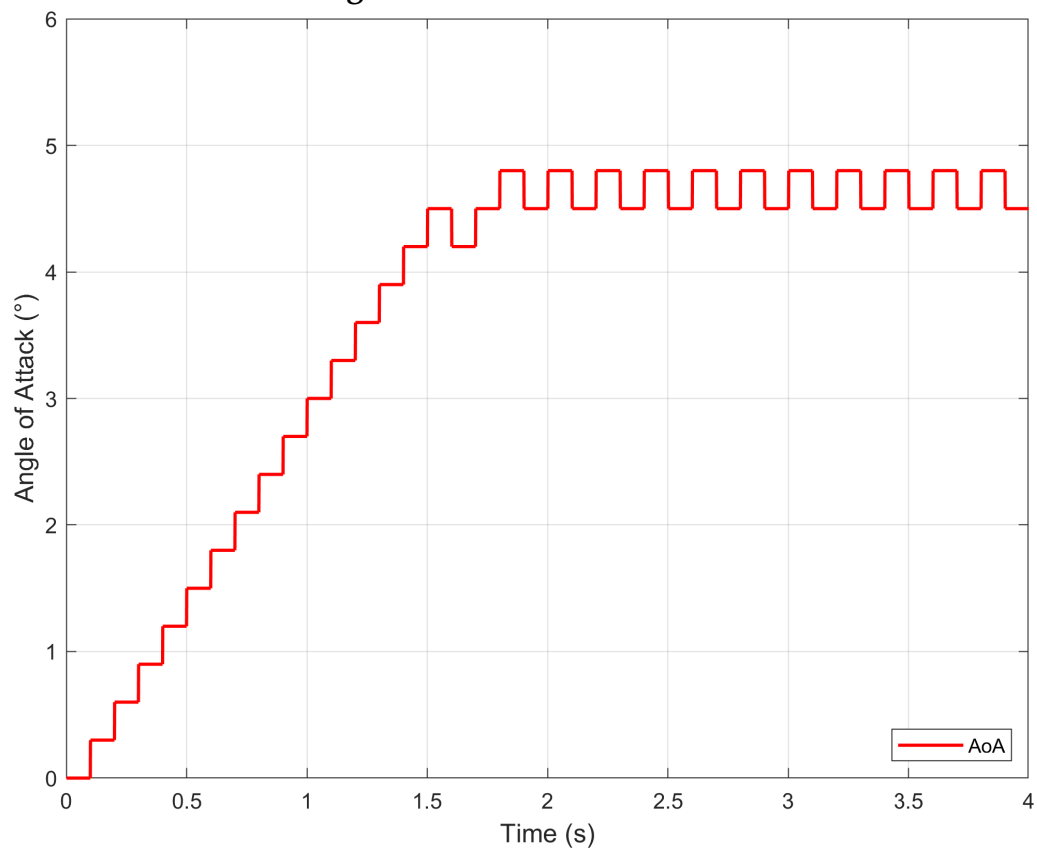
$$\text{Planform Area (A)} = \text{Chord Length (0.2 m)} \times \text{Span (1 m)} = 0.2 \text{ m}^2$$

The primary objective of this steady-flow simulation is to achieve a target lift force of approximately 5000 N. This value is selected to represent a significant aerodynamic load, offering a meaningful test for the control system's ability to stabilize the hydrofoil while dynamically adjusting its angle of attack (AoA).

The lift data obtained from the steady-flow 2D CFD simulation provides insights into the performance and behavior of the PID control system. The analysis consists of two primary sets of plots: one representing the raw lift data alongside the PID controller corrections, and the other depicting the smoothed lift data after applying a median filter.

The raw lift data, shown in Figure B.1, illustrates the instantaneous fluctuations in lift force as the PID controller adjusts the angle of attack (AoA) in real-time to achieve the target lift. These variations often lead to sharp lift spikes, which arise due to the controller's attempts to correct for errors between the target and actual lift. The AoA changes, as depicted in Figure 4.10, reflect the PID controller's dynamic adjustments in response to these fluctuations. The parameters used, such as the PID gains and other settings, are detailed in the code provided for this analysis in A.5.

In order to mitigate the disruptive effect of these lift spikes and to focus on the overall trend in the lift force, a median filter, sourced from MATLAB's toolbox, is applied to the raw lift data for better analysis and data visualization. As shown in Figure 4.9, the median filter reduces the sharp, transient deviations while preserving the general lift force trends. The filter operates by replacing each data point with the median of its neighboring points within a predefined window, thereby smoothing out high-frequency noise while maintaining the integrity of the signal's primary features.

**Figure 4.9: Lift Force vs Time****Figure 4.10: AoA vs Time**

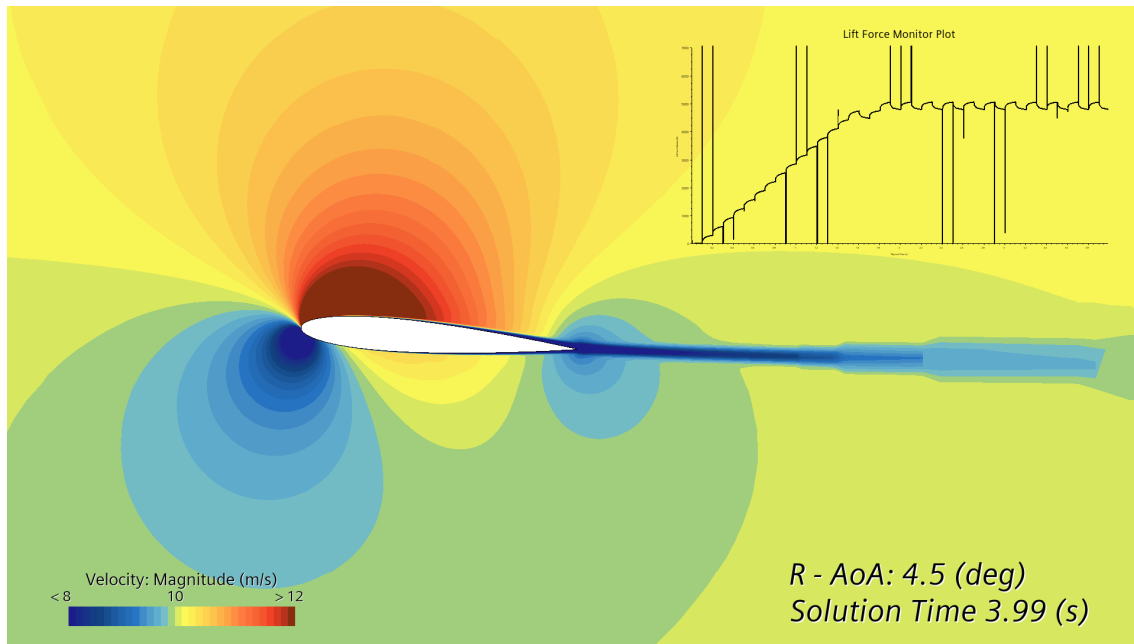


Figure 4.11: Steady flow—final velocity scene

4.2.1.1 Key Observations

Based on the lift data, it appears that the coupling between the CFD model and the PID control system is functioning as expected through analytical evaluation, although further optimizations are required for improved performance. The current setup, which approximates the hydrofoil as a 2D model, is limited by the assumption of a unit span of 1 meter, as the spanwise direction is not considered. Consequently, only the chord length can be varied, and the system is unable to account for three-dimensional effects. Despite these limitations, the PID control system is able to track the target lift force reasonably well under steady-flow conditions.

However, it can be observed that the angle of attack (AoA) oscillates around the steady-flow value due to the angle limiter of 0.3 degrees, which constrains the adaptive reduction of AoA changes. This limitation prevents the PID controller from smoothly adjusting the AoA in response to smaller corrections, leading to angular oscillations in the system's behavior. To address this, an algorithm could be implemented to adjust the angle limiter dynamically, allowing for more adaptive and refined control actions. This would help reduce the oscillations and improve the overall stability of the control system, especially in steady-flow operation.

While the current results demonstrate the functionality of the CFD-PID coupling, the system's performance could be significantly enhanced through further refinements. A final velocity scene is presented in Figure 4.11, providing an overall picture of the flow dynamics and control performance.

4.2.2 Unsteady-Flow 2D Setup

In contrast to the steady-flow setup, which assumes constant flow conditions, the unsteady-flow scenario introduces dynamic velocity fluctuations to replicate real-world wave-induced disturbances. This configuration is designed to assess the performance of the PID controller in maintaining stable lift and adjusting the AoA in response to periodic flow velocity changes, which closely mimic the operating conditions encountered by a hydrofoil in dynamic environments. The unsteady-flow 2D setup builds upon the steady-flow model by incorporating a time-varying velocity input represented as a sine function, simulating periodic fluctuations in flow speed. The simulation settings remain similar to the steady-flow setup, as described in A.5, with modifications to incorporate time-dependent velocity variations within the STAR-CCM+ case file.

The PID control system in this setup functions in a similar manner to the steady-flow case, where the controller continuously adjusts the AoA to achieve the desired lift force. However, due to the dynamic nature of the input velocity, the PID controller must respond to more complex and unpredictable flow variations. As seen in Figure 4.12 and Figure 4.13, the lift force and AoA fluctuate over time as the system adjusts to the periodic changes in velocity.

4.2.2.1 Key Observations

While the PID control system demonstrates functionality and starts to oscillate around the target lift, it fails to reach a true steady-flow value. This issue can be attributed to the discrete nature of the AoA updates, which are performed at fixed intervals. The current sampling rate allows for AoA adjustments of only 0.3 degrees per sampling interval, a value that may be insufficient to maintain a stable target lift. Increasing the step size could improve the controller's responsiveness but would likely introduce large lift spikes, potentially destabilizing the system and reducing the overall effectiveness of the control.

Alternatively, continuously sampling the AoA at each CFD timestep might lead to quicker adjustments, thereby improving the accuracy of the lift force control. However, this alternative approach neglects the physical limitations of the motor's angular velocity and could result in unrealistic, physically infeasible AoA changes. Furthermore, rapid adjustments may not give the CFD solver sufficient time to stabilize the flow at each new AoA configuration, undermining the effectiveness of the simulation. Thus, a balance must be struck between the rate of AoA adjustment and the stability of both the control system and the CFD solver. Future improvements could involve adaptive AoA control strategies that take into account both the physical dynamics of the system and the computational stability of the solver. A final velocity scene is presented in Figure 4.14, providing an overall picture of the flow dynamics and control performance.

4. Results

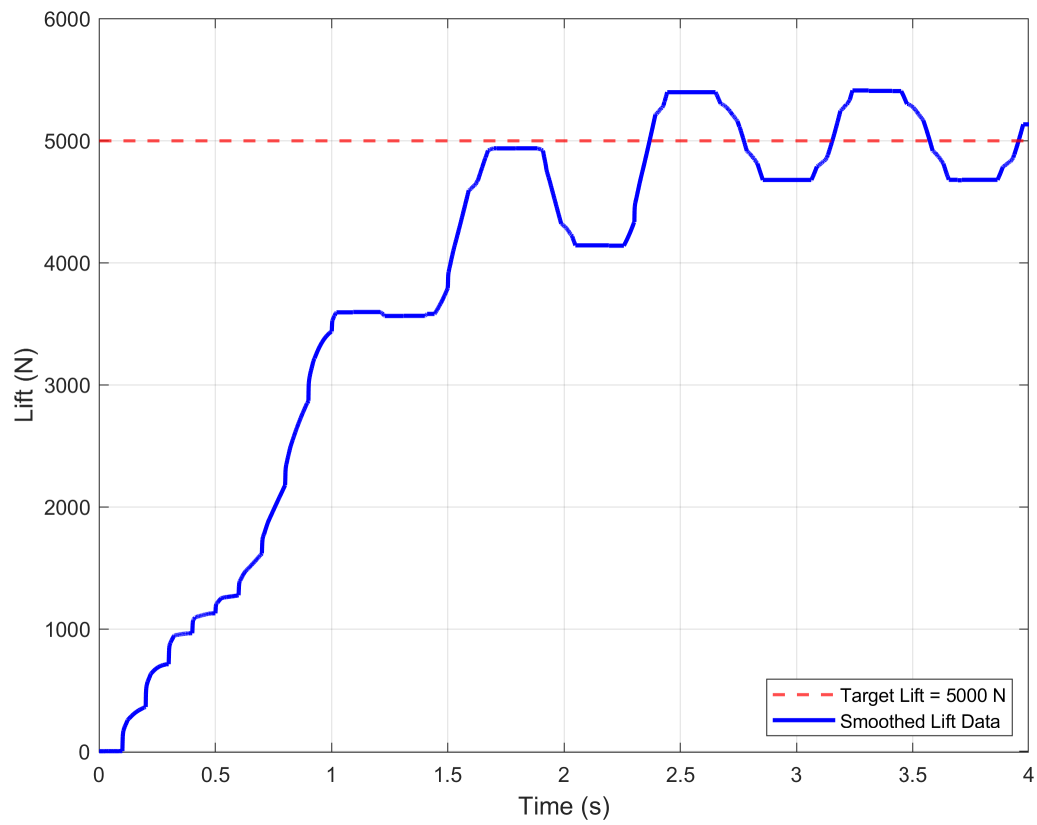


Figure 4.12: Lift Force vs Time

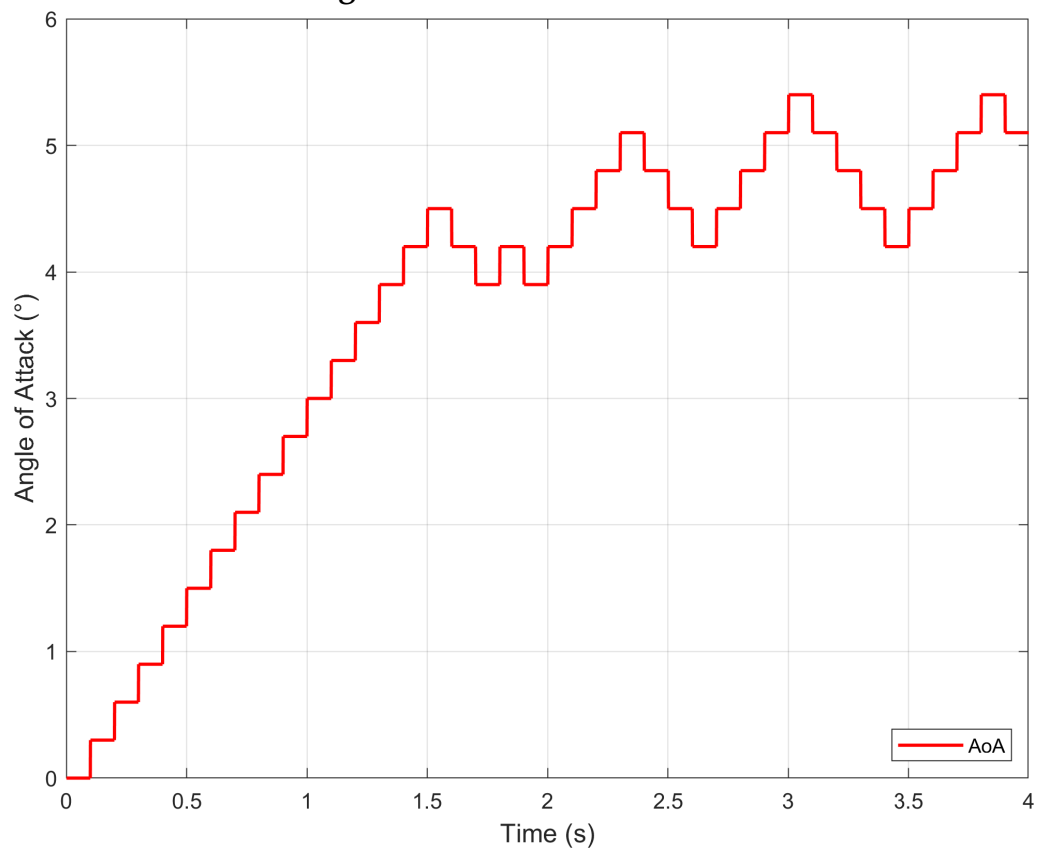


Figure 4.13: AoA vs Time

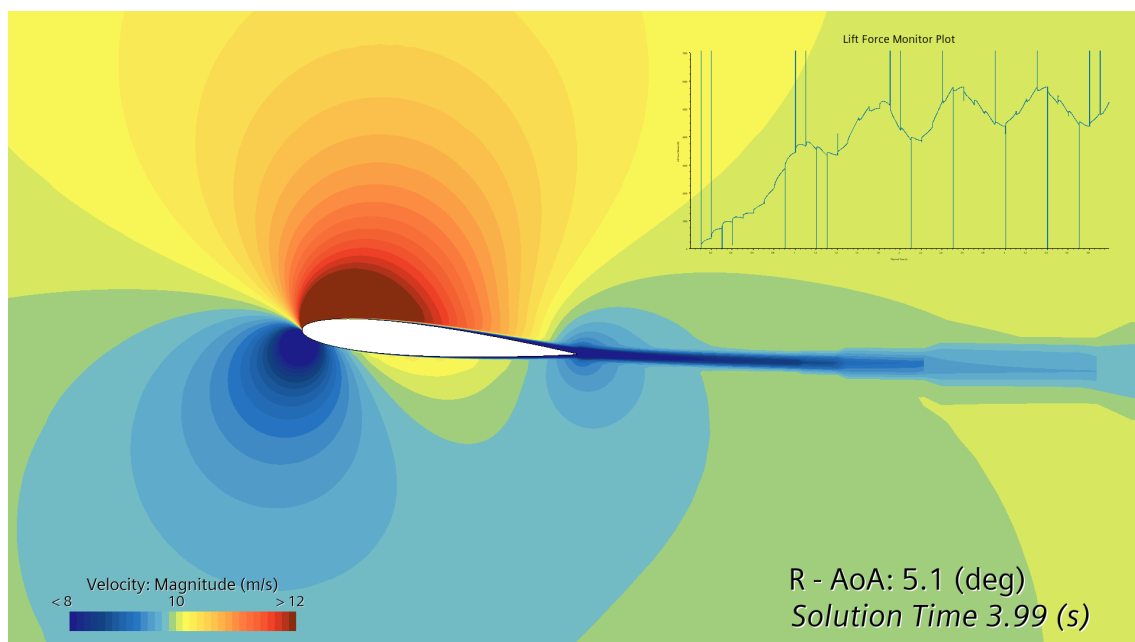


Figure 4.14: Unsteady flow—final velocity scene

4.2.3 Semi-2D Setup with Multiphase Flow Integration

The transition from a purely 2D setup to a semi-2D configuration represents a crucial step in developing a more realistic numerical framework for hydrofoil control. In the initial 2D simulations, the objective was to establish fundamental lift control strategies within a simplified domain. However, to better capture the interactions between the hydrofoil and the surrounding flow, the study progressed to a semi-2D setup, where the hydrofoil is modeled with a finite thickness of 0.02 m. This configuration retains certain 2D characteristics while introducing three-dimensional effects, such as multiphase flow interactions with the free surface. Consequently, the semi-2D approach provides a more representative physical environment for evaluating the hydrofoil's dynamic response.

A significant challenge encountered in this setup was the presence of lift fluctuations, which resulted in a noisy lift signal. Since the lift signal directly feeds into the PID controller as an error term (computed as the difference between the target lift and the measured lift), these fluctuations introduced inconsistencies in control performance. The variations in lift, particularly in wave-influenced conditions, led to abrupt angle-of-attack (AoA) adjustments by the controller, potentially destabilizing the system. Therefore, to ensure a more stable and accurate control response, it became essential to preprocess the lift signal through filtering techniques before feeding it into the PID controller.

To address this issue, two different filtering approaches were considered: a mean filter and an Exponential Moving Average (EMA) filter. After selecting these filtering strategies, the next critical step involved optimizing the filter parameters to strike a balance between responsiveness and stability.

Mathematically, the EMA filter updates the lift signal at each timestep t based on the following formula:

$$L_{\text{EMA}}(t) = \alpha L(t) + (1 - \alpha)L_{\text{EMA}}(t - 1) \quad (4.1)$$

Where:

- $L(t)$ is the current lift value,
- $L_{\text{EMA}}(t - 1)$ is the previous EMA value,
- α is the smoothing factor, which determines the weight assigned to the current lift value ($0 \leq \alpha \leq 1$).

With an improved lift control strategy in place, the semi-2D setup was further extended to incorporate the DFBI model, as described in 3.6. The integration of DFBI allowed for a more comprehensive simulation where the hydrofoil's motion and interaction with the surrounding fluid were dynamically resolved. This marked a significant advancement from the initial lift control framework, as the hydrofoil was now free to adjust its ride height and pitch angle based on fluid forces rather than being externally constrained. Consequently, the semi-2D setup provided a more realistic foundation for investigating hydrodynamic control strategies in a coupled CFD-PID environment.

4.2.3.1 Filter Selection for Lift Signal

To enhance the performance of the PID controller and ensure accurate lift control, the selection of an appropriate filtering mechanism for the lift signal was evaluated through three distinct simulations. The first simulation used no filtering, wherein the raw lift data—characterized by significant noise and transient spikes—was directly input into the PID controller. This approach provided a baseline to observe the system’s inherent behavior without any smoothing. The second simulation employed a mean filter, which calculated the average of all past lift samples up to each CFD timestep and passed this smoothed value to the PID controller at each timestep. This method effectively reduced noise but introduced a slight delay in system responsiveness. The third simulation utilized an exponential moving average (EMA) filter described in Eq. (4.1), which assigned weights to past lift values, allowing recent data to contribute more significantly. This weighted approach aimed to balance noise reduction with faster responses to variations in lift. The parameters used, such as the PID gains and other settings, are detailed in the code provided for this analysis in A.6.

To better interpret the results shown in Figure 4.15 and trends from these filtering mechanisms, the mean lift values were extracted as the simulation progressed as observed in Figure 4.17. This technique calculated the average of all past lift values, generating a smooth trend line that highlighted the overall lift behavior. The moving window mean served as a reference for evaluating the filters based on key parameters such as signal stability (to assess noise and spike suppression), system responsiveness (to determine how quickly the controller could respond to changes), and trend accuracy (to evaluate the alignment of the filtered signal with the moving average trend). These factors were essential for understanding the performance of each filtering method.

It is important to note that the selection of an optimal filter may vary depending on the simulation’s specific objectives. In this case, the primary goal was to maintain a target lift of 50 N for a single hydrofoil operating under wave-induced flow conditions. For this task, it was essential to strike a balance between stability and responsiveness. However, in scenarios with different control objectives—such as minimizing drag or optimizing energy efficiency—the most appropriate filter might change. To ensure that the selected filter performs well across various conditions, it is important to test how different filter settings affect the system’s performance under each scenario. This testing, often referred to as a parameter sensitivity study, helps determine the best filter for each specific control goal.

After extracting and plotting the data from all three filter implementations, the PID performance and evaluation parameters were assessed to determine the most suitable filter for our control objective. These parameters, including rise time, overshoot, settling time, and various error metrics, are discussed in 2.6.3.1 and 2.6.3.2. The goal of this evaluation is to identify which filter provides the best overall control performance, ensuring the system responds quickly and accurately while minimizing errors.

Based on the analysis of the performance metrics for the three filters, the EMA

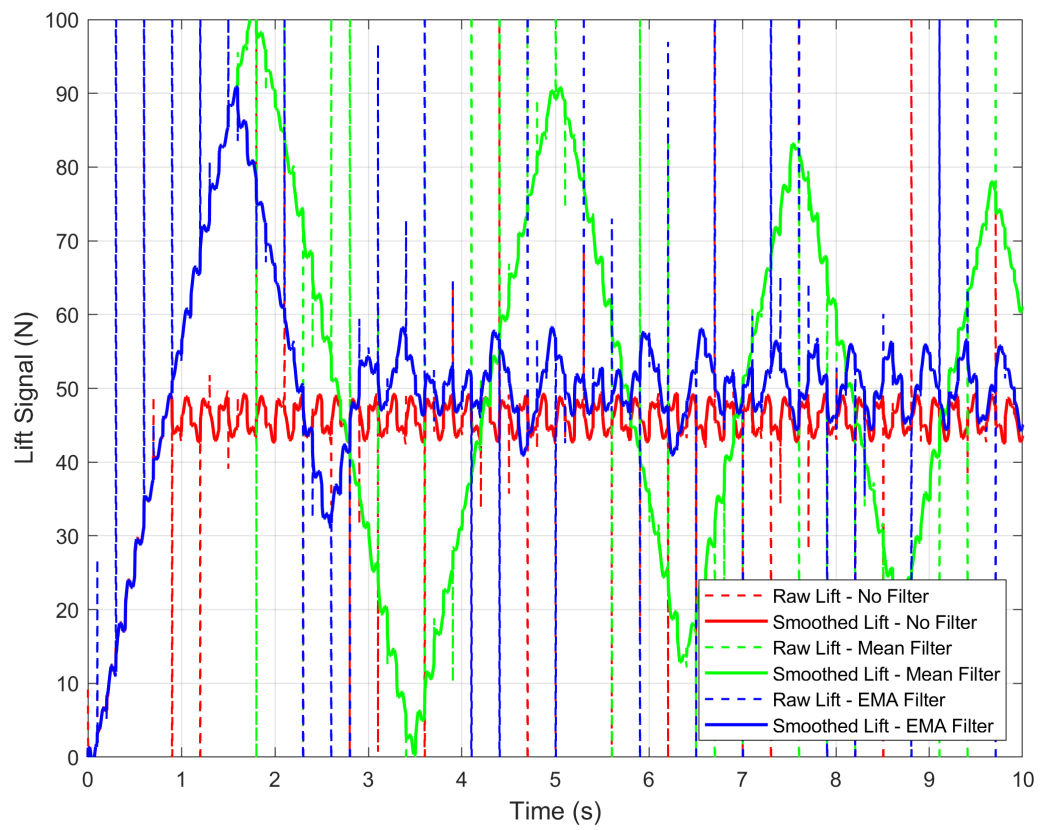


Figure 4.15: Lift Force vs Time

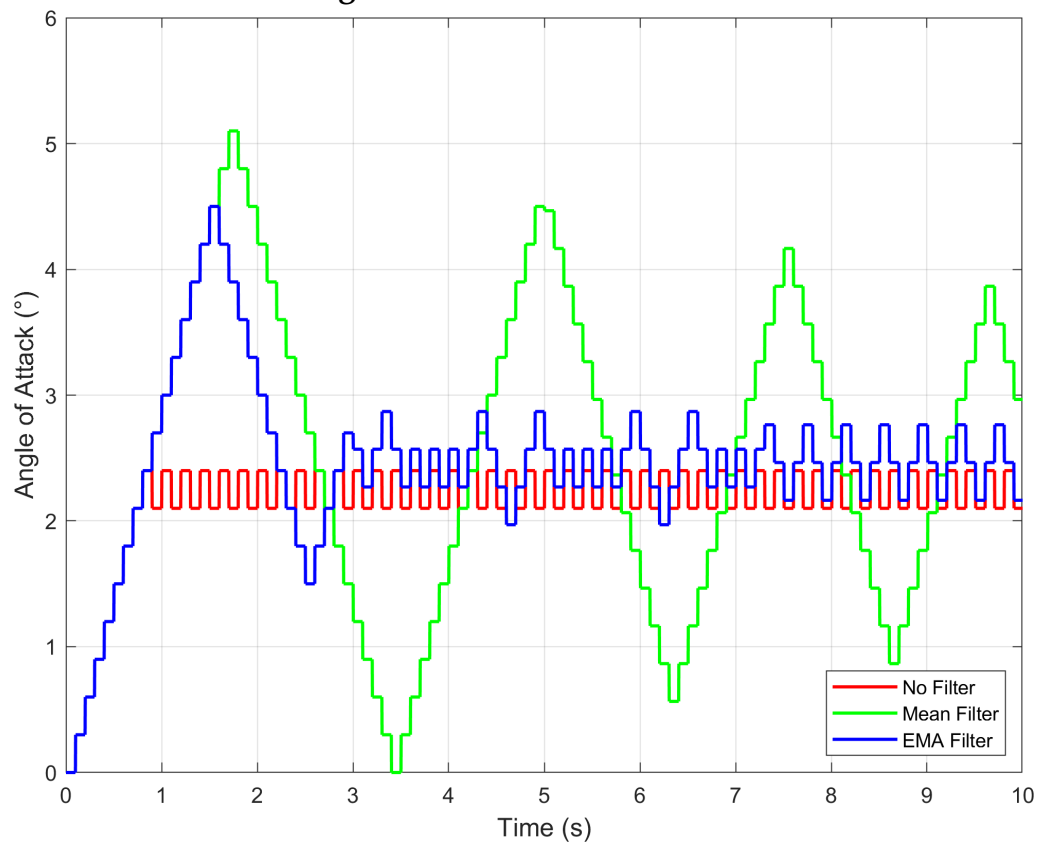


Figure 4.16: AoA vs Time

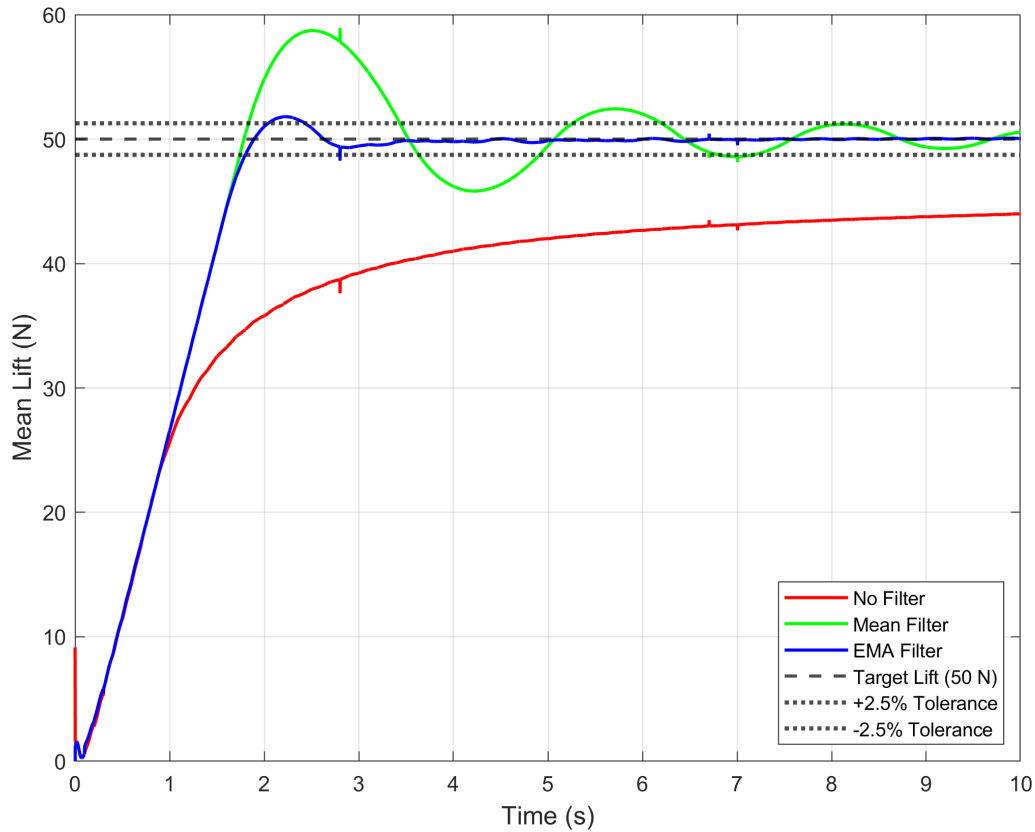


Figure 4.17: Mean Lift vs Time

Filter	Rise Time (s)	Overshoot (%)	Settling Time (s)	IAE	ITAE	RMSE
No Filter	N/A	-12.03	N/A	120.57	395.84	15.19
Mean Filter	1.62	17.84	7.18	66.23	105.63	12.88
EMA Filter	1.62	3.61	2.80	49.10	34.58	12.55

Table 4.4: Performance Metrics for Different Filters

filter emerges as the most suitable for the control objective. Initially, the No Filter approach shows poor control performance. The rise time is undefined due to an issue in the data where the lift does not reach 90% of the target, which results in an absence of a measurable rise time. Additionally, the overshoot is negative, indicating that the raw lift data consistently undershoots the target lift value, and the settling time remains undefined as the signal fails to stabilize within the acceptable tolerance range. The higher values for IAE, ITAE, and RMSE further reinforce the inadequate control performance of the No Filter approach.

In contrast, the Mean Filter provides a more reasonable response. It achieves a rise time of 1.62 seconds, which is a good indication of a timely approach towards the target lift. However, the overshoot at 17.84% is notably higher than that of the EMA filter, leading to more significant deviation from the target lift. The settling time of the Mean Filter is 7.18 seconds, which is slower compared to the EMA filter, indicating that the system takes longer to stabilize. Additionally, while the IAE and ITAE values are lower than those of the No Filter, they are still relatively

higher than those of the EMA filter, with the RMSE also showing a slightly higher error.

The EMA Filter demonstrates superior control performance overall. It matches the Mean Filter in rise time with a slight deviation, achieving 1.62 seconds, but it significantly outperforms the Mean Filter in several key areas. The overshoot is minimal at just 3.61%, suggesting better control accuracy and less deviation from the target lift. The settling time is the quickest among the filters, at only 2.80 seconds, indicating that the EMA filter is the most responsive in bringing the system to a steady-flow. Furthermore, the EMA filter exhibits the lowest values for IAE, ITAE, and RMSE, indicating that it has the smallest integral error, time-weighted error, and root mean square error, all of which are crucial for effective control performance.

It is important to acknowledge that the selection of filter and controller gains is not a straightforward, one-size-fits-all process. The results can indeed vary significantly depending on the chosen values for these parameters. For example, adjusting the PID controller gains or filter settings can produce different outcomes in terms of stability and responsiveness, leading to varying performance results. This variability is particularly true when the control objectives change, such as optimizing for lift control, drag minimization, or energy efficiency.

Given this, the approach followed was to test different filter and controller gain configurations to understand how each combination influenced the system's behavior under wave-induced conditions. While the outcome may seem arbitrary at times, this strategy allows for a comprehensive exploration of the system's performance across a range of parameter settings. The rationale behind this approach is to identify a set of parameters that provide satisfactory performance in a given scenario, rather than to expect a singular optimal configuration. Thus, conducting such a sensitivity study is key to gaining insight into the robustness of the control system and how various configurations influence the overall performance.

4.2.3.2 Optimization of Filter Parameters

In this section, the optimization of the Exponential Moving Average (EMA) filter parameters is addressed, as it was selected in the previous analysis for reducing noise in the lift signal. The EMA filter is particularly useful in filtering out high-frequency noise while retaining the essential dynamics of the lift signal. The filter works by assigning weights to the past lift values, with more recent values being given higher weights. This dynamic weighting system allows the filter to be more responsive to recent changes in the lift force, while still smoothing out older, less relevant data. The parameters used, such as the PID gains and other settings, are detailed in the code provided for this analysis in A.7.

The selection of the smoothing factor α in the Exponential Moving Average (EMA) filter significantly influences the trade-off between noise reduction and system responsiveness. Higher values of α prioritize recent data, making the filter more responsive to fluctuations but more susceptible to noise. Conversely, lower values

Alpha Value	IAE for Front Smoothed Data	IAE for Rear Smoothed Data
0.1	1238.85	171.62
0.3	789.10	43.19
0.5	778.30	43.06
0.7	785.03	61.21
0.9	787.82	91.25

Table 4.5: IAE Values for Different Alpha Values (Smoothed Data)

of α weigh past data more heavily, resulting in smoother signals but slower responsiveness. Optimizing α is therefore critical to ensure that noise is effectively suppressed while maintaining sufficient responsiveness for the PID controller to track the target values accurately.

To determine the optimal α , simulations were conducted with two independent PID controllers in a dual-hydrofoil setup, one for each hydrofoil, where the front hydrofoil aimed to achieve 90% of the target lift (90 N) and the rear hydrofoil aimed to achieve 10% of the target lift (10 N). The lift data was sampled at PID control intervals rather than at every simulation timestep, reducing data size and simplifying the analysis. This approach focused on capturing the overall trends in the system's behavior while avoiding the inefficiencies associated with processing an overly large dataset. As a result, the raw lift data displayed linear interpolation between sampled points rather than smooth curves as seen in the Figure B.3. The data was smoothed to enhance clarity and facilitate better analysis and visualization, as illustrated in Figure 4.18.

The performance of various α values (0.1, 0.3, 0.5, 0.7, and 0.9) was analyzed in terms of integrated absolute error (IAE), which quantifies the deviation of the lift from its target value over time. For low α values, such as 0.1, the lift data appeared smoother with reduced noise. However, this came at the cost of slower responsiveness to flow condition changes, leading to delays in reaching the target lift. Higher α values, such as 0.7 and 0.9, enabled faster responses to lift variations but retained more noise, causing increased instability, particularly in the rear hydrofoil's smaller target lift.

The analysis identified $\alpha = 0.5$ as the optimal value, offering the best balance between noise reduction and responsiveness. This value resulted in the lowest IAE for both the front and rear lift data, indicating its superior ability to minimize deviations from the target lifts. Notably, $\alpha = 0.3$ performed nearly as well for the rear lift, suggesting it as a viable alternative. For α values greater than 0.5, performance decreased, particularly for the rear lift, while $\alpha = 0.1$ was less effective for both front and rear lift due to higher IAE values.

In summary, the findings demonstrate that selecting α within the range of 0.3 to 0.5 achieves a desirable balance between accuracy and computational efficiency. By reducing the dataset to PID sampling intervals, the analysis remained focused and effective while capturing the critical dynamics of the system.

4. Results

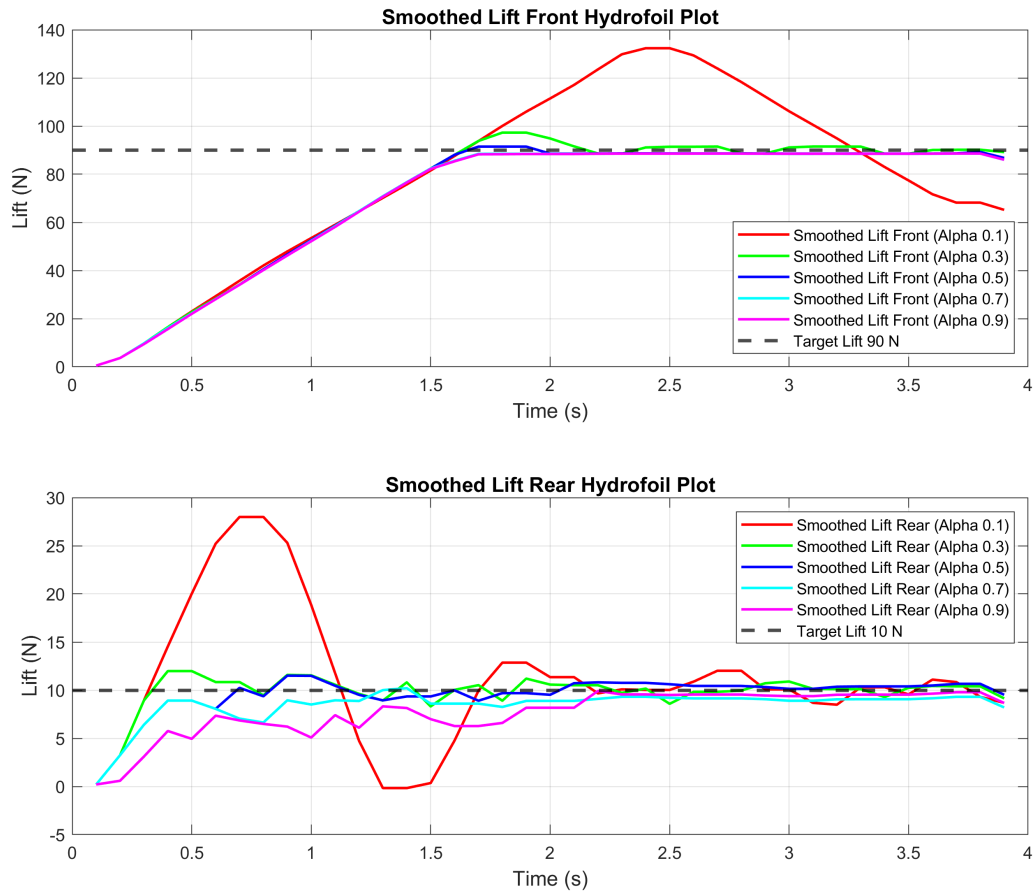


Figure 4.18: Smoothed Lift vs Time

4.2.3.3 Hydrofoil Performance at Different Depths

The depth at which a hydrofoil operates relative to the free surface plays a crucial role in its hydrodynamic performance and stability. In the semi-2D setup, various simulations were conducted to analyze how changes in hydrofoil submergence impact lift generation, flow behavior, and control response. The parameters used, such as the PID gains and other settings, are detailed in the code provided for this analysis in A.8.

Effect of Depth on Lift and Flow Characteristics

As the hydrofoil moves closer to the free surface, it experiences free-surface effects, which significantly alter the pressure distribution around the foil. These effects can lead to:

- Reduced lift generation: The proximity to the free surface modifies the pressure field, reducing the effective lift force.
- Wave-induced disturbances: Shallower depths introduce stronger wave interactions, increasing fluctuations in lift.
- Ventilation and air entrainment: At very shallow depths, air can be drawn down onto the hydrofoil, leading to lift loss and unsteady behavior.

Conversely, at greater depths, the hydrofoil operates in a more stable fluid en-

vironment with minimal wave influence, resulting in smoother lift generation. However, deeper submergence increases hydrodynamic drag, which can impact efficiency.

Implications for Hydrofoil Control

- At shallow depths, aggressive control strategies might be required to compensate for wave disturbances. However, excessive corrections can introduce instability.
- At greater depths, the control system operates in a more stable regime, reducing the need for aggressive AoA adjustments but increasing drag.
- Optimal depth selection: A balance must be achieved between minimizing wave-induced disturbances and avoiding excessive drag penalties.

Simulation Approach To understand the influence of depth on hydrofoil performance and control behavior, a series of simulations were conducted where a single hydrofoil was tasked with maintaining a target lift force of 50 N. The simulations were performed at three different depths relative to the chord length (CL) of the hydrofoil, which was 0.2 m:

- 0.5 CL (0.1 m below the free surface)
- 1.5 CL (0.3 m below the free surface)
- 3 CL (0.6 m below the free surface)

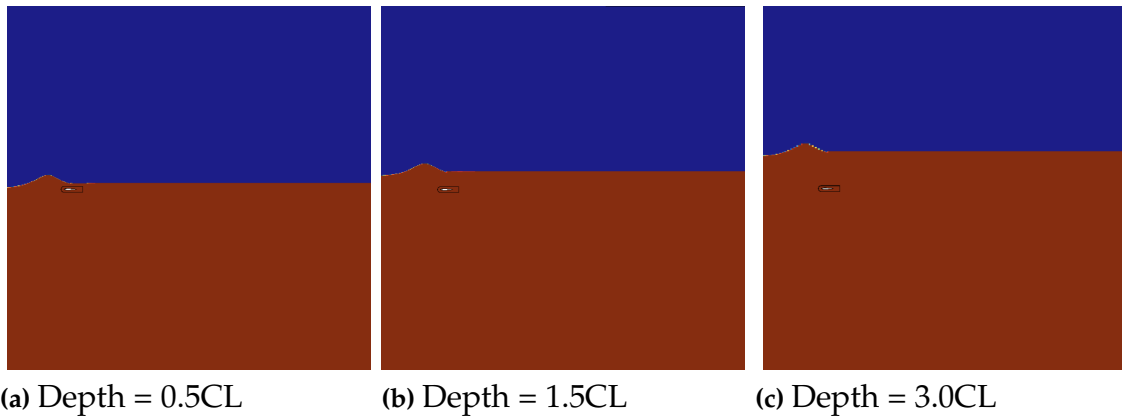


Figure 4.19: Hydrofoil at varying depths from the free surface in Density Scene

At each depth, the PID control system was responsible for adjusting the AoA to maintain the desired lift. However, to evaluate the influence of control responsiveness, four different angle limit values were imposed on the PID controller. These limits defined the maximum permissible AoA change per sampling interval, with values of 0.3° , 0.5° , 0.7° , and 1° .

Key Findings and Control Considerations

The results from the depth simulations provide insights into the interaction between hydrofoil submergence, control responsiveness, and overall lift stability. Since the sampling period remained fixed across all simulations, the variation in

angle limiter ($\text{Max}_{\Delta\text{AoA}}$) values played a crucial role in determining the system's response dynamics.

At shallower depths, where wave-induced disturbances are more dominant, higher angle limit values (e.g., 1 degree per sampling interval) enabled the PID controller to respond more aggressively. However, this also introduced significant oscillations, making the control less stable. Conversely, lower angle limits (e.g., 0.3 degrees per interval) resulted in smoother AoA corrections, but at the cost of a slower response time, leading to temporary deviations from the target lift as observed in Figure 4.20.

For intermediate depths as shown in Figure 4.22, a balance was observed, where moderate angle limits provided an optimal trade-off between responsiveness and stability. The hydrofoil at 1.5CL depth demonstrated relatively stable lift control, suggesting the idea that this depth offers a suitable compromise between minimizing wave effects and maintaining effective lift generation.

At greater depths (3.0CL) in Figure 4.24, the hydrofoil operated in a more stable flow regime with minimal disturbances from the free surface. The mean lift signal remained closer to the target lift, indicating reduced fluctuations. However, the observed gap between the target lift and mean lift signals suggested that deeper operation enhances lift generation efficiency. This improvement can be attributed to reduced free-surface interference and enhanced flow stability around the hydrofoil.

To better interpret the results and identify trends, the mean lift data was extracted using a moving window approach, similar to previous analyses. This visualization method enables one to observe that deeper submergence leads to better adherence to the target lift, as the hydrofoil operates in a more predictable flow environment.

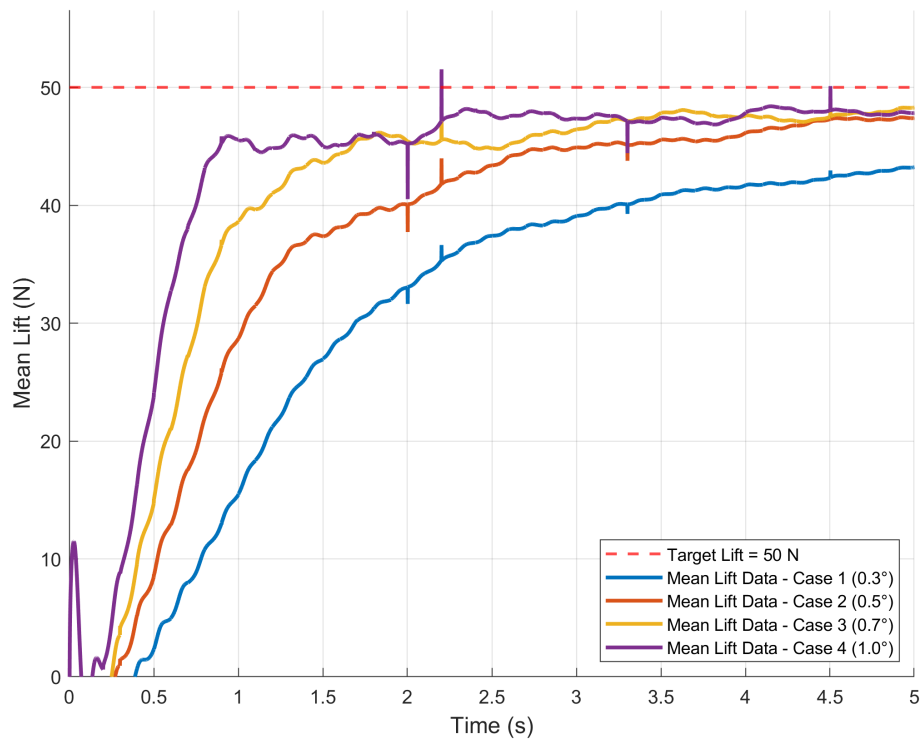


Figure 4.20: Mean Lift Data vs Time for Depth 0.5CL

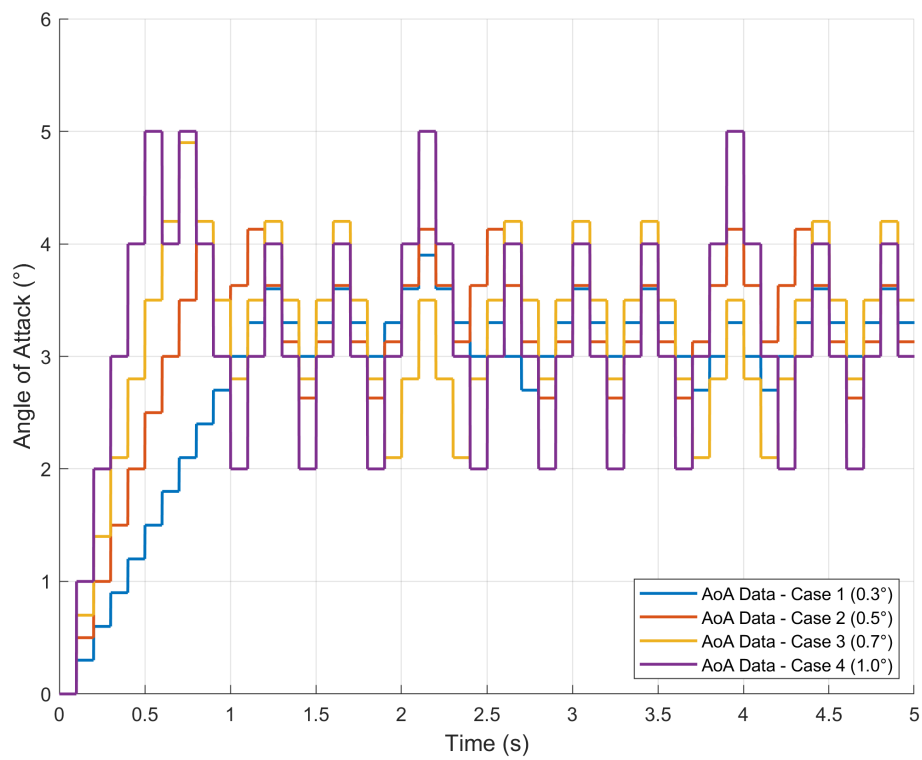


Figure 4.21: AoA vs Time for Depth 0.5CL

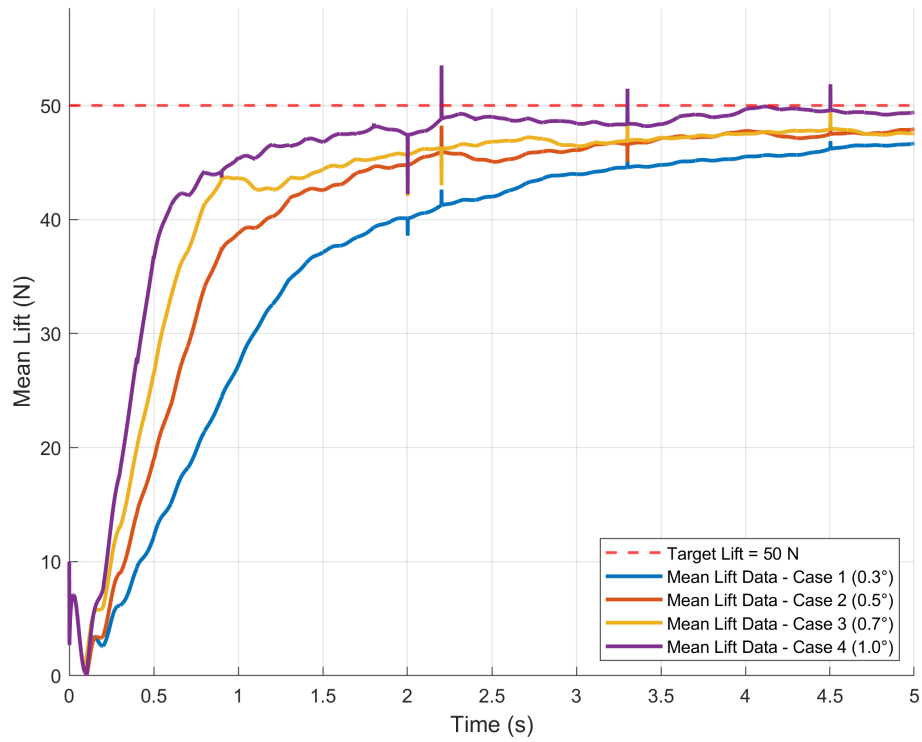


Figure 4.22: Mean Lift Data vs Time for Depth 1.5CL

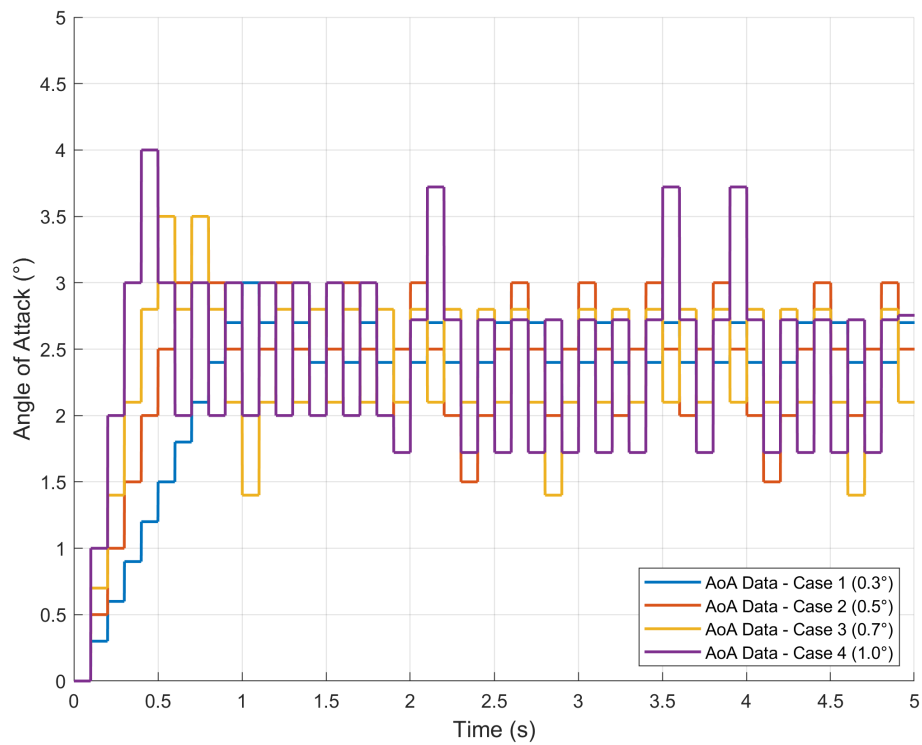


Figure 4.23: AoA vs Time for Depth 1.5CL

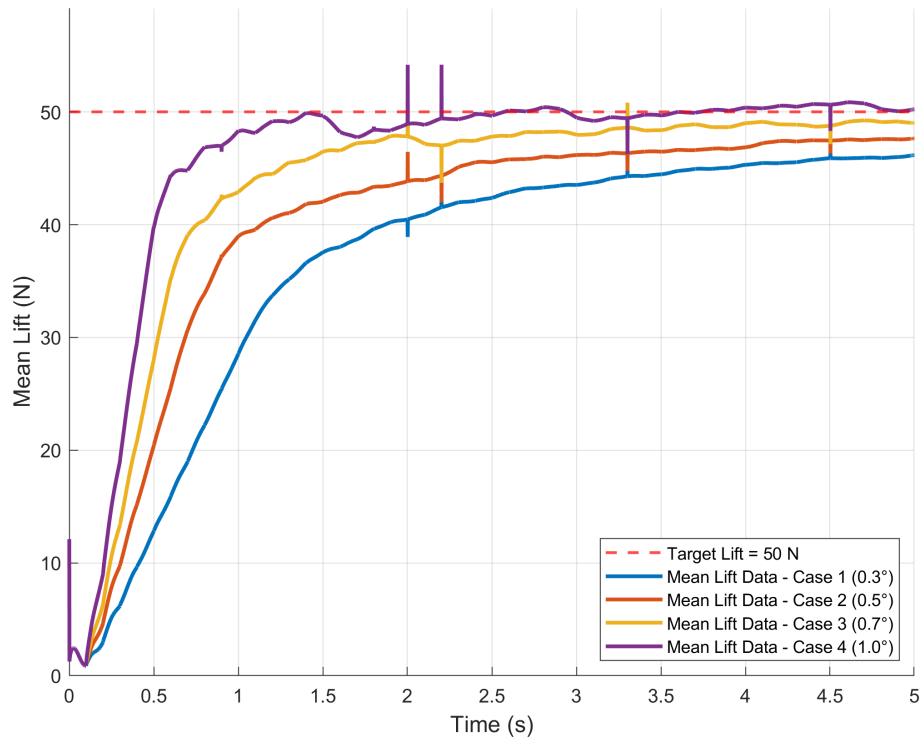


Figure 4.24: Mean Lift Data vs Time for Depth 3.0CL

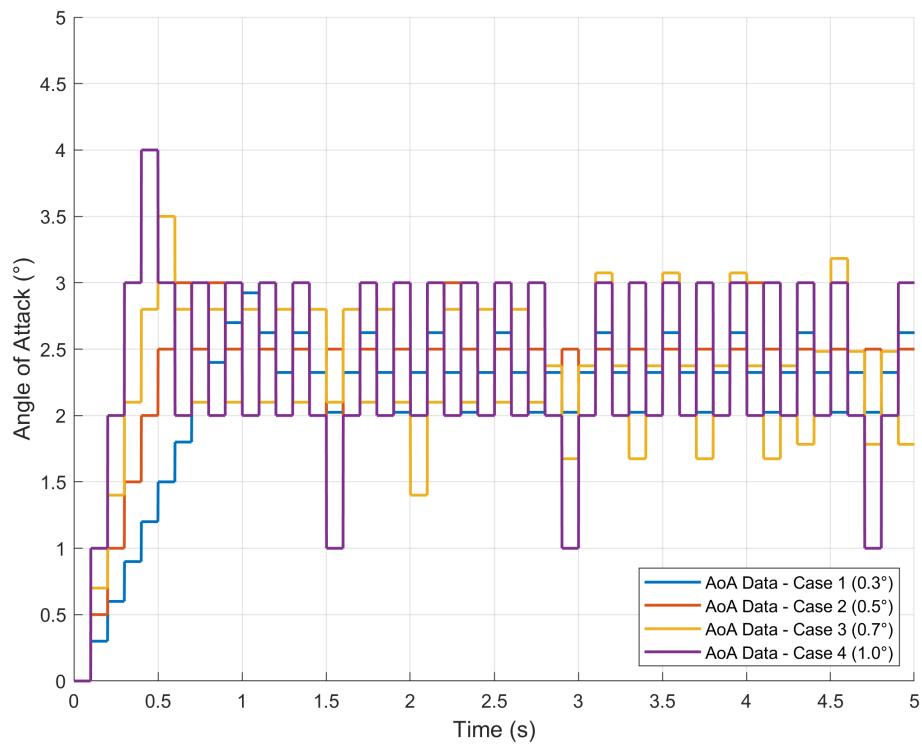


Figure 4.25: AoA vs Time for Depth 3.0CL

4.2.3.4 Ride Height and Trim Angle Control

The ride height and pitch angle control objectives were assessed under wave-induced conditions, providing valuable insights into the challenges of controlling a hydrofoil in such dynamic environments. The PID controller, designed to maintain the target ride height and pitch angle, demonstrated its ability to adjust the hydrofoil's angle of attack (AoA) and regulate the vehicle's attitude in response to wave disturbances. However, several challenges arose during the simulations, particularly in the context of the coupled simulation involving wave interactions and the semi-2D domain setup.

In a single-cell semi-2D setup, simulating wave propagation is inherently limited. Unlike in 3D simulations, waves in the semi-2D domain cannot disperse sideways, leading to the accumulation of pressure gradients within the domain. This pressure buildup is a key factor contributing to wave instability, resulting in distorted wave patterns. Furthermore, the inability to fully capture lateral energy dispersion causes significant turbulence within the computational domain. The excessive accumulation of turbulent viscosity, in turn, often led to simulation crashes, despite the PID controller functioning correctly.

Additionally, the dynamic control of the hydrofoil's AoA introduced further complexity to the simulation. The continuous adjustments made by the controller to counteract wave-induced disturbances inadvertently caused secondary disturbances that propagated towards the free surface. These disturbances amplified the existing wave patterns, further modifying the local flow field and introducing fluctuations in the ride height and pitch response.

To mitigate these effects, one possible approach is to use the standard Low-Reynolds Number (Low-RE) $k - \epsilon$ turbulence model with all $Y+$ wall treatment. This model helps minimize the growth of turbulent viscosity, thereby reducing the likelihood of turbulence-induced numerical instabilities. Additionally, extending the computational domain towards the outlet boundary proved beneficial in addressing instability issues. This extension allows for smoother wave propagation and minimizes the effects of flow reversal near the outlet, which can otherwise introduce additional numerical disturbances. By shifting the outlet boundary further downstream, the controller is less influenced by the reversed flow, leading to improved stability and control performance.

Another effective mitigation strategy involved increasing the timestep of the simulation. A larger timestep reduced the frequency of pressure and velocity field updates, which in turn mitigated excessive turbulence generation and minimized unphysical sloshing effects at the free surface. This adjustment led to smoother wave propagation and more stable wave interactions. Additionally, a larger timestep lowered computational cost by decreasing the number of iterations required for solution convergence, making the simulation more efficient. However, increasing the timestep also resulted in a lower local resolution of transient flow phenomena, particularly in regions of rapid velocity or pressure changes around the hydrofoil.

While this trade-off meant that small-scale fluctuations in the flow field were less

accurately resolved, the overall stability of the numerical solution improved. As long as the increased timestep did not introduce excessive numerical diffusion or compromise the ability of the solver to maintain physical accuracy, this approach was considered a viable compromise between computational efficiency and simulation robustness.

To further evaluate the performance of the ride height and pitch angle control, additional tests were conducted under varying conditions. Different ride height scenarios were analyzed while maintaining the same controller settings. These tests provided insights into the controller's adaptability and effectiveness in different flow conditions. However, to further enhance control performance, fine-tuning the PID parameters may be necessary. Adjusting the proportional, integral, and derivative gains based on the specific dynamic characteristics of the hydrofoil and wave interactions could lead to more refined and stable control responses.

Evaluation of Ride Height Control Across Different Operating Submergence

In this section, the performance of the ride height control system was evaluated under various submergence conditions to assess its ability to maintain the target ride height despite external disturbances. The objective was to examine how variations in operating depth influence the controller's effectiveness in achieving stable ride height regulation. Five distinct ride height scenarios were tested, where only the target ride height was modified in the macro script while keeping all PID parameters unchanged. These parameters are described in A.9.

In addition to the variations in submergence, the PID controller also encountered various challenges due to its operation within the DFBI framework. Specifically, the controller was activated only after the system release time (1 second), ensuring that the DFBI model had sufficient time to develop stable hydrodynamic interactions. During the initial ramp-up phase, the DFBI model progressively established the system's dynamic behavior, and it was observed that the full development of the model occurred approximately 4 seconds into the simulation. Only after this period did the controller take over to regulate the ride height effectively. This sequential activation was necessary to prevent premature control actions that could interfere with the natural development of the system's hydrodynamic equilibrium.

To ensure practical and physically feasible control behavior, specific constraints were applied to the hydrofoil's AoA. The hydrofoils were allowed to operate within a predefined range, with a maximum allowable angle of attack ($AoA_{MAX} = 10^\circ$) and a minimum allowable angle of attack ($AoA_{MIN} = -8^\circ$). These constraints were set based on an analysis of control performance and hydrodynamic limitations:

- **Tighter AoA constraints (narrower operating range):** If the AoA limits were further reduced, the control task could be achieved more precisely since the PID controller would have a more restricted range to operate within. However, this would also limit the controller's ability to make larger corrective actions, potentially reducing its adaptability to significant disturbances.

4. Results

- **Relaxed AoA constraints (wider operating range):** On the other hand, if the AoA limits were increased beyond this range, the hydrofoil could reach excessive angles of attack, leading to the risk of flow separation and stall, which would severely degrade performance. Therefore, the selected AoA constraints provided a balance between precise control and avoiding flow instability.

In addition to these constraints, another critical parameter was the **maximum permissible change in AoA per control update**, denoted as $\text{Max}_{\Delta\text{AoA}}$. This parameter defines the upper bound on how much the PID controller is allowed to modify the AoA in a single sampling. The primary reason for this constraint is to **prevent non-physical, abrupt changes** in the hydrofoil's angle, which could cause unrealistic transient behaviors or numerical instabilities in the simulation.

- **No lower limit on AoA change:** The minimum allowable change in AoA per step is inherently determined by the PID controller based on the magnitude of the control error. If the error is small, the PID naturally makes smaller corrections, ensuring smooth adjustments.
- **Necessity of an upper limit:** Without a constraint on the maximum AoA change, the controller might attempt aggressive corrections, leading to sudden and large variations in hydrofoil orientation. This could introduce unnecessary fluctuations in lift and ride height, destabilizing the system. By enforcing $\text{Max}_{\Delta\text{AoA}}$, the system maintains realistic control adjustments while preventing unintended oscillatory behavior.

The combined effects of these constraints ensured that the ride height control system remained both effective and stable across different depths.

Ride Height (m)	Mean Ride Height (m)	Std Dev (m)	Mean Trim Angle (°)	Std Dev (°)	Mean AoA (Front/Bow) (°)	Mean AoA (Rear/Stern) (°)
0.3	0.152	0.162	0.422	1.874	5.373	1.438
0.2	0.080	0.149	0.409	1.627	4.675	0.737
0.1	0.005	0.136	0.295	1.352	4.464	0.568
0.0	-0.087	0.121	0.279	1.510	4.749	1.075
-0.1	-0.172	0.133	0.377	1.579	3.935	0.883
-0.2	-0.260	0.145	0.368	1.495	3.726	0.571

Table 4.6: Data Summary for Hydrofoil Ride Height Control System

A detailed analysis of the data is provided in Table 4.6. The subsequent sections provide a deeper examination of these parameters, with particular focus on the extreme ride heights, including their influence on hydrofoil performance and the overall stability of the system.

The analysis will cover the variations in ride height control effectiveness, the ad-

justments made to the trim angle, and the corresponding AoA changes required for optimal lift generation. Special attention is given to the trends in Mean Lift distribution for both front and rear hydrofoils, as they play a pivotal role in maintaining the desired ride height and pitch stability, especially under extreme submergence conditions.

Ride Height Stability

The mean ride height values remain close to the target, with room for improvement, indicating that the control system effectively regulates vertical positioning while leaving potential for further optimization. However, deviations from the setpoints suggest hydrodynamic disturbances and control limitations. As observed in Table 4.6, at $0.3m$ (near the free surface) shown in Figure 4.26, the system achieves an average ride height of $0.152m$, undershooting the target due to increased wave-induced disturbances and free surface effects. In contrast, at $-0.2m$ (deepest setting) shown in Figure 4.27, the mean ride height is $-0.260m$, slightly lower than intended, likely due to increased water pressure and delayed hydrodynamic response.

Additionally, at deeper submergence levels, the standard deviation of ride height increases due to several hydrodynamic factors. The higher water density and added mass effect at greater depths increase system inertia, making it slower to respond to control inputs and more prone to oscillations. Furthermore, the absence of free surface interactions reduces natural damping, allowing oscillations to persist longer. The delayed hydrodynamic response due to the stronger fluid forces also introduces control lag, leading to overshooting and oscillatory behavior. Moreover, changes in the lift curve slope at deeper depths can result in nonlinear lift variations, causing further fluctuations in ride height.

From these observations, it becomes evident that defining a stable operational submergence range for the hydrofoil is crucial to maintaining effective control performance. The initial submergence level used in this study appears to be a good fit, as it offers a balance between free surface effects and deeper water stability. However, the optimal submergence depth should be carefully assessed to ensure that hydrodynamic stability, control responsiveness, and overall system efficiency are maintained. Furthermore, the hull dimensions and placement relative to the hydrofoils may need to be redefined to ensure that the induced flow interactions do not negatively impact control performance. A refined hull-hydrofoil configuration could help minimize undesirable pressure fluctuations and enhance the hydrofoil's ability to maintain a steady ride height. Future studies should explore adaptive control mechanisms or variable-depth operational strategies to further optimize hydrofoil performance across different submergence depths.

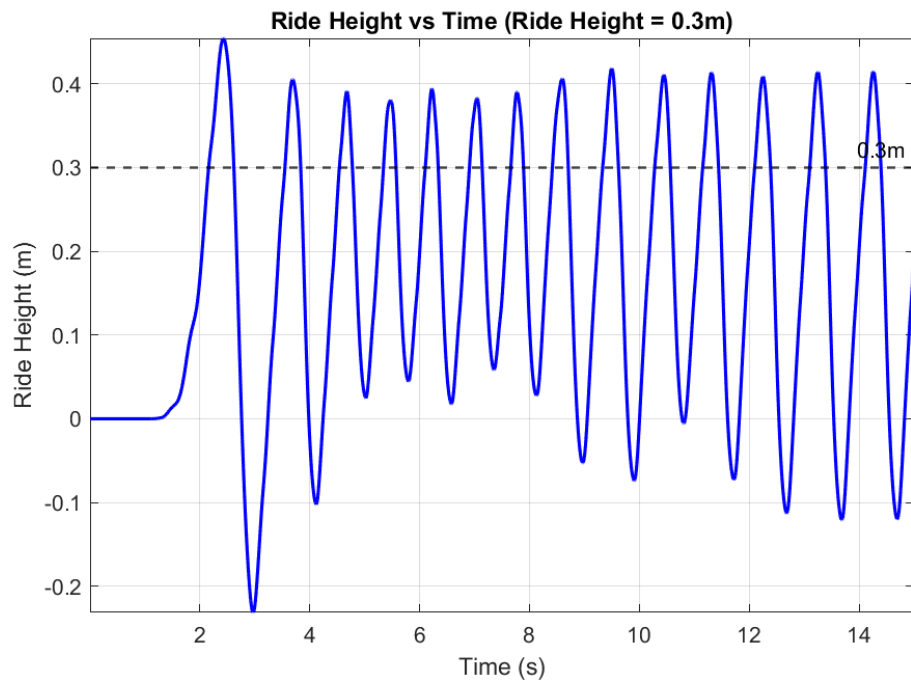


Figure 4.26: Ride Height vs Time for $RideHeight = 0.3m$

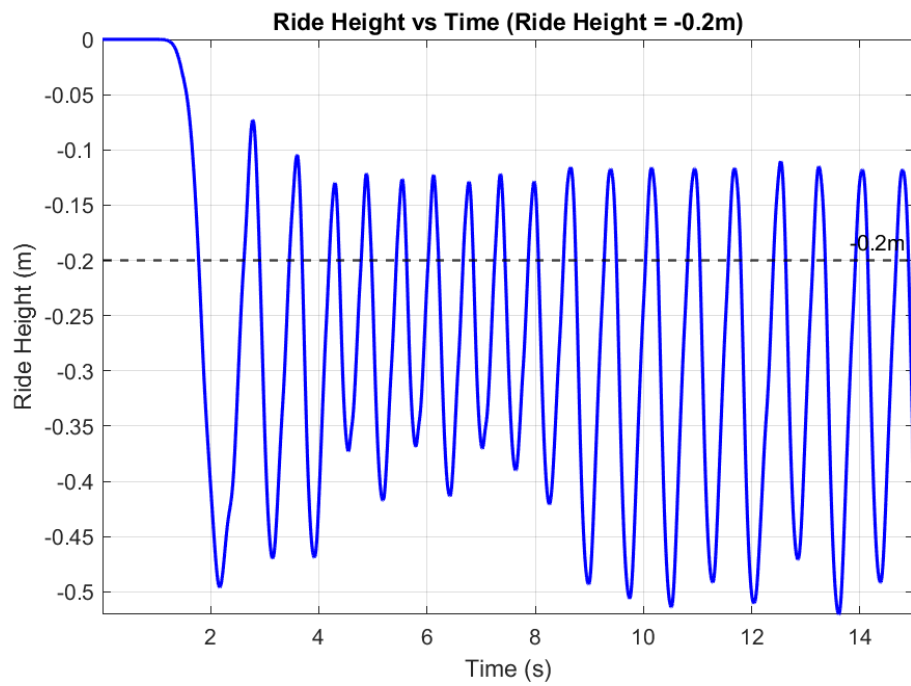


Figure 4.27: Ride Height vs Time for $RideHeight = -0.2m$

Trim Angle Performance

The mean trim angle remains within a narrow range (0.279° to 0.422°), suggesting that the pitch control is well-regulated. However, at $0.3m$, the standard deviation is highest ($\sigma = 1.874^\circ$), indicating greater oscillations in pitch stability. This can be attributed to dynamic wave interactions and unsteady forces acting near the free surface. Conversely, at $-0.2m$, the standard deviation is lower ($\sigma = 1.495^\circ$), as deeper operation reduces wave-induced disturbances, resulting in smoother trim control. The trim angle control for ride heights of $0.3m$ and $-0.2m$ is shown in Figures 4.28 and 4.29.

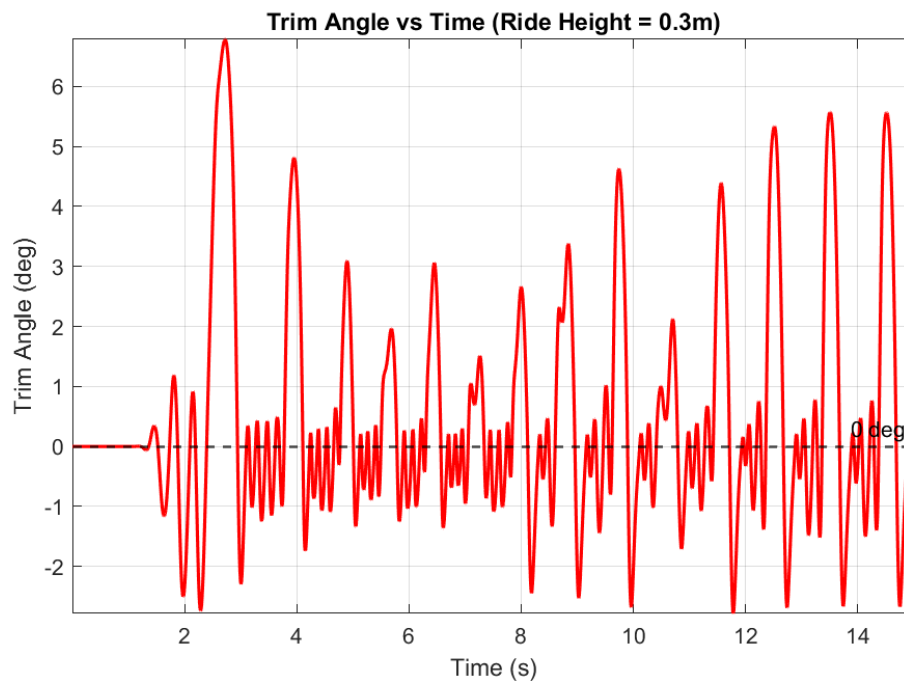


Figure 4.28: Trim Angle vs Time for *RideHeight* = $0.3m$

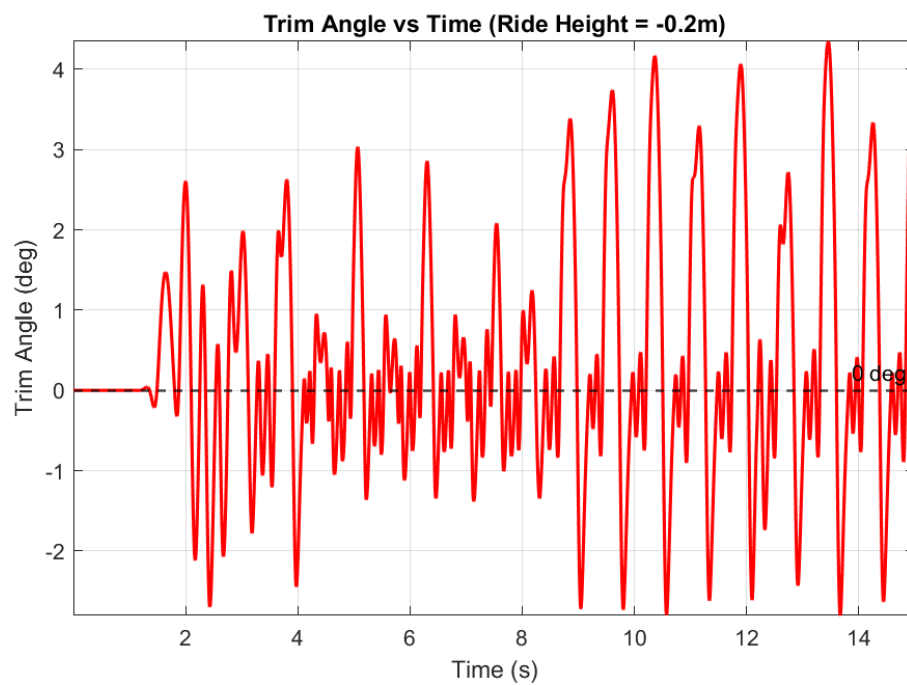


Figure 4.29: Trim Angle vs Time for $RideHeight = -0.2m$

Hydrofoil Angle of Attack (AoA) Adjustments

The front hydrofoil AoA increases with ride height, peaking at 5.373° for $0.3m$, which is expected since higher lift is required to sustain altitude near the free surface. The rear hydrofoil follows a similar trend but with smaller AoA adjustments due to its secondary role in lift generation. In deeper water ($-0.2m$), the AoA values are lower (3.726° for the front hydrofoil and 0.571° for the rear), reflecting reduced lift demands in denser fluid conditions.

Additionally, the hydrofoil adjustments (i.e., the PID controller output) frequently reach the predefined limits of AoA_{MAX} and AoA_{MIN} , indicating that the PID controller attempts to reach the target ride height as quickly as possible. This behavior is particularly evident in the front hydrofoil seen in Figures 4.30 and 4.31, which primarily governs ride height by generating the necessary lift. Consequently, the front hydrofoil exhibits a stepwise variation in AoA, transitioning between 10° and -8° , highlighting the controller's aggressive response in high-demand conditions.

This stepping behavior is influenced by the $Max_{\Delta AoA}$ parameter, which defines the maximum allowable change in AoA per control update. If this value is small, the hydrofoil gradually adjusts its angle in a smooth, ramp-like manner, leading to slower response times. Conversely, if $Max_{\Delta AoA}$ is large, the hydrofoil exhibits abrupt changes in AoA, resulting in step-like transitions. This effect is evident in the oscillations observed in the ride height response, where rapid AoA adjustments introduce transient disturbances in the system.

One might argue that tuning the derivative gain of the PID controller could mitigate these oscillations by dampening excessive corrective actions. However, in practice, the body inertia also plays a significant role in the system's response. The vehicle structure requires time to adapt to new hydrofoil configurations, meaning that even with optimal derivative gain tuning, achieving an instantaneous correction in ride height is physically challenging.

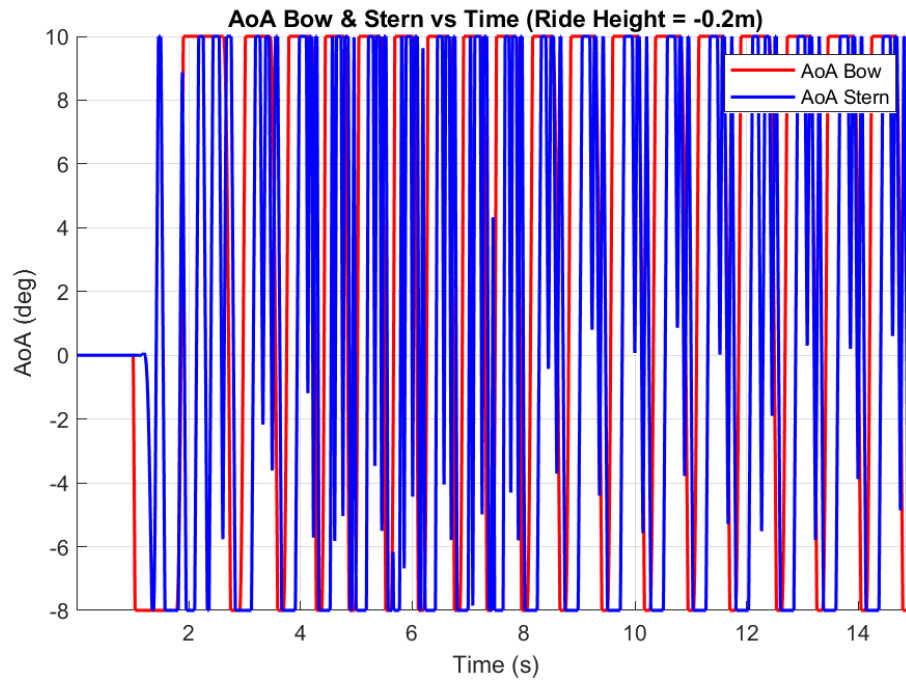


Figure 4.30: AoA vs Time for $RideHeight = 0.3m$

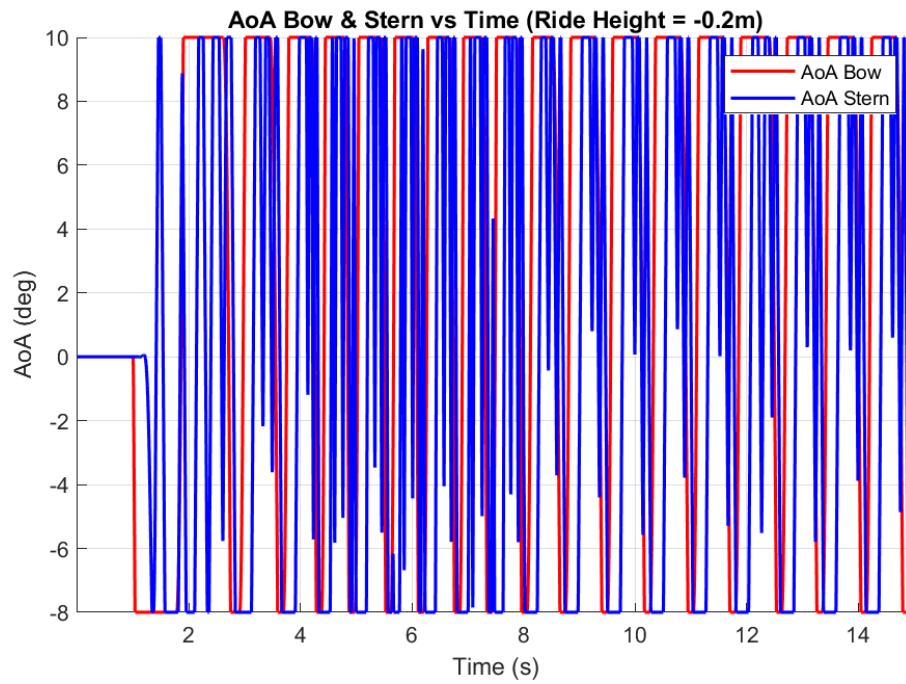


Figure 4.31: AoA vs Time for $RideHeight = -0.2m$

Analysis of Mean Lift Distribution Across Ride Heights

The lift forces on the front and rear hydrofoils exhibit a distinct trend as the ride height changes, highlighting key hydrodynamic effects and control system responses. The front hydrofoil, responsible for the majority of lift generation, shows an increasing trend in mean lift with ride height. At deeper submergence levels ($-0.2m$ and $-0.1m$), the lift is relatively lower due to increased hydrostatic pressure and fluid inertia, which dampen lift response and slow down control adjustments. As the hydrofoil moves closer to the free surface, the lift increases significantly, reaching a peak of approximately $95N$ at $0.3m$. This is attributed to the increased pressure differential near the free surface, which enhances lift production but also introduces unsteady effects.

The rear hydrofoil, primarily responsible for pitch stabilization, exhibits much lower lift values, following a similar increasing trend with ride height. At deeper depths, the rear hydrofoil generates minimal lift, as its contribution to vertical force is secondary. However, as the ride height increases, the rear hydrofoil produces slightly higher lift, peaking at approximately $1.39N$ at $0.3m$. This suggests that greater pitch control effort is required at higher ride heights, where destabilizing forces become more pronounced. The increase in lift from deeper submergence to near the free surface can be attributed to changes in hydrodynamic pressure distribution, as well as the hydrofoil's increased effectiveness in generating lift due to reduced fluid density gradients.

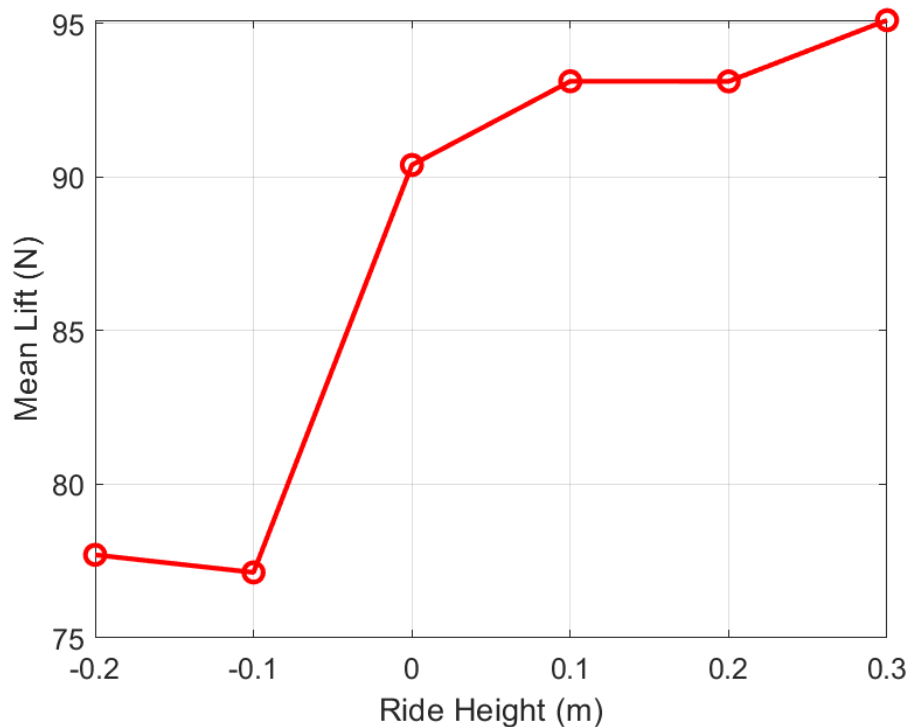


Figure 4.32: Mean Lift vs Ride Height for Front Hydrofoil

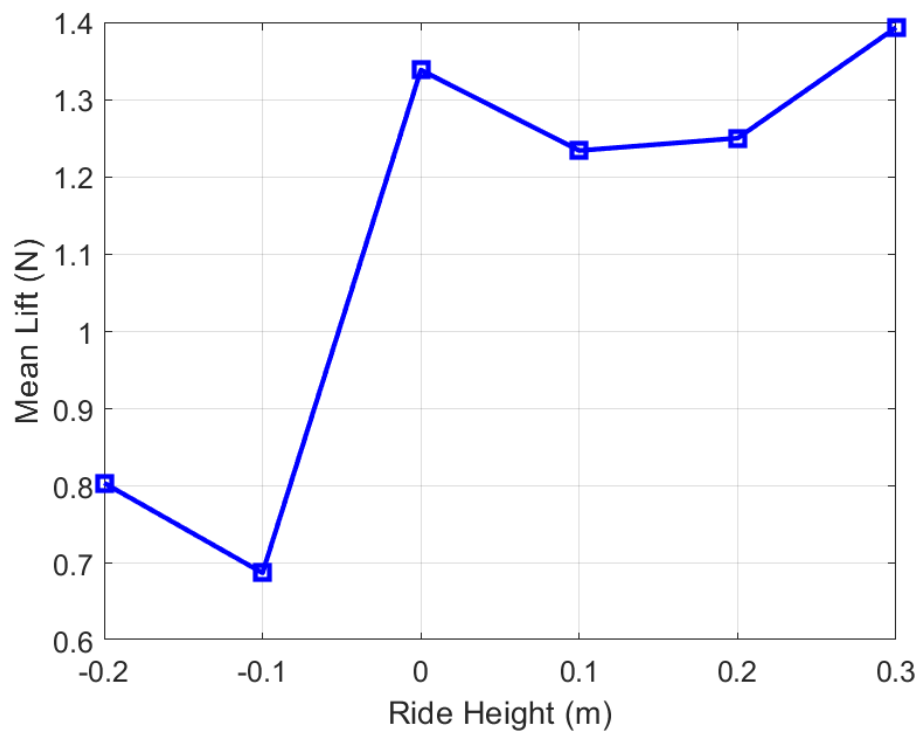


Figure 4.33: Mean Lift vs Ride Height for Rear Hydrofoil

Inference and Areas for Improvement

The hydrofoil control system performs well in maintaining ride height and trim angles, but there are opportunities for optimization, especially in extreme operational conditions.

- **Free Surface Effects:** At shallow ride heights, wave-induced disturbances lead to instability in ride height and trim angle. Adaptive control strategies, such as model predictive control, could better account for dynamic surface interactions and improve system stability.
- **Hydrofoil AoA Adjustments:** The stepping behavior observed in the front hydrofoil AoA results in oscillations and overshooting. Smoother transitions in AoA adjustments could be achieved by refining the PID controller, which would reduce transient disturbances and improve overall stability.
- **Lift Distribution:** The rear hydrofoil generates less lift at deeper submergence depths. Optimizing hydrofoil geometry and refining placement relative to the hull could improve lift-to-drag performance, particularly in deeper waters, enhancing system efficiency and reducing fluctuations.

Visualization of Ride Height Simulations

To better illustrate the ride height variations and the corresponding hydrofoil responses, animations were generated from the simulation results in STAR-CCM+. These animations visualize the density field and hydrodynamic interactions as the system evolves over time. The density-based visualization provides insights into the free surface effects and wave formation around the hydrofoils at different submergence levels. The links to the respective animations for each ride height scenario are provided in Table 4.7.

Ride Height (m)	Animation Link
0.3	Link
0.2	Link
0.1	Link
0.0	Link
-0.1	Link
-0.2	Link

Table 4.7: Links for animations of ride height simulations, visualizing the density field from STAR-CCM+

5

Conclusion

This research focused on the hydrodynamic behavior of foiling craft and the development of a PID-based control system aimed at ensuring hydrofoil stability while optimizing ride height and trim angle. The study successfully integrated CFD simulations with real-time control algorithms, allowing for a comprehensive analysis of the dynamic behavior of hydrofoils under varying operational conditions. The results obtained from the coupled CFD-PID simulations demonstrated the controller's ability to regulate the hydrofoil's AoA effectively, achieving the desired lift force and maintaining stability under steady-flow conditions. However, several challenges emerged, particularly in dynamic environments influenced by wave effects and unsteady inlet conditions.

The effectiveness of the PID controller was evident in scenarios where the flow conditions remained stable. The system successfully tracked target lift forces and adjusted the hydrofoil's AoA accordingly, ensuring a controlled lift. However, under more complex conditions, such as sinusoidal inlet velocity profiles and wave-induced disturbances, oscillations and fluctuations were observed in the system response. This indicated that while the PID control strategy was well-suited for steady environments, it struggled to maintain stability in dynamic and rapidly changing flow conditions. The stepping behavior in AoA adjustments, driven by the discrete nature of the control updates, further contributed to overshooting and oscillatory tendencies, suggesting the need for refined tuning and smoother control transitions.

As the study advanced to a semi-2D simulation framework, one of the key limitations encountered was the computational domain's inability to allow lateral energy dispersion of waves. This constraint led to significant pressure buildup, wave instabilities, and excessive turbulence viscosity, occasionally resulting in numerical instability and simulation failures. To address these challenges, several mitigation strategies were implemented. Extending the computational domain downstream improved wave propagation and reduced the likelihood of flow reversal near the outlet, which had previously introduced additional disturbances affecting control stability. Additionally, increasing the simulation timestep helped suppress excessive turbulence generation and unphysical sloshing effects. However, this approach presented a trade-off, as it lowered the resolution of transient flow phenomena around the hydrofoil, necessitating a careful balance between computational efficiency and accuracy.

The study further investigated the impact of hydrofoil submergence on ride

height control performance. By evaluating different ride heights while maintaining constant PID parameters, the controller's adaptability to varying depths was assessed. The findings highlighted that at shallower depths, wave-induced disturbances had a pronounced impact, necessitating higher AoA limits for responsiveness. However, this also introduced control oscillations, reducing overall stability. Intermediate submergence depths provided a balance between responsiveness and stability, whereas deeper submergence minimized free-surface effects and led to improved lift stability, albeit with some deviations between the mean and target lift values.

A crucial aspect of the control implementation was the imposition of constraints on the hydrofoil's AoA. The predefined operational range of -8° to 10° was chosen to prevent excessive deflections that could lead to flow separation and stall. Additionally, a maximum allowable change in AoA per control update was enforced to ensure smooth adjustments and prevent abrupt shifts that could destabilize the system. The results indicated that while this constraint helped maintain numerical stability, it also introduced oscillations in the AoA response, particularly in steady-flow conditions. A trade-off was evident between responsiveness and stability, where a higher limit improved lift tracking but increased oscillations, whereas a lower limit provided smoother control at the cost of slower corrections.

While the PID controller demonstrated effective control performance under steady and controlled conditions, its limitations became evident when exposed to the complexities of non-linearities arising from wave interactions, the DFBI framework, and other numerical aspects of the CFD simulation. The controller showed promising results in maintaining hydrofoil stability to some extent. However, the dynamic nature of wave-induced disturbances and the inherent complexities of the DFBI model, which introduces additional non-linear forces, led to performance degradation.

The PID control algorithm, designed primarily for linear systems, struggled to maintain stability in these more intricate environments. The delayed activation of the controller, which was intended to allow for the hydrodynamic equilibrium to stabilize within the DFBI framework, highlighted the need for more sophisticated control strategies. The PID controller's inability to robustly handle unsteady, non-linear flows and interactions resulted in suboptimal performance, particularly when the system encountered turbulence or other complex flow dynamics. This limitation underscores the fact that while PID controllers excel in simpler, linear contexts, they may not be sufficient for the intricacies of real-world, non-linear marine environments.

Ultimately, this research provides a structured methodology for integrating CFD simulations with control strategies, offering valuable insights into the real-time control of hydrofoils. While the current approach demonstrates the feasibility of PID-based control in marine applications, further refinements in control strategy, turbulence modeling, and hydrofoil geometry optimization are required to enhance overall performance and robustness.

5.1 Future Scope of Research

This research opens several areas for future exploration in hydrofoil-based control systems and marine hydrodynamics. While the current study provided insights into hydrofoil dynamics and PID-based control, further advancements can be made to enhance accuracy, robustness, and real-world applicability. The key areas for future research include:

- **Advanced Control Algorithms:** Future studies could explore adaptive control, model predictive control (MPC), or sliding mode control to improve the system's responsiveness to unsteady flow conditions. These advanced control strategies could enhance disturbance rejection and system stability, particularly in complex marine environments with fluctuating wave conditions and turbulence effects.
- **Hydrofoil Geometry Optimization:** Refining the hydrofoil's lift-to-drag ratio at different submergence depths would enable more stable lift generation while reducing hydrodynamic resistance. This is particularly relevant for deep-water operations where fluid density variations and turbulence effects significantly impact hydrofoil performance.
- **Multiphase Flow and Turbulence Modeling:** Improved CFD models are required to accurately capture air-water interactions, particularly under turbulent or wave-induced conditions. Enhanced modeling approaches would improve the accuracy of lift force predictions and hydrofoil stability assessments, ensuring better performance in real-world applications.
- **Transition to 3D Simulations:** A key limitation of the current study was the use of a semi-2D model, which restricted lateral flow development and energy dispersion. Future research should extend the simulations to a fully 3D model to provide a more realistic representation of hydrofoil dynamics. A 3D model would introduce three independent control surfaces, enabling precise regulation of: Ride height (heave), Pitch, Roll. The additional degrees of freedom would allow for more advanced control strategies and a deeper understanding of hydrofoil behavior under real-world operating conditions.
- **FMU Coupling for CFD-Control Integration:** The potential for Functional Mock-up Unit (FMU) coupling should be explored as an alternative method to validate whether the current Java macro-based CFD-controller coupling is the optimal way to integrate real-time control with CFD simulations. FMU coupling could improve computational efficiency, simulation stability, and the accuracy of real-time feedback control in a coupled hydrofoil system.

In conclusion, while the current study provides valuable insights into the potential of hydrofoil-based control systems for ride height and trim angle regulation, several areas for improvement remain. The integration of advanced control algorithms, optimization of hydrofoil geometry, and the development of real-time adaptive systems offer promising pathways for future research. With further re-

5. Conclusion

finements, these innovations could lead to the development of more efficient, stable, and adaptable hydrofoil-based vessels, paving the way for the next generation of high-performance, energy-efficient marine craft.

Bibliography

- [1] Tamer M Ahmed, Yousri Welaya, and Serag M Abdulmotaleb. "Numerical modeling of the hydrodynamic performance of hydrofoils for auxiliary propulsion of ships in regular head-waves". In: *International Conference on Offshore Mechanics and Arctic Engineering*. Vol. 57786. American Society of Mechanical Engineers. 2017, V010T09A015.
- [2] Alan J Alexander, James L Grogono, Donald J Nigg, et al. "Hydrofoil sailing". In: (1972).
- [3] C.L. Amoroso, A. Liverani, D. Francia, and A. Ceruti. "Dynamics augmentation for high speed flying yacht hulls through PID control of foiling appendages". In: *Ocean Engineering* 221 (2021), p. 108115. ISSN: 0029-8018. DOI: <https://doi.org/10.1016/j.oceaneng.2020.108115>.
- [4] John Anderson. *EBOOK: Fundamentals of Aerodynamics (SI units)*. McGraw hill, 2011.
- [5] K. H. Ang, G. Chong, and Yun Li. "PID Control System Analysis, Design, and Technology". In: *IEEE Transactions on Control Systems Technology* 13.4 (2005), pp. 559–576. DOI: 10.1109/TCST.2005.847331. URL: <https://doi.org/10.1109/TCST.2005.847331>.
- [6] Karl Johan Åström and Tore Hägglund. *Advanced PID Control*. ISA, 2006. URL: <https://www.isa.org/products/advanced-pid-control>.
- [7] Jangwhan Bai and Yonghwan Kim. "Control of the vertical motion of a hydrofoil vessel". In: *Ships and Offshore Structures* 5.3 (2010), pp. 189–198. DOI: 10.1080/17445300903354224. eprint: <https://doi.org/10.1080/17445300903354224>. URL: <https://doi.org/10.1080/17445300903354224>.
- [8] N. Basinger. *Design and Build of a Hydrofoil Boat*. 2022. URL: <https://dspace.mit.edu/bitstream/handle/1721.1/144707/basinger-nbase-sb-meche-2022-thesis.pdf?sequence=1>.
- [9] K.A. Belibassakis. "Hydrodynamic analysis of flapping-foil thrusters operating beneath the free surface and in waves". In: *Engineering Analysis with Boundary Elements* 41 (Apr. 2014), pp. 47–59. DOI: 10.1016/j.enganabound.2014.01.008.
- [10] Eric Besnard, Adeline Schmitz, Kalle Kaups, George Tzong, Hamid Hefazi, Orhan Kural, Hsun Chen, and Tuncer Cebeci. "Hydrofoil design and optimization for fast ships". In: *ASME International Mechanical Engineering Congress and Exposition*. Vol. 15892. American Society of Mechanical Engineers. 1998, pp. 265–275.

- [11] B. Blocken and C. Gualtieri. “Ten iterative steps for model development and evaluation applied to Computational Fluid Dynamics for Environmental Fluid Mechanics”. In: *Environmental Modelling & Software* 33 (2012), pp. 1–22. ISSN: 1364-8152. DOI: <https://doi.org/10.1016/j.envsoft.2012.02.001>. URL: <https://www.sciencedirect.com/science/article/pii/S1364815212000370>.
- [12] Byunghun Choi, Youkyung Hong, Byunghyun Lee, Minhee Kim, H Jin Kim, and Chongam Kim. “Adaptive flow separation control over an asymmetric airfoil”. In: *International Journal of Aeronautical and Space Sciences* 19 (2018), pp. 305–315.
- [13] Wikipedia contributors. *Hydrofoil*. Retrieved December 27, 2024. 2024. URL: <https://en.wikipedia.org/wiki/Hydrofoil#:~:text=A%20hydrofoil%20is%20a%20lifting,are%20also%20simply%20termed%20hydrofoils>.
- [14] Wikipedia contributors. *Proportional–integral–derivative controller - Wikipedia*. December 25. 2024. URL: https://en.wikipedia.org/wiki/Proportional%E2%80%93integral%E2%80%93derivative_controller.
- [15] Wikipedia contributors. URL: https://hpc-wiki.info/hpc/HPC_Wiki.
- [16] *Control System Basics* | Ledin Engineering, Inc. n.d. URL: <https://ledin.com/control-systems-basics/>.
- [17] *Control Systems/System Metrics - Wikibooks, open books for an open world*. n.d. URL: https://en.wikibooks.org/wiki/Control_Systems/System_Metrics#Visually.
- [18] Safae Elmisaoui, Imad Kissami, and Jean-Michel Ghidaglia. “High Performance Computing to Accelerate Large-Scale Computational Fluid Dynamics Simulations: A Comprehensive Study”. In: *International Conference on Advanced Intelligent Systems for Sustainable Development (AI2SD’2023)*. Ed. by Mostafa Ezziyyani, Janusz Kacprzyk, and Valentina Emilia Balas. Cham: Springer Nature Switzerland, 2024, pp. 352–360.
- [19] John D Fenton. “A fifth-order Stokes theory for steady waves”. In: *Journal of waterway, port, coastal, and ocean engineering* 111.2 (1985), pp. 216–234.
- [20] Georg Hager and Gerhard Wellein. *Introduction to High Performance Computing for Scientists and Engineers*. CRC Press, 2010. ISBN: 978-1439811924. URL: <https://www.routledge.com/Introduction-to-High-Performance-Computing-for-Scientists-and-Engineers/Hager-Wellein/p/book/9781439811924>.
- [21] Jibran Haider. “Numerical Modelling of Evaporation and Condensation Phenomena”. PhD thesis. Aug. 2013.
- [22] Peter Heppel. “Flight dynamics of sailing foilers”. In: *5th High Performance Yacht Design Conference*. 2015, pp. 180–189.
- [23] *Hydrofoils*. n.d. URL: <https://www.flying-fish.tech/hydrofoils#calculator-wrapper>.
- [24] Carl B Jenssen, T Kvamdal, HI Andersson, B Pettersen, P Fox, N Satofuka, A Ecer, and Jacques Periaux. *Parallel computational fluid dynamics 2000: trends and applications*. Gulf Professional Publishing, 2001.
- [25] Michael A Johnson and Mohammad H Moradi. *PID control*. Springer, 2005.

-
- [26] ChangSeop Kwon, SeongMo Yeon, Dong-Jin Kim, Kunhang Yun, Yeon-Gyu Kim, and SeungHyun Hwang. "A study on PID controlled self-propulsion and turning simulations based on the URANS CFD free running approach". In: *International Journal of Naval Architecture and Ocean Engineering* 15 (2023), p. 100556.
- [27] R. Meyer and J. Wilkins. *Hydrofoil Development and Applications*. 1992. URL: <https://www.foils.org/wp-content/uploads/2017/10/0193-Hydrofoil-Development-and-Applications-Meyer-Wilkins-Jun-92.pdf>.
- [28] *New to waves. Here's everything you need to know*. n.d. URL: <https://www.nortekgroup.com/knowledge-center/wiki/new-to-waves>.
- [29] Peter Pacheco. *An Introduction to Parallel Programming*. Morgan Kaufmann, 2011. ISBN: 978-0123742605. URL: <https://www.sciencedirect.com/book/9780123742605/an-introduction-to-parallel-programming>.
- [30] Binaya Raj Pandey, Megh Raj K C, Brian Crookston, and Gerald Zenz. "Numerical Investigation of Different Stepped Spillway Geometries over a Mild Slope for Safe Operation Using Multi-Phase Model". In: *Water* 16.11 (2024). ISSN: 2073-4441. DOI: 10.3390/w16111635.
- [31] M. Reichel and A. Bednarek. *The Experimental Studies on Hydrofoil Resistance*. 2018. URL: https://www.researchgate.net/publication/257678280_The_experimental_studies_on_hydrofoil_resistance.
- [32] Jussara MS Ribeiro, Murillo F Santos, MJ Carmo, and MF Silva. "Comparison of PID controller tuning methods: analytical/classical techniques versus optimization algorithms". In: *2017 18th international Carpathian control conference (ICCC)*. IEEE. 2017, pp. 533–538.
- [33] W. C. Schultz and V. C. Rideout. "Control system performance measures: Past, present, and future". In: *IRE Transactions on Automatic Control* AC-6.1 (1961), pp. 22–35. DOI: 10.1109/TAC.1961.6429306.
- [34] P. B. Serra. *Hydrofoil Design and Construction*. 2019. URL: <https://upcommons.upc.edu/bitstream/handle/2117/172005/REPORT.pdf>.
- [35] SKYbrary Aviation Safety. *Angle of Attack (AOA)*. May 2023. URL: <https://skybrary.aero/articles/angle-attack-aoa>.
- [36] Siemens Digital Industries Software. *STAR-CCM+ User Guide*. Accessed: 2025-03-18. 2023. URL: <https://www.plm.automation.siemens.com/global/en/products/simcenter/star-ccm.html>.
- [37] University of Southampton. *Development of a Hydrofoiling Experimental Research Platform*. 2017. URL: https://foils.org/wp-content/uploads/2017/10/Innovsail_Report-Univ-of-Southampton.pdf.
- [38] Thomas Sterling, Matthew Anderson, and Maciej Brodowicz. *High Performance Computing: Modern Systems and Practices*. Morgan Kaufmann, 2017. ISBN: 978-0124201583. URL: <https://www.sciencedirect.com/book/9780124201583/high-performance-computing>.
- [39] Fred Stern, Robert V Wilson, Hugh W Coleman, Eric G Paterson, et al. "Verification and validation of CFD simulations". In: *Iowa Institute of Hydraulic Research IIHR 407* (1999).

- [40] HENRIK STRÖMQVIST. *Robust motion control strategies for hydrofoil-equipped naval vessels considering scaling effects*. 2019.
- [41] Shankar Subramaniam. “Multiphase flows: Rich physics, challenging theory, and big simulations”. In: *Phys. Rev. Fluids* 5 (11 Nov. 2020), p. 110520. DOI: 10.1103/PhysRevFluids.5.110520. URL: <https://link.aps.org/doi/10.1103/PhysRevFluids.5.110520>.
- [42] J. Sun. *Control Engineering: Fundamentals*. De Gruyter Textbook. De Gruyter, 2018.
- [43] Zhongwei Sun and Shengwei Wang. “A CFD-based test method for control of indoor environment and space ventilation”. In: *Building and Environment* 45.6 (2010), pp. 1441–1447. DOI: <https://doi.org/10.1016/j.buildenv.2009.12.007>.
- [44] Wikipedia contributors. *High-performance computing — Wikipedia, The Free Encyclopedia*. Accessed: 2024-02-12. 2024. URL: https://en.wikipedia.org/wiki/High-performance_computing.
- [45] Guan Heng Yeoh and Jiyuan Tu. “Chapter 1 - Introduction”. In: *Computational Techniques for Multiphase Flows*. Ed. by Guan Heng Yeoh and Jiyuan Tu. Oxford: Butterworth-Heinemann, 2010, pp. 1–20. ISBN: 978-0-08-046733-7. DOI: <https://doi.org/10.1016/B978-0-08-046733-7.00001-1>.
- [46] Guan Heng Yeoh and Jiyuan Tu. “Chapter 2 - Governing Equations and Boundary Conditions”. In: *Computational Techniques for Multiphase Flows*. Ed. by Guan Heng Yeoh and Jiyuan Tu. Oxford: Butterworth-Heinemann, 2010, pp. 21–94. ISBN: 978-0-08-046733-7. DOI: <https://doi.org/10.1016/B978-0-08-046733-7.00002-3>.
- [47] Guan Heng Yeoh and Jiyuan Tu. “Chapter 6 - Gas-Liquid Flows”. In: *Computational Techniques for Multiphase Flows*. Ed. by Guan Heng Yeoh and Jiyuan Tu. Oxford: Butterworth-Heinemann, 2010, pp. 351–456. ISBN: 978-0-08-046733-7. DOI: <https://doi.org/10.1016/B978-0-08-046733-7.00006-0>.
- [48] Guan Heng Yeoh and Jiyuan Tu. “Chapter 7 - Free Surface Flows”. In: *Computational Techniques for Multiphase Flows*. Ed. by Guan Heng Yeoh and Jiyuan Tu. Oxford: Butterworth-Heinemann, 2010, pp. 457–500. ISBN: 978-0-08-046733-7. DOI: <https://doi.org/10.1016/B978-0-08-046733-7.00007-2>.

A

Appendix 1

A.1 Lift Force and AoA Calculation Function

The Airfoil Calculator is a MATLAB-based function used to analytically compute the lift force or angle of attack (AoA) based on user input. Predefined parameters, such as fluid density ($\rho = 997.561 \text{ kg/m}^3$) and reference area ($S = 0.2 \times 0.02 \text{ m}^2$), simplify the setup for consistent calculations.

The user can input either:

- AoA (in degrees) to calculate the corresponding lift force, or
- Lift Force (in Newtons) to determine the AoA.

The tool provides initial insights into the aerodynamic behavior of the airfoil, aiding the interpretation of CFD simulation results. It serves as an analytical approximation, useful for understanding the basic relationship between AoA, lift, and fluid velocity, though it does not account for factors like turbulence or transient effects found in CFD simulations.

Listing A.1: MATLAB Code

```
1 function airfoil_calculator()
2     % Create a UI figure
3     fig = uifigure('Name', 'Airfoil Calculator', 'Position',
4         [100 100 400 350]);
5
6     % Create a dropdown for selecting the calculation type
7     % (Lift or AoA)
8     uilabel(fig, 'Text', 'Select Calculation Type:',
9         'Position', [20, 280, 150, 30]);
10    calc_type = uidropdown(fig, 'Items', {'Calculate Lift
11        from AoA', 'Calculate AoA from Lift'}, ...
12        'Position', [180, 285, 180, 22], 'ValueChangedFcn',
13        @(dd,event) update_inputs());
14
15    % Create a dynamic label and input field
16    dynamic_label = uilabel(fig, 'Text', 'Angle of Attack
17        (degrees):', 'Position', [20, 220, 150, 30]);
18    dynamic_input = uieditfield(fig, 'numeric', 'Position',
```

```
[180, 220, 100, 30]);

14
15 % Fixed input for flow velocity
16 uilabel(fig, 'Text', 'Flow Velocity (m/s):', 'Position',
    [20, 170, 150, 30]);
17 velocity_input = uieditfield(fig, 'numeric', 'Value', 10,
    'Position', [180, 170, 100, 30]);
18
19 % Create a button to calculate the result
20 calc_button = uibutton(fig, 'Text', 'Calculate',
    'Position', [120, 120, 150, 40], ...
21     'ButtonPushedFcn', @(btn,event)
        perform_calculation(calc_type.Value,
            dynamic_input.Value, velocity_input.Value));
22
23 % Create a label for displaying the result
24 result_label = uilabel(fig, 'Text', '', 'Position', [100,
    60, 200, 30], 'HorizontalAlignment', 'center');
25
26 % Function to update the UI based on the selected
    calculation type
27 function update_inputs()
28     if strcmp(calc_type.Value, 'Calculate Lift from AoA')
29         dynamic_label.Text = 'Angle of Attack (degrees):';
30         dynamic_input.Value = NaN; % Reset input value
31     else
32         dynamic_label.Text = 'Lift Force (Newtons):';
33         dynamic_input.Value = NaN; % Reset input value
34     end
35 end
36
37 % Function to perform the selected calculation
38 function perform_calculation(calc_type, input_value, v)
39     % Airfoil and fluid parameters
40     rho = 997.561; % Density of water in kg/m^3
41     S = 0.2*0.02; % Reference area in m^2
42     slope = 0.1; % Lift curve slope (in per degree)
43     q = 0.5 * rho * v^2; % Dynamic pressure
44
45     % Check calculation type and perform the respective
        calculation
46     if strcmp(calc_type, 'Calculate Lift from AoA')
47         % Calculate Lift based on AoA
48         AoA_deg = input_value; % Angle of Attack in
            degrees
49         Cl = AoA_deg * slope;
50         L = Cl * q * S; % Calculate Lift Force
51         result_label.Text = sprintf('Lift Force: %.2f N',
            L);
```



```

52         else
53             % Calculate AoA based on Lift
54             L = input_value; % Lift Force in Newtons
55             Cl = L / (q * S);
56             AoA_deg = Cl / slope; % Calculate Angle of Attack
57             result_label.Text = sprintf('Angle of Attack:
58                                     %.4f ', AoA_deg);
59         end
60     end
end

```

A.2 Calculation of Moments of Inertia for DFBI model

This MATLAB code is used to calculate the **moments of inertia** for a rectangular box representing the hydrofoil system. The moments of inertia are essential for analyzing the rotational dynamics of the system and are calculated about the three principal axes (x, y, and z). The system is modeled with a mass of **10 kg** and dimensions of **8 m (x-axis)**, **0.5 m (y-axis)**, and **2.2 m (z-axis)**. The standard formula for the moments of inertia of a rectangular prism is applied:

$$I_x = \frac{1}{12} \cdot m \cdot (l_y^2 + l_z^2)$$

$$I_y = \frac{1}{12} \cdot m \cdot (l_x^2 + l_z^2)$$

$$I_z = \frac{1}{12} \cdot m \cdot (l_x^2 + l_y^2)$$

where:

- m is the mass of the system,
- l_x , l_y , and l_z are the lengths along the x, y, and z axes, respectively.

Listing A.2: MATLAB Code

```

1 % Given data
2 mass = 10; % Mass in kg
3 length_x = 8; % Length along the x-axis in meters
4 length_y = 0.5; % Length along the y-axis in meters
5 length_z = 2.2; % Length along the z-axis in meters
6
7 % Calculate moments of inertia
8 I_x = (1/12) * mass * (length_y^2 + length_z^2); % MoI about
9 x-axis
10 I_y = (1/12) * mass * (length_x^2 + length_z^2); % MoI about
y-axis
I_z = (1/12) * mass * (length_x^2 + length_y^2); % MoI about
z-axis

```

```

11
12 % Display results
13 fprintf('Moment of Inertia about x-axis: %.4f kg m \n',
    I_x);
14 fprintf('Moment of Inertia about y-axis: %.4f kg m \n',
    I_y);
15 fprintf('Moment of Inertia about z-axis: %.4f kg m \n',
    I_z);

```

A.3 Code for visualizing First-order and Fifth-order wave

This MATLAB script simulates and compares first-order (linear) and fifth-order Stokes wave profiles for given wave parameters (wavelength, height, depth, etc.). It calculates wave elevation using linear and higher-order nonlinear terms, checks the validity of the fifth-order theory using the Ursell number, and plots the results. The script is useful for analyzing wave behavior in different water depths and nonlinear regimes.

Listing A.3: MATLAB Code

```

1  clc; clear; close all;
2
3  % Wave parameters
4  lambda = 2;      % Wavelength (m)
5  H = 0.8;        % Wave height (m) -> H = 2 * A
6  A = H / 2;      % Amplitude (m)
7  d = 5;          % Water depth (m)
8  g = 9.81;       % Gravity (m/s^2)
9  k = 2 * pi / lambda; % Wave number
10 omega = sqrt(g * k * tanh(k * d)); % Angular frequency
    (linear dispersion relation)
11
12 % Compute depth-dependent coefficients from Fenton (1985)
13 tanh_kd = tanh(k * d);
14 B2 = (1/2) * (k * A / tanh_kd);
15 B3 = (3/8) * (k * A / tanh_kd)^2;
16 B4 = (1/3) * (k * A / tanh_kd)^3;
17 B5 = (5/16) * (k * A / tanh_kd)^4;
18
19 % X-domain: 4 wavelengths
20 x = linspace(0, 4* lambda, 1000);
21 t = 0; % Fixed time for visualization
22 theta = k * x - omega * t; % Phase
23
24 % First-order (linear) wave
25 eta_linear = A * cos(theta);
26

```

```

27 % Fifth-order Stokes wave (with depth effects)
28 eta_fifth = A * cos(theta) + B2 * A^2 * cos(2 * theta) + ...
29           B3 * A^3 * cos(3 * theta) + B4 * A^4 * cos(4 *
           theta) + ...
30           B5 * A^5 * cos(5 * theta);
31
32 % Plot results
33 figure;
34 plot(x, eta_linear, 'b', 'LineWidth', 1.5); hold on;
35 plot(x, eta_fifth, 'r', 'LineWidth', 1.5);
36 xlabel('Position (m)');
37 ylabel('Wave Elevation (m)');
38 legend('First-Order Wave', 'Fifth-Order Stokes Wave');
39 grid on;
40 title('Comparison of Linear and Fifth-Order Stokes Waves');
41
42 hold off;

```

A.4 Code for generating velocity scene animation

This MATLAB script creates an animation from a sequence of velocity scene images (.png files) named with a specific pattern. It extracts numerical values from filenames to sort the images, reads the first image to determine resolution, and initializes a VideoWriter object to save the animation as velocity_animation.avi. The script loops through the sorted images, displays each frame, captures it, and writes it to the video file at 10 fps, ensuring smooth playback. This is ideal for visualizing time-dependent velocity fields or dynamic data as a seamless animation.

Listing A.4: MATLAB Code

```

1 % Set the file path where the density scene image files are
  located
2 filePath = './'; % Current directory
3
4 % Define the file pattern for density scene images
5 scenePattern = 'Velocity_VelocityScene__';
6
7 % Replace this for Density or any other scene generated
8 % scenePattern = 'Density_DensityScene__';
9
10 % Get a list of all .png files in the directory that match
    the pattern
11 files = dir(fullfile(filePath, [scenePattern, '*.png']));
12
13 % Extract filenames and corresponding numerical values to
    sort them
14 fileNames = {files.name};

```

A. Appendix 1

```
15 sceneNumbers = zeros(1, length(fileNames));
16
17 for i = 1:length(fileNames)
18     % Extract the number from the filename (e.g.,
19         1.000000e-01)
20     sceneNumbers(i) = sscanf(fileNames{i}, [scenePattern,
21         '%e']);
22 end
23
24 % Sort the filenames based on extracted numeric values
25 [~, sortedIndices] = sort(sceneNumbers);
26 sortedFiles = files(sortedIndices);
27
28 % Read first image to get dimensions
29 firstImage = imread(fullfile(sortedFiles(1).folder,
30     sortedFiles(1).name));
31 [imgHeight, imgWidth, ~] = size(firstImage);
32
33 % Create figure with the same resolution as images
34 figureHandle = figure('Position', [100, 100, imgWidth,
35     imgHeight], 'Color', 'w', ...
36     'MenuBar', 'none', 'ToolBar', 'none',
37     'Resize', 'off');
38 axis off;
39
40 % Set up the VideoWriter object to match the image resolution
41 videoWriter = VideoWriter('velocity_animation.avi');
42 videoWriter.FrameRate = 10; % Adjust frame rate as needed
43 open(videoWriter);
44
45 % Loop through the sorted scene files to create the animation
46 for i = 1:length(sortedFiles)
47     % Load the current image
48     sceneFile = fullfile(sortedFiles(i).folder,
49         sortedFiles(i).name);
50     img = imread(sceneFile);
51
52     % Display the image with exact dimensions
53     clf; % Clear previous frame
54     imshow(img, 'Border', 'tight');
55
56     % Capture the current frame for the video
57     frame = getframe(figureHandle); % Capture the figure
58         window as a frame
59     writeVideo(videoWriter, frame); % Write frame to video
60
61     % Pause for smooth animation
62     pause(0.0001);
63 end
```

```

57
58 % Close the video file after writing all frames
59 close(videoWriter);

```

A.5 Java Macro for Steady-Flow 2D Setup

This Java macro was utilized to analyze the PID controller's performance under steady-flow conditions. It automates data extraction, processes simulation results.

Listing A.5: Java Code

```

1 package macro;
2
3 import star.common.*;
4 import star.base.neo.*;
5 import java.util.*;
6 import java.io.*;
7 import star.base.report.*;
8 import star.sixdof.*;
9 import star.vis.*;
10 import star.flow.*;
11
12 public class simplePID extends StarMacro {
13     private static final double TARGET_LIFT = 5000.0; //
14         Target lift in Newtons
15     private static final double INITIAL_SAMPLING_INTERVAL = 0.1; //
16         Initial sampling interval (seconds)
17     private static final double AOA_MIN = -5.0; //
18         Minimum AoA (degrees)
19     private static final double AOA_MAX = 15.0; // Maximum
20         AoA (degrees)
21     private static final double MAX_DELTA_AOA = 0.3; // Max AoA
22         change per step (degrees)
23
24     private double kp = 10.0; // Default proportional gain
25     private double ki = 0.05; // Default integral gain
26     private double kd = 1.0; // Default derivative gain
27     private double samplingInterval = INITIAL_SAMPLING_INTERVAL;
28     private double integralSum = 0.0;
29     private double previousError = 0.0;
30     private double currentAoA = 0.0;
31     private double absoluteErrorSum = 0.0;
32     private double lastRecordedTime = 0.0;
33
34     @Override
35     public void execute() {
36         Simulation simulation = getActiveSimulation();
37         ForceReport LiftForceReport = (ForceReport) simulation.
38             getReportManager().getReport("LiftForce");
39
40         Map<String, ParameterConfig> globalParameters = new HashMap
41             <>();

```

```

35     globalParameters.put("Implicit Unsteady Time-Step", new
        ParameterConfig(1.0E-4, "s"));
36     globalParameters.put("Inner Iterations", new
        ParameterConfig(15, ""));
37     setGlobalParameters(simulation, globalParameters);
38
39
40     // Prepare output directory and files
41     String outputDir = simulation.getSessionDir() + "/"
        "Hydrofoil_Control_Results";
42     createDirectory(outputDir);
43     String csvFilePath = outputDir + "/pid_results.csv";
44     String adaptiveTuningCsvPath = outputDir + "/"
        "adaptive_pid_tuning.csv";
45     setupCsv(csvFilePath);
46     setupAdaptiveTuningCsv(adaptiveTuningCsvPath);
47
48     String velocityScenesDir = outputDir + "/VelocityScenes";
49     String densityScenesDir = outputDir + "/DensityScenes";
50
51     createDirectory(velocityScenesDir);
52     createDirectory(densityScenesDir);
53
54     //saveScenes(simulation, "Density", densityScenesDir + "/"
        "DensityScene_", 0.01, 4.0);
55     saveScenes(simulation, "Velocity", velocityScenesDir + "/"
        "VelocityScene_", 0.01, 4.0);
56     // Clear initial solution and start control loop
57     simulation.getSolution().clearSolution();
58     controlLift(simulation, LiftForceReport, csvFilePath,
        adaptiveTuningCsvPath);
59     exportSimulationData(outputDir);
60 }
61
62 private void controlLift(Simulation simulation, ForceReport
    LiftForceReport, String csvFilePath, String
    adaptiveTuningCsvPath) {
63     UserFieldFunction aoaFunction = (UserFieldFunction)
        simulation.getFieldFunctionManager().getFunction("FF -
        AoA");
64     aoaFunction.setDefinition("0*<1 deg>");
65
66     while (simulation.getSolution().getPhysicalTime() <= 4.0) {
67         double currentTime = simulation.getSolution().
            getPhysicalTime();
68
69         if (currentTime - lastRecordedTime >= samplingInterval)
            {
70             double LiftForce = LiftForceReport.
                getReportMonitorValue();
71             absoluteErrorSum += Math.abs(LiftForce -
                TARGET_LIFT) * samplingInterval;
72
73             adjustAoA(LiftForce, simulation, aoaFunction);
74             recordData(csvFilePath, currentTime, LiftForce);
75             recordAdaptiveTuningData(adaptiveTuningCsvPath,

```

```

        currentTime);
        lastRecordedTime = currentTime;
    }

    simulation.getSimulationIterator().step(1);
}

exportFinalSummary(csvFilePath);
savePlots(simulation);
}

private void adjustAoA(double Lift, Simulation simulation,
    UserFieldFunction aoaFunction) {
    double error = TARGET_LIFT - Lift;

    // Proportional term
    double proportional = kp * error;

    integralSum += error * samplingInterval;

    double integral = ki * integralSum;

    // Derivative term with filtering
    double derivative = (error - previousError) /
        samplingInterval;

    // Total correction
    double correction = proportional + integral + derivative;

    // Limit AoA change
    correction = Math.max(Math.min(correction, MAX_DELTA_AOA),
        -MAX_DELTA_AOA);

    // Update AoA
    currentAoA += correction;
    //currentAoA = Math.max(Math.min(currentAoA, AOA_MAX),
        AOA_MIN);

    aoaFunction.setDefinition(String.format("%f*<1 deg>",
        currentAoA));
    previousError = error;
}

private static class ParameterConfig {
    private final double value;
    private final String units;

    public ParameterConfig(double value, String units) {
        this.value = value;
        this.units = units;
    }

    public double getValue() {
        return value;
    }
}

```

```
126
127     public String getUnits() {
128         return units;
129     }
130 }
131
132 private void setGlobalParameters(Simulation simulation, Map<
133     String, ParameterConfig> parameters) {
134     GlobalParameterManager globalParameterManager = simulation.
135         get(GlobalParameterManager.class);
136
137     for (Map.Entry<String, ParameterConfig> entry : parameters.
138         entrySet()) {
139         String parameterName = entry.getKey();
140         ParameterConfig config = entry.getValue();
141
142         ScalarGlobalParameter parameter = (ScalarGlobalParameter
143             ) globalParameterManager.getObject(parameterName);
144         if (parameter != null) {
145             Units units = (Units) simulation.getUnitsManager().
146                 getObject(config.getUnits());
147             parameter.getQuantity().setValueAndUnits(config.
148                 getValue(), units);
149             simulation.println(parameterName + " set to " +
150                 config.getValue() + " " + config.getUnits());
151         } else {
152             simulation.println("Global parameter not found: " +
153                 parameterName);
154         }
155     }
156 }
157
158 private void recordData(String csvFilePath, double time, double
159     Lift) {
160     try (BufferedWriter writer = new BufferedWriter(new
161         FileWriter(csvFilePath, true))) {
162         writer.write(String.format("%f,%f,%f\n", time, Lift,
163             currentAoA));
164     } catch (IOException e) {
165         getActiveSimulation().println("Error writing to CSV: "
166             + e.getMessage());
167     }
168 }
169
170 private void recordAdaptiveTuningData(String
171     adaptiveTuningCsvPath, double currentTime) {
172     try (BufferedWriter writer = new BufferedWriter(new
173         FileWriter(adaptiveTuningCsvPath, true))) {
174         writer.write(String.format("%f,%f,%f,%f\n", currentTime
175             , kp, ki, kd));
176     } catch (IOException e) {
177         getActiveSimulation().println("Error writing to
178             adaptive tuning CSV: " + e.getMessage());
179     }
180 }
```



```

166 private void setupCsv(String csvFilePath) {
167     try (BufferedWriter writer = new BufferedWriter(new
168         FileWriter(csvFilePath))) {
169         writer.write("Time,Lift Force,AoA\n");
170     } catch (IOException e) {
171         getActiveSimulation().println("Error creating CSV file:
172         " + e.getMessage());
173     }
174 }
175
176 private void setupAdaptiveTuningCsv(String
177     adaptiveTuningCsvPath) {
178     try (BufferedWriter writer = new BufferedWriter(new
179         FileWriter(adaptiveTuningCsvPath))) {
180         writer.write("Time,kp,ki,kd\n");
181     } catch (IOException e) {
182         getActiveSimulation().println("Error creating adaptive
183         tuning CSV: " + e.getMessage());
184     }
185 }
186
187 private void exportFinalSummary(String csvFilePath) {
188     try (BufferedWriter writer = new BufferedWriter(new
189         FileWriter(csvFilePath, true))) {
190         writer.write("Final Summary\n");
191         writer.write(String.format("Cumulative IAE,%f\n",
192             absoluteErrorSum));
193         writer.write(String.format("Final AoA,%f\n", currentAoA
194             ));
195     } catch (IOException e) {
196         getActiveSimulation().println("Error writing final
197         summary: " + e.getMessage());
198     }
199 }
200
201 private void exportSimulationData(String baseDirectory) {
202     Simulation simulation = getActiveSimulation();
203
204     // Create directories for Lift and Angle of Attack data
205     String liftDirectory = baseDirectory + "\\LiftData";
206     String aoaDirectory = baseDirectory + "\\AoAData";
207     createDirectory(liftDirectory);
208     createDirectory(aoaDirectory);
209
210     // Export Lift and AoA data
211     exportMonitorPlotData(simulation, "Lift Force Plot",
212         liftDirectory + "/lift_data.csv");
213     exportMonitorPlotData(simulation, "AoA Monitor Plot",
214         aoaDirectory + "/aoa_data.csv");
215     exportMonitorPlotData(simulation, "Mean Lift Monitor Plot",
216         aoaDirectory + "/Mean_lift_data.csv");
217 }
218
219 private void exportMonitorPlotData(Simulation simulation, String
220     plotName, String filePath) {
221     // Get the specified monitor plot by name

```

```

209     MonitorPlot monitorPlot = (MonitorPlot) simulation.
        getPlotManager().getPlot(plotName);
210
211     if (monitorPlot != null) {
212         // Optionally configure axis bounds (adjust values as
            needed)
213         Cartesian2DAxisManager axisManager = (
            Cartesian2DAxisManager) monitorPlot.getAxisManager();
214         axisManager.setAxesBounds(new Vector(Arrays.<AxisManager.
            AxisBounds>asList(
215             new AxisManager.AxisBounds("Left Axis", 0.0, false,
                1.0, false),
216             new AxisManager.AxisBounds("Bottom Axis", 0.0, false,
                1.0, false)
217         )));
218
219         // Export data to the specified file path
220         monitorPlot.export(resolvePath(filePath), ",");
221         simulation.println(plotName + " data exported successfully
            to: " + filePath);
222     } else {
223         simulation.println("Plot not found: " + plotName);
224     }
225 }
226
227 private void saveScenes(Simulation simulation, String
        sceneName, String filePath, double deltaTime, double
        stopTime) {
228     // Get the specified scene by name
229     Scene scene = simulation.getSceneManager().getScene(sceneName
        );
230     if (scene != null) {
231         SceneUpdate sceneUpdate = scene.getSceneUpdate();
232
233         // Enable animation saving
234         sceneUpdate.setSaveAnimation(true);
235
236         // Set update mode to Delta Time
237         sceneUpdate.getUpdateModeOption().setSelected(
            StarUpdateModeOption.Type.DELTATIME);
238
239         // Configure Delta Time settings
240         DeltaTimeUpdateFrequency deltaTimeUpdateFrequency =
            sceneUpdate.getDeltaTimeUpdateFrequency();
241         Units timeUnits = ((Units) simulation.getUnitsManager().
            getObject("s"));
242         deltaTimeUpdateFrequency.setDeltaTime(String.valueOf(
            deltaTime), timeUnits); // Convert deltaTime to String
243
244         // Configure Stop Event based on range quantity
245         deltaTimeUpdateFrequency.setStopEnabled(true);
246         deltaTimeUpdateFrequency.setStopTime(stopTime, timeUnits)
            ;
247
248         // Set animation file path and file name prefix
249         String directoryPath = filePath.substring(0, filePath.

```

```

250         lastIndexOf("/"));
251         String fileNamePrefix = filePath.substring(filePath.
252             lastIndexOf("/") + 1);
253
254         sceneUpdate.setAnimationFilePath(directoryPath); // Set
255             directory path
256         sceneUpdate.setAnimationFilenameBase(fileNamePrefix); //
257             Set file name prefix
258
259         simulation.println(sceneName + " animation settings
260             configured successfully. Files will be saved in: " +
261             directoryPath);
262     } else {
263         simulation.println("Scene not found: " + sceneName);
264     }
265 }
266
267 private void savePlots(Simulation simulation) {
268     String plotsDir = simulation.getSessionDir() + "/Plots
269         Directory";
270     createDirectory(plotsDir);
271
272     String[] plotNames = {
273         "Lift Force Plot",
274         "Maximum Co",
275         "Reports Plot",
276         "AoA Monitor Plot"
277     };
278
279     for (String plotName : plotNames) {
280         StarPlot plot = (StarPlot) simulation.getPlotManager().
281             getObject(plotName);
282         if (plot != null) {
283             plot.encode(plotsDir + "/" + plotName.replaceAll("
284                 ", "_") + ".png", "PNG", 1920, 1080); // Set
285                 resolution here
286         } else {
287             simulation.println("Plot not found: " + plotName);
288         }
289     }
290 }
291
292 private void createDirectory(String path) {
293     File dir = new File(path);
294     if (!dir.exists()) {
295         dir.mkdirs();
296     }
297 }
298 }

```

A.6 Java Macro for Lift Signal Filtering Comparison

This Java macro was utilized to compare different filtering mechanisms for the lift signal in the PID-controlled hydrofoil system. The primary objective was to evaluate the effects of different filters on the system's performance. Three simulations were conducted, each employing a different approach:

- **No Filtering:** The raw lift force data was directly fed into the PID controller, serving as a baseline to observe system behavior with unprocessed data.
- **Mean Filtering:** The lift signal was averaged over a time window before being used in the control algorithm, reducing noise but introducing a small lag.
- **Exponential Moving Average (EMA) Filtering:** A weighted smoothing method was applied, giving recent values greater significance to improve responsiveness while reducing noise.

The filtering method was implemented by modifying the **error term** in the `adjustAoA` function of the macro. By replacing the error calculation with different filtered values, the system's behavior under each filtering condition was analyzed and compared. The macro also handled adaptive PID tuning, data logging, and scene exports for post-processing and visualization.

Listing A.6: Java Code

```

1 package macro;
2
3
4 import star.common.*;
5 import star.base.neo.*;
6 import java.util.*;
7 import java.io.*;
8 import star.base.report.*;
9 import star.sixdof.*;
10 import star.vis.*;
11 import star.flow.*;
12
13 public class WithEMA extends StarMacro {
14     private static final double TARGET_LIFT = 50.0;           // Target
15         lift in Newtons
16     private static final double INITIAL_SAMPLING_INTERVAL = 0.1; //
17         Initial sampling interval (seconds)
18     private static final double MAX_INTEGRAL_SUM = 20.0; // Anti-
19         windup limit
20     private static final double AOA_MIN = -5.0;             //
21         Minimum AoA (degrees)
22     private static final double AOA_MAX = 15.0;             // Maximum
23         AoA (degrees)
24     private static final double MAX_DELTA_AOA = 0.3;         // Max AoA
25         change per step (degrees)
26
27     private double kp = 10.0; // Default proportional gain
28     private double ki = 0.05; // Default integral gain
29     private double kd = 1.0; // Default derivative gain

```

```

24     private double samplingInterval = INITIAL_SAMPLING_INTERVAL;
25
26     private double alpha = 0.1; // EMA smoothing factor
27     private double ema = 0.0;
28
29     private double integralSum = 0.0;
30     private double previousError = 0.0;
31     private double currentAoA = 0.0;
32
33     private double absoluteErrorSum = 0.0;
34     private double lastRecordedTime = 0.0;
35
36     @Override
37     public void execute() {
38         Simulation simulation = getActiveSimulation();
39         ForceReport LiftForceReport = (ForceReport) simulation.
40             getReportManager().getReport("LiftForce");
41         // StatisticsReport meanLiftReport = (StatisticsReport)
42             simulation.getReportManager().getReport("Mean Lift");
43
44         Map<String, ParameterConfig> globalParameters = new HashMap
45             <>();
46         globalParameters.put("Implicit Unsteady Time-Step", new
47             ParameterConfig(1.0E-4, "s"));
48         globalParameters.put("Inner Iterations", new
49             ParameterConfig(15, ""));
50         setGlobalParameters(simulation, globalParameters);
51
52         // Prepare output directory and files
53         String outputDir = simulation.getSessionDir() + "/"
54             "Hydrofoil_Control_Results";
55         createDirectory(outputDir);
56         String csvFilePath = outputDir + "/pid_results.csv";
57         String adaptiveTuningCsvPath = outputDir + "/"
58             "adaptive_pid_tuning.csv";
59         setupCsv(csvFilePath);
60         setupAdaptiveTuningCsv(adaptiveTuningCsvPath);
61
62         String velocityScenesDir = outputDir + "/VelocityScenes";
63         String densityScenesDir = outputDir + "/DensityScenes";
64
65         createDirectory(velocityScenesDir);
66         createDirectory(densityScenesDir);
67
68         saveScenes(simulation, "Density", densityScenesDir + "/"
69             "DensityScene_", 0.01, 4.0);
70         saveScenes(simulation, "Velocity", velocityScenesDir + "/"
71             "VelocityScene_", 0.01, 4.0);
72         // Clear initial solution and start control loop
73         simulation.getSolution().clearSolution();
74         controlLift(simulation, LiftForceReport, csvFilePath,
75             adaptiveTuningCsvPath);
76         exportSimulationData(outputDir);
77     }
78
79     //Modify this function according to the filter used

```

```

70
71     private void controlLift(Simulation simulation, ForceReport
       LiftForceReport, String csvFilePath, String
       adaptiveTuningCsvPath) {
72         UserFieldFunction aoaFunction = (UserFieldFunction)
           simulation.getFieldFunctionManager().getFunction("FF -
73             AoA");
           aoaFunction.setDefinition("0*<1 deg>");
74
75         while (simulation.getSolution().getPhysicalTime() <= 10.0)
           {
76             double currentTime = simulation.getSolution().
               getPhysicalTime();
77
78             if (currentTime - lastRecordedTime >= samplingInterval)
               {
79                 double Lift = LiftForceReport.getReportMonitorValue
                   ();
80                 absoluteErrorSum += Math.abs(Lift - TARGET_LIFT) *
                   samplingInterval;
81
82                 ema = alpha * Lift + (1 - alpha) * ema;
83
84                 adjustAoA(ema, simulation, aoaFunction);
85                 recordData(csvFilePath, currentTime, Lift, ema);
86                 recordAdaptiveTuningData(adaptiveTuningCsvPath,
                   currentTime);
87                 lastRecordedTime = currentTime;
88             }
89
90             simulation.getSimulationIterator().step(1);
91         }
92
93         exportFinalSummary(csvFilePath);
94         savePlots(simulation);
95     }
96
97     private void adjustAoA(double ema, Simulation simulation,
       UserFieldFunction aoaFunction) {
98         double error = TARGET_LIFT - ema;
99         // double error = TARGET_LIFT - meanLift; (For Mean Filter
       )
100        // double error = TARGET_LIFT - Lift; (For Raw Data)
101
102        // Proportional term
103        double proportional = kp * error;
104
105        // Integral term with conditional update (anti-windup)
106        if (currentAoA > AOA_MIN && currentAoA < AOA_MAX) {
107            integralSum += error * samplingInterval;
108            integralSum = Math.max(Math.min(integralSum,
               MAX_INTEGRAL_SUM), -MAX_INTEGRAL_SUM);
109        }
110        double integral = ki * integralSum;
111
112        // Derivative term with filtering

```

```

113     double derivative = (error - previousError) /
        samplingInterval;
114
115     // Total correction
116     double correction = proportional + integral + derivative;
117
118     // Limit AoA change
119     correction = Math.max(Math.min(correction, MAX_DELTA_AOA),
        -MAX_DELTA_AOA);
120
121     // Update AoA
122     currentAoA += correction;
123     currentAoA = Math.max(Math.min(currentAoA, AOA_MAX),
        AOA_MIN);
124
125     aoaFunction.setDefinition(String.format("%f*<1 deg>",
        currentAoA));
126     previousError = error;
127
128     // Adaptive PID tuning
129     adaptPIDParameters(error, derivative);
130 }
131
132 private void adaptPIDParameters(double error, double derivative
133 ) {
134     // Adjust kp, ki, kd based on system behavior
135
136     // If the error is large, increase proportional gain
137     if (Math.abs(error) > 5.0) {kp = Math.min(kp + 0.2, 30.0);
        } // Prevent runaway gain
138     else { kp = Math.max(kp - 0.05, 1.0); } // Slow decrease in
        kp
139
140     // If the lift is stable, increase integral gain
141     if (Math.abs(error) < 1.0) { ki = Math.min(ki + 0.01, 0.2);
        } // Increase ki when stable
142     else {ki = Math.max(ki - 0.005, 0.05); } // Decrease ki
        when system is unstable
143
144     // Adjust derivative gain based on rate of change of error
145     if (Math.abs(derivative) > 0.5) { kd = Math.min(kd + 0.1,
        5.0); } // Increase kd if system is oscillating
146     else { kd = Math.max(kd - 0.05, 0.5); } // Decrease kd if
        system is stable
147
148 }
149
150 private static class ParameterConfig {
151     private final double value;
152     private final String units;
153
154     public ParameterConfig(double value, String units) {
155         this.value = value;
156         this.units = units;
157     }

```

```

158     public double getValue() {
159         return value;
160     }
161
162     public String getUnits() {
163         return units;
164     }
165 }
166
167 private void setGlobalParameters(Simulation simulation, Map<
String, ParameterConfig> parameters) {
168     GlobalParameterManager globalParameterManager = simulation
        .get(GlobalParameterManager.class);
169
170     for (Map.Entry<String, ParameterConfig> entry : parameters
        .entrySet()) {
171         String parameterName = entry.getKey();
172         ParameterConfig config = entry.getValue();
173
174         ScalarGlobalParameter parameter = (
            ScalarGlobalParameter) globalParameterManager.
            getObject(parameterName);
175         if (parameter != null) {
176             Units units = (Units) simulation.getUnitsManager()
                .getObject(config.getUnits());
177             parameter.getQuantity().setValueAndUnits(config.
                getValue(), units);
178             simulation.println(parameterName + " set to " +
                config.getValue() + " " + config.getUnits());
179         } else {
180             simulation.println("Global parameter not found: "
                + parameterName);
181         }
182     }
183 }
184
185 private void recordData(String csvFilePath, double time, double
Lift, double ema) {
186     try (BufferedWriter writer = new BufferedWriter(new
        FileWriter(csvFilePath, true))) {
187         writer.write(String.format("%f,%f,%f,%f\n", time, Lift,
            ema, currentAoA));
188     } catch (IOException e) {
189         getActiveSimulation().println("Error writing to CSV: "
            + e.getMessage());
190     }
191 }
192
193 private void recordAdaptiveTuningData(String
adaptiveTuningCsvPath, double currentTime) {
194     try (BufferedWriter writer = new BufferedWriter(new
        FileWriter(adaptiveTuningCsvPath, true))) {
195         writer.write(String.format("%f,%f,%f,%f\n", currentTime
            , kp, ki, kd));
196     } catch (IOException e) {
197         getActiveSimulation().println("Error writing to

```



```

198         adaptive tuning CSV: " + e.getMessage());
199     }
200 }
201 private void setupCsv(String csvFilePath) {
202     try (BufferedWriter writer = new BufferedWriter(new
203         FileWriter(csvFilePath))) {
204         writer.write("Time,Lift Force,EMA Filtered Lift,AoA\n");
205         ;
206     } catch (IOException e) {
207         getActiveSimulation().println("Error creating CSV file:
208             " + e.getMessage());
209     }
210 }
211 private void setupAdaptiveTuningCsv(String
212     adaptiveTuningCsvPath) {
213     try (BufferedWriter writer = new BufferedWriter(new
214         FileWriter(adaptiveTuningCsvPath))) {
215         writer.write("Time,kp,ki,kd\n");
216     } catch (IOException e) {
217         getActiveSimulation().println("Error creating adaptive
218             tuning CSV: " + e.getMessage());
219     }
220 }
221 private void exportFinalSummary(String csvFilePath) {
222     try (BufferedWriter writer = new BufferedWriter(new
223         FileWriter(csvFilePath, true))) {
224         writer.write("Final Summary\n");
225         writer.write(String.format("Cumulative IAE,%f\n",
226             absoluteErrorSum));
227         writer.write(String.format("Final AoA,%f\n", currentAoA
228             ));
229     } catch (IOException e) {
230         getActiveSimulation().println("Error writing final
231             summary: " + e.getMessage());
232     }
233 }
234 private void exportSimulationData(String baseDirectory) {
235     Simulation simulation = getActiveSimulation();
236
237     // Create directories for Lift and Angle of Attack data
238     String liftDirectory = baseDirectory + "\\LiftData";
239     String aoaDirectory = baseDirectory + "\\AoAData";
240     String meanLiftDirectory = baseDirectory + "\\MeanLiftData";
241     createDirectory(liftDirectory);
242     createDirectory(aoaDirectory);
243     createDirectory(meanLiftDirectory);
244
245     // Export Lift and AoA data
246     exportMonitorPlotData(simulation, "Lift Force Plot",
247         liftDirectory + "\\lift_data.csv");
248     exportMonitorPlotData(simulation, "AoA Monitor Plot",
249         aoaDirectory + "\\aoa_data.csv");

```

```
241     exportMonitorPlotData(simulation, "Mean Lift Monitor Plot",
242                           meanLiftDirectory + "\\Mean_Lift_data.csv");
243 }
244 private void exportMonitorPlotData(Simulation simulation, String
245     plotName, String filePath) {
246     // Get the specified monitor plot by name
247     MonitorPlot monitorPlot = (MonitorPlot) simulation.
248         getPlotManager().getPlot(plotName);
249
250     if (monitorPlot != null) {
251         // Optionally configure axis bounds (adjust values as
252         // needed)
253         Cartesian2DAxisManager axisManager = (
254             Cartesian2DAxisManager) monitorPlot.getAxisManager();
255         axisManager.setAxesBounds(new Vector(Arrays.<AxisManager.
256             AxisBounds>asList(
257                 new AxisManager.AxisBounds("Left Axis", 0.0, false,
258                     1.0, false),
259                 new AxisManager.AxisBounds("Bottom Axis", 0.0, false,
260                     1.0, false)
261             )));
262
263         // Export data to the specified file path
264         monitorPlot.export(resolvePath(filePath), ",");
265         simulation.println(plotName + " data exported successfully
266             to: " + filePath);
267     } else {
268         simulation.println("Plot not found: " + plotName);
269     }
270 }
271
272 private void saveScenes(Simulation simulation, String
273     sceneName, String filePath, double deltaTime, double
274     stopTime) {
275     // Get the specified scene by name
276     Scene scene = simulation.getSceneManager().getScene(sceneName
277     );
278     if (scene != null) {
279         SceneUpdate sceneUpdate = scene.getSceneUpdate();
280
281         // Enable animation saving
282         sceneUpdate.setSaveAnimation(true);
283
284         // Set update mode to Delta Time
285         sceneUpdate.getUpdateModeOption().setSelected(
286             StarUpdateModeOption.Type.DELTATIME);
287
288         // Configure Delta Time settings
289         DeltaTimeUpdateFrequency deltaTimeUpdateFrequency =
290             sceneUpdate.getDeltaTimeUpdateFrequency();
291         Units timeUnits = ((Units) simulation.getUnitsManager().
292             getObject("s"));
293         deltaTimeUpdateFrequency.setDeltaTime(String.valueOf(
294             deltaTime), timeUnits); // Convert deltaTime to String
```

```

281 // Configure Stop Event based on range quantity
282 deltaTimeUpdateFrequency.setStopEnabled(true);
283 deltaTimeUpdateFrequency.setStopTime(stopTime, timeUnits)
284 ;
285 // Set animation file path and file name prefix
286 String directoryPath = filePath.substring(0, filePath.
    lastIndexOf("/"));
287 String fileNamePrefix = filePath.substring(filePath.
    lastIndexOf("/") + 1);
288
289 sceneUpdate.setAnimationFilePath(directoryPath); // Set
    directory path
290 sceneUpdate.setAnimationFilenameBase(fileNamePrefix); //
    Set file name prefix
291
292 simulation.println(sceneName + " animation settings
    configured successfully. Files will be saved in: " +
    directoryPath);
293 } else {
294     simulation.println("Scene not found: " + sceneName);
295 }
296 }
297
298 private void savePlots(Simulation simulation) {
299     String plotsDir = simulation.getSessionDir() + "/Plots
        Directory";
300     createDirectory(plotsDir);
301
302     String[] plotNames = {
303         "Lift Force Plot",
304         "Maximum Co",
305         "Reports Plot",
306         "AoA Monitor Plot",
307         "Mean Lift Monitor Plot"
308     };
309
310     for (String plotName : plotNames) {
311         StarPlot plot = (StarPlot) simulation.getPlotManager().
            getObject(plotName);
312         if (plot != null) {
313             plot.encode(plotsDir + "/" + plotName.replaceAll("
                ", "_") + ".png", "PNG", 1920, 1080); // Set
                resolution here
314         } else {
315             simulation.println("Plot not found: " + plotName);
316         }
317     }
318 }
319
320 private void createDirectory(String path) {
321     File dir = new File(path);
322     if (!dir.exists()) {
323         dir.mkdirs();
324     }
325 }

```

```
326     }  
327 }
```

A.7 Java Macro for Optimization of Filter Parameters

In this section, the optimization of the Exponential Moving Average (EMA) filter parameters was carried out by adjusting the smoothing factor, α , in the code. The primary goal was to find the optimal value of α that would strike a balance between noise reduction and responsiveness to changes in the lift signal.

In the code, different α values were tested to evaluate their impact on system performance. By adjusting α , the filter's ability to retain essential dynamics while reducing noise was optimized. Several simulations were run, each using a different α value, and the results were compared to determine the best setting for the system.

This section of the code iterates over the lift signal data, applying the EMA filter to each data point. The value of α is modified during different simulations to assess its effect on the system's performance.

Listing A.7: Java Code

```
1  
2 package macro;  
3  
4 import star.common.*;  
5 import star.base.neo.*;  
6 import java.util.*;  
7 import java.io.*;  
8 import star.flow.*;  
9 import java.time.LocalDateTime;  
10 import java.time.format.DateTimeFormatter;  
11  
12 public class HPC_Run extends StarMacro {  
13     private static final double TARGET_LIFT_FRONT = 90.0;  
14     private static final double TARGET_LIFT_REAR = 10.0;  
15     private static final double INITIAL_SAMPLING_INTERVAL = 0.01;  
16     private static final double MAX_INTEGRAL_SUM = 50.0;  
17     private static final double AOA_MIN = 0.0;  
18     private static final double AOA_MAX = 15.0;  
19     private static final double MAX_DELTA_AOA = 0.3;  
20     private static final double ERROR_THRESHOLD = 5.0;  
21  
22     private double kp_front = 20.0;  
23     private double ki_front = 0.05;  
24     private double kd_front = 1.0;  
25  
26     private double kp_rear = 10.0;  
27     private double ki_rear = 0.05;  
28     private double kd_rear = 1.0;  
29  
30     private double samplingInterval = INITIAL_SAMPLING_INTERVAL;  
31     private double alpha = 0.1; // EMA smoothing factor
```

```

32 private double emaFront = 0.0;
33 private double emaRear = 0.0;
34 private double integralSumFront = 0.0;
35 private double integralSumRear = 0.0;
36 private double previousErrorFront = 0.0;
37 private double previousErrorRear = 0.0;
38 private double currentAoAFront = 0.0;
39 private double currentAoARear = 0.0;
40 private double absoluteErrorSum = 0.0;
41 private double lastRecordedTime = 0.0;
42
43 @Override
44 public void execute() {
45     Simulation simulation = getActiveSimulation();
46     ForceReport LiftFrontReport = (ForceReport) simulation.
47         getReportManager().getReport("Lift1");
48     ForceReport LiftRearReport = (ForceReport) simulation.
49         getReportManager().getReport("Lift2");
50
51     // double[] samplingIntervals = { 0.01, 0.05, 0.1 };
52     // double[] alphaValues = { 0.1, 0.3, 0.5, 0.7, 0.9 };
53
54     double[] samplingIntervals = { 0.1 };
55     double[] alphaValues = { 0.1, 0.3, 0.5, 0.7, 0.9 };
56
57     String logFilePath = simulation.getSessionDir() + "/"
58         + "simulation_status.txt";
59     logSimulation(logFilePath, "Simulation Log Start");
60
61     for (double interval : samplingIntervals) {
62         for (double smoothingAlpha : alphaValues) {
63             resetSimulationState(simulation);
64
65             this.samplingInterval = interval;
66             this.alpha = smoothingAlpha;
67
68             String combinationFolder = "Sampling_" + interval +
69                 "_Alpha_" + smoothingAlpha;
70             String outputDir = simulation.getSessionDir() + "/"
71                 + "pid_control_results/" + combinationFolder;
72             createDirectory(outputDir);
73             String csvFilePath = outputDir + "/pid_results.csv"
74                 ;
75             logSimulation(logFilePath, "Starting Simulation: Interval="
76                 + interval + ", Alpha=" + smoothingAlpha);
77
78             setupCsv(csvFilePath);
79             controllift(simulation, LiftFrontReport,
80                 LiftRearReport, csvFilePath, outputDir);
81
82             // Summarize results
83             logSimulation(logFilePath, String.format("Completed
84                 Simulation: SamplingInterval=%f, Alpha=%f",
85                 interval, smoothingAlpha));
86
87         }
88     }
89 }

```

```
78     }
79 }
80 logSimulation(logFilePath, "Simulation Log End");
81 }
82
83 private void resetSimulationState(Simulation simulation) {
84     simulation.getSolution().clearSolution();
85
86     // Reset all internal variables
87     emaFront = 0.0;
88     emaRear = 0.0;
89     integralSumFront = 0.0;
90     integralSumRear = 0.0;
91     previousErrorFront = 0.0;
92     previousErrorRear = 0.0;
93     currentAoAFront = 0.0;
94     currentAoARear = 0.0;
95     absoluteErrorSum = 0.0;
96     lastRecordedTime = 0.0;
97
98     // Reset PID parameters to their initial values
99     kp_front = 20.0;
100    ki_front = 0.05;
101    kd_front = 1.0;
102
103    kp_rear = 10.0;
104    ki_rear = 0.05;
105    kd_rear = 1.0;
106 }
107
108 private void controlLift(Simulation simulation, ForceReport
109 LiftFrontReport, ForceReport LiftRearReport, String
110 csvFilePath, String outputDir) {
111     UserFieldFunction aoaFrontFunction = (UserFieldFunction)
112         simulation.getFieldFunctionManager().getFunction("FF -
113         Front AoA");
114     UserFieldFunction aoaRearFunction = (UserFieldFunction)
115         simulation.getFieldFunctionManager().getFunction("FF -
116         Rear AoA");
117     aoaFrontFunction.setDefinition("0*<1 deg>");
118     aoaRearFunction.setDefinition("0*<1 deg>");
119
120     while (simulation.getSolution().getPhysicalTime() <= 4.0) {
121         double currentTime = simulation.getSolution().
122             getPhysicalTime();
123
124         if (currentTime - lastRecordedTime >= samplingInterval)
125         {
126             double LiftFront = LiftFrontReport.
127                 getReportMonitorValue();
128             double LiftRear = LiftRearReport.
129                 getReportMonitorValue();
130
131             emaFront = alpha * LiftFront + (1 - alpha) *
132                 emaFront;
133             emaRear = alpha * LiftRear + (1 - alpha) * emaRear;
```

```

123
124         double errorFront = TARGET_LIFT_FRONT - emaFront;
125         double errorRear = TARGET_LIFT_REAR - emaRear;
126
127         adaptPIDGains(emaFront, emaRear);
128
129         adjustAoA(errorFront, errorRear, simulation,
130                 aoaFrontFunction, aoaRearFunction);
131
132         absoluteErrorSum += Math.abs(errorFront) + Math.abs
133             (errorRear);
134
135         recordData(csvFilePath, currentTime, LiftFront,
136                 LiftRear, emaFront, emaRear, currentAoAFront,
137                 currentAoARear,
138                 kp_front, ki_front, kd_front, kp_rear,
139                 ki_rear, kd_rear,
140                 errorFront, errorRear);
141         lastRecordedTime = currentTime;
142     }
143
144     simulation.getSimulationIterator().step(1);
145 }
146
147 savePlots(simulation, outputDir);
148 }
149
150 private void adaptPIDGains(double emaFront, double emaRear) {
151     double liftErrorFront = Math.abs(TARGET_LIFT_FRONT -
152         emaFront);
153     double liftErrorRear = Math.abs(TARGET_LIFT_REAR - emaRear)
154         ;
155
156     if (liftErrorFront > ERROR_THRESHOLD) {
157         kp_front *= (1 + alpha);
158         ki_front *= (1 + alpha);
159         kd_front *= (1 + alpha);
160     } else {
161         kp_front /= (1 + alpha);
162         ki_front /= (1 + alpha);
163         kd_front /= (1 + alpha);
164     }
165
166     if (liftErrorRear > ERROR_THRESHOLD) {
167         kp_rear *= (1 + alpha);
168         ki_rear *= (1 + alpha);
169         kd_rear *= (1 + alpha);
170     } else {
171         kp_rear /= (1 + alpha);
172         ki_rear /= (1 + alpha);
173         kd_rear /= (1 + alpha);
174     }
175 }
176
177 private void adjustAoA(double errorFront, double errorRear,
178     Simulation simulation, UserFieldFunction aoaFrontFunction,

```

```

171     UserFieldFunction aoaRearFunction) {
172         double correctionFront = calculatePIDControl(errorFront,
173             kp_front, ki_front, kd_front, integralSumFront,
174             previousErrorFront);
175         currentAoAFront += correctionFront;
176         currentAoAFront = Math.max(Math.min(currentAoAFront,
177             AOA_MAX), AOA_MIN);
178         aoaFrontFunction.setDefinition(String.format("%f*<1 deg>",
179             currentAoAFront));
180         previousErrorFront = errorFront;
181
182         double correctionRear = calculatePIDControl(errorRear,
183             kp_rear, ki_rear, kd_rear, integralSumRear,
184             previousErrorRear);
185         currentAoARear += correctionRear;
186         currentAoARear = Math.max(Math.min(currentAoARear, AOA_MAX),
187             AOA_MIN);
188         aoaRearFunction.setDefinition(String.format("%f*<1 deg>",
189             currentAoARear));
190         previousErrorRear = errorRear;
191     }
192
193     private double calculatePIDControl(double error, double kp,
194         double ki, double kd, double integralSum, double
195         previousError) {
196         double proportional = kp * error;
197         integralSum += error * samplingInterval;
198         integralSum = Math.max(Math.min(integralSum,
199             MAX_INTEGRAL_SUM), -MAX_INTEGRAL_SUM);
200         double integral = ki * integralSum;
201
202         double derivative = (error - previousError) /
203             samplingInterval;
204         double derivativeTerm = kd * derivative;
205
206         double correction = proportional + integral +
207             derivativeTerm;
208         return Math.max(Math.min(correction, MAX_DELTA_AOA), -
209             MAX_DELTA_AOA);
210     }
211
212     private void recordData(String csvFilePath, double time, double
213         LiftFront, double LiftRear, double emaFront, double
214         emaRear, double aoaFront, double aoaRear,
215         double kpFront, double kiFront, double
216         kdFront, double kpRear, double
217         kiRear, double kdRear,
218         double errorFront, double errorRear) {
219         try (BufferedWriter writer = new BufferedWriter(new
220             FileWriter(csvFilePath, true))) {
221             writer.write(String.format("%.2f,%.3f,%.3f,%.3f,%.3f
222                 ,%.2f,%.2f,%.2f,%.2f,%.2f,%.2f,%.2f,%.3f,%.3f\\
223                 n",
224                 time, LiftFront, LiftRear, emaFront, emaRear, aoaFront,
225                 aoaRear, kpFront, kiFront, kdFront, kpRear, kiRear,
226                 kdRear, errorFront, errorRear));

```



```

203     } catch (IOException e) {
204         e.printStackTrace();
205     }
206 }
207
208
209 private void logSimulation(String logFilePath, String message)
210 {
211     try (BufferedWriter writer = new BufferedWriter(new FileWriter(
212         logFilePath, true))) {
213         // Add timestamp to the log message
214         String timestamp = LocalDateTime.now().format(
215             DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss"));
216         writer.write "[" + timestamp + "] " + message + "\n";
217     } catch (IOException e) {
218         e.printStackTrace();
219     }
220 }
221
222 private void createDirectory(String path) {
223     File dir = new File(path);
224     if (!dir.exists()) {
225         dir.mkdirs();
226     }
227 }
228
229 private void setupCsv(String csvFilePath) {
230     try (BufferedWriter writer = new BufferedWriter(new
231         FileWriter(csvFilePath))) {
232         writer.write("Time, Lift Front, Lift Rear, EMA front ,
233             EMA Rear, AoA Front, AoA Rear, Kp Front, Ki Front,
234             Kd Front, Kp Rear, Ki Rear, Kd Rear, Error Front,
235             Error Rear");
236     } catch (IOException e) {
237         e.printStackTrace();
238     }
239 }
240
241 private void savePlots(Simulation simulation, String outputDir)
242 {
243     String plotsDir = outputDir + "/Plots";
244     createDirectory(plotsDir);
245
246     String[] plotNames = {
247         "Lift Plot",
248         "Maximum Co",
249         "Mean Lift Plot",
250         "AoA Plot",
251         "Residuals"
252     };
253
254     for (String plotName : plotNames) {
255         StarPlot plot = (StarPlot) simulation.getPlotManager().
256             getObject(plotName);
257         if (plot != null) {

```

```
250         plot.encode(plotsDir + "/" + plotName.replaceAll("
251             ", "_") + ".png", "PNG", 1920, 1080);
252     } else {
253         simulation.println("Plot not found: " + plotName);
254     }
255 }
256 }
```

A.8 Java Macro for Hydrofoil Performance at Different Depths

In this analysis, the impact of varying hydrofoil depth on performance was investigated through a series of simulations. The hydrofoil depth, relative to the free surface, plays a crucial role in influencing the lift generation, flow behavior, and stability. To evaluate these effects, the hydrofoil depth was systematically modified within the STAR-CCM+ case files, and each simulation was rerun to observe the changes in performance metrics.

The corresponding adjustments to the angle of attack were handled by implementing an angle limiter in the control system. The limiter was enforced by the MAX_DELTA_AOA parameter, which ensures that the angle of attack is constrained to predefined limits, preventing large, abrupt changes that could negatively affect the system's stability. This mechanism was incorporated into the control code to dynamically adjust the hydrofoil's angle, based on the current depth and flow conditions.

Listing A.8: Java Code

```
1
2 package macro;
3
4 import star.common.*;
5 import star.base.neo.*;
6 import java.util.*;
7 import java.io.*;
8 import star.base.report.*;
9 import star.sixdof.*;
10 import star.vis.*;
11 import star.flow.*;
12
13 public class WithEMA extends StarMacro {
14     private static final double TARGET_LIFT = 50.0;           // Target
15         lift in Newtons
16     private static final double INITIAL_SAMPLING_INTERVAL = 0.1; //
17         Initial sampling interval (seconds)
18     private static final double MAX_INTEGRAL_SUM = 20.0; // Anti-
19         windup limit
20     private static final double AOA_MIN = -5.0;              //
21         Minimum AoA (degrees)
22     private static final double AOA_MAX = 15.0;              // Maximum
23         AoA (degrees)
```

```

19 private static final double MAX_DELTA_AOA = 1; // Max AoA
    change per step (degrees) //AngleLimiter- Change here for
    0.3,0.5,0.7 deg
20
21 private double kp = 10.0; // Default proportional gain
22
23 private double ki = 0.05; // Default integral gain
24
25 private double kd = 1.0; // Default derivative gain
26
27 private double samplingInterval = INITIAL_SAMPLING_INTERVAL;
28
29 private double alpha = 0.3; // EMA smoothing factor
30 private double ema = 0.0;
31 private static final double errorThreshold = 0.01 * TARGET_LIFT
    ;
32 private double integralSum = 0.0;
33 private double previousError = 0.0;
34 private double currentAoA = 0.0;
35
36 private double absoluteErrorSum = 0.0;
37 private double lastRecordedTime = 0.0;
38
39 @Override
40 public void execute() {
41     Simulation simulation = getActiveSimulation();
42     ForceReport LiftForceReport = (ForceReport) simulation.
        getReportManager().getReport("LiftForce");
43     // StatisticsReport meanLiftReport = (StatisticsReport)
        simulation.getReportManager().getReport("Mean Lift");
44
45     Map<String, ParameterConfig> globalParameters = new HashMap
        <>();
46     globalParameters.put("Implicit Unsteady Time-Step", new
        ParameterConfig(1.0E-4, "s"));
47     globalParameters.put("Inner Iterations", new
        ParameterConfig(15, ""));
48     setGlobalParameters(simulation, globalParameters);
49
50     // Prepare output directory and files
51     String outputDir = simulation.getSessionDir() + "/"
        "Hydrofoil_Control_Results";
52     createDirectory(outputDir);
53     String csvFilePath = outputDir + "/pid_results.csv";
54     String adaptiveTuningCsvPath = outputDir + "/"
        "adaptive_pid_tuning.csv";
55     setupCsv(csvFilePath);
56     setupAdaptiveTuningCsv(adaptiveTuningCsvPath);
57
58     String velocityScenesDir = outputDir + "/VelocityScenes";
59     String densityScenesDir = outputDir + "/DensityScenes";
60
61     createDirectory(velocityScenesDir);
62     createDirectory(densityScenesDir);
63
64     saveScenes(simulation, "Density", densityScenesDir + "/"

```

```

        DensityScene_", 0.01, 4.0);
65     saveScenes(simulation, "Velocity", velocityScenesDir + "/"
        VelocityScene_", 0.01, 4.0);
66     // Clear initial solution and start control loop
67     simulation.getSolution().clearSolution();
68     controlLift(simulation, LiftForceReport, csvFilePath,
        adaptiveTuningCsvPath);
69     exportSimulationData(outputDir);
70 }
71
72 private void controlLift(Simulation simulation, ForceReport
    LiftForceReport, String csvFilePath, String
    adaptiveTuningCsvPath) {
73     UserFieldFunction aoaFunction = (UserFieldFunction)
        simulation.getFieldFunctionManager().getFunction("FF -
        AoA");
74     aoaFunction.setDefinition("0*<1 deg>");
75
76     while (simulation.getSolution().getPhysicalTime() <= 5.0) {
77         double currentTime = simulation.getSolution().
            getPhysicalTime();
78
79         if (currentTime - lastRecordedTime >= samplingInterval)
            {
80             double Lift = LiftForceReport.getReportMonitorValue
                ();
81             absoluteErrorSum += Math.abs(Lift - TARGET_LIFT) *
                samplingInterval;
82
83             ema = alpha * Lift + (1 - alpha) * ema;
84
85             adjustAoA(ema, simulation, aoaFunction);
86             recordData(csvFilePath, currentTime, Lift, ema);
87             recordAdaptiveTuningData(adaptiveTuningCsvPath,
                currentTime);
88             lastRecordedTime = currentTime;
89         }
90
91         simulation.getSimulationIterator().step(1);
92     }
93
94     exportFinalSummary(csvFilePath);
95     savePlots(simulation);
96 }
97
98 private void adjustAoA(double ema, Simulation simulation,
    UserFieldFunction aoaFunction) {
99     double error = TARGET_LIFT - ema;
100
101     // Proportional term
102     double proportional = kp * error;
103
104     // Integral term with conditional update (anti-windup)
105     if (currentAoA > AOA_MIN && currentAoA < AOA_MAX) {
106         integralSum += error * samplingInterval;
107         integralSum = Math.max(Math.min(integralSum,

```

```

108         MAX_INTEGRAL_SUM), -MAX_INTEGRAL_SUM);
109     }
110     double integral = ki * integralSum;
111
112     // Derivative term with filtering
113     double derivative = (error - previousError) /
114         samplingInterval;
115
116     // Total correction
117     double correction = proportional + integral + derivative;
118
119     // Limit AoA change
120     correction = Math.max(Math.min(correction, MAX_DELTA_AOA),
121         -MAX_DELTA_AOA);
122
123     // Update AoA
124     currentAoA += correction;
125     currentAoA = Math.max(Math.min(currentAoA, AOA_MAX),
126         AOA_MIN);
127
128     aoaFunction.setDefinition(String.format("%f*<1 deg>",
129         currentAoA));
130     previousError = error;
131
132     // Adaptive PID tuning
133     adaptPIDParameters(error, derivative);
134 }
135
136 // Utility method to clamp values within a specified range
137 private void adaptPIDParameters(double error, double derivative
138 ) {
139     // Adjust kp, ki, kd based on system behavior
140
141     // Proportional gain (kp) adjustment based on the magnitude
142     // of the error
143     if (Math.abs(error) > 0.05 * TARGET_LIFT) {
144         kp = Math.min(kp + 0.1, 20.0); // Gradual increase,
145         // capped at 20.0
146     } else if (Math.abs(error) < 0.05 * TARGET_LIFT) {
147         kp = Math.max(kp - 0.05, 1.0); // Gradual decrease,
148         // floored at 1.0
149     }
150
151     // Integral gain (ki) adjustment based on small steady-state
152     // errors
153     if (Math.abs(error) < 0.01 * TARGET_LIFT) {
154         ki = Math.min(ki + 0.005, 0.5); // Slowly increase ki for
155         // fine-tuning
156     } else {
157         ki = Math.max(ki - 0.01, 0.1); // Decrease ki to prevent
158         // integral windup
159     }
160
161     // Derivative gain (kd) adjustment based on the rate of error
162     // change
163     if (Math.abs(derivative) > 1.0) {

```

```
151         kd = Math.min(kd + 0.05, 5.0); // Increase kd for fast-
152         } else {
153             kd = Math.max(kd - 0.02, 0.1); // Decrease kd when the
154             }
155     }
156     private static class ParameterConfig {
157         private final double value;
158         private final String units;
159
160         public ParameterConfig(double value, String units) {
161             this.value = value;
162             this.units = units;
163         }
164
165         public double getValue() {
166             return value;
167         }
168
169         public String getUnits() {
170             return units;
171         }
172     }
173
174     private void setGlobalParameters(Simulation simulation, Map<
175         String, ParameterConfig> parameters) {
176         GlobalParameterManager globalParameterManager = simulation
177             .get(GlobalParameterManager.class);
178
179         for (Map.Entry<String, ParameterConfig> entry : parameters
180             .entrySet()) {
181             String parameterName = entry.getKey();
182             ParameterConfig config = entry.getValue();
183
184             ScalarGlobalParameter parameter = (
185                 ScalarGlobalParameter) globalParameterManager.
186                 getObject(parameterName);
187             if (parameter != null) {
188                 Units units = (Units) simulation.getUnitsManager()
189                     .getObject(config.getUnits());
190                 parameter.getQuantity().setValueAndUnits(config.
191                     getValue(), units);
192                 simulation.println(parameterName + " set to " +
193                     config.getValue() + " " + config.getUnits());
194             } else {
195                 simulation.println("Global parameter not found: "
196                     + parameterName);
197             }
198         }
199     }
200
201     private void recordData(String csvFilePath, double time, double
202         Lift, double ema) {
203         try (BufferedWriter writer = new BufferedWriter(new
204             FileWriter(csvFilePath, true))) {
```

```

194         writer.write(String.format("%f,%f,%f,%f\n", time, Lift,
195             ema, currentAoA));
196     } catch (IOException e) {
197         getActiveSimulation().println("Error writing to CSV: "
198             + e.getMessage());
199     }
200 }
201
202 private void recordAdaptiveTuningData(String
203     adaptiveTuningCsvPath, double currentTime) {
204     try (BufferedWriter writer = new BufferedWriter(new
205         FileWriter(adaptiveTuningCsvPath, true))) {
206         writer.write(String.format("%f,%f,%f,%f\n", currentTime
207             , kp, ki, kd));
208     } catch (IOException e) {
209         getActiveSimulation().println("Error writing to
210             adaptive tuning CSV: " + e.getMessage());
211     }
212 }
213
214 private void setupCsv(String csvFilePath) {
215     try (BufferedWriter writer = new BufferedWriter(new
216         FileWriter(csvFilePath))) {
217         writer.write("Time,Lift Force,EMA Filtered Lift,AoA\n");
218         ;
219     } catch (IOException e) {
220         getActiveSimulation().println("Error creating CSV file:
221             " + e.getMessage());
222     }
223 }
224
225 private void setupAdaptiveTuningCsv(String
226     adaptiveTuningCsvPath) {
227     try (BufferedWriter writer = new BufferedWriter(new
228         FileWriter(adaptiveTuningCsvPath))) {
229         writer.write("Time,kp,ki,kd\n");
230     } catch (IOException e) {
231         getActiveSimulation().println("Error creating adaptive
232             tuning CSV: " + e.getMessage());
233     }
234 }
235
236 private void exportFinalSummary(String csvFilePath) {
237     try (BufferedWriter writer = new BufferedWriter(new
238         FileWriter(csvFilePath, true))) {
239         writer.write("Final Summary\n");
240         writer.write(String.format("Cumulative IAE,%f\n",
241             absoluteErrorSum));
242         writer.write(String.format("Final AoA,%f\n", currentAoA
243             ));
244     } catch (IOException e) {
245         getActiveSimulation().println("Error writing final
246             summary: " + e.getMessage());
247     }
248 }

```

```
234 private void exportSimulationData(String baseDirectory) {
235     Simulation simulation = getActiveSimulation();
236
237     // Create directories for Lift and Angle of Attack data
238     String liftDirectory = baseDirectory + "/LiftData";
239     String aoaDirectory = baseDirectory + "/AoAData";
240     String meanLiftDirectory = baseDirectory + "/MeanLiftData";
241     createDirectory(liftDirectory);
242     createDirectory(boaDirectory);
243     createDirectory(meanLiftDirectory);
244
245     // Export Lift and AoA data
246     exportMonitorPlotData(simulation, "Lift Force Plot",
247         liftDirectory + "/lift_data.csv");
248     exportMonitorPlotData(simulation, "AoA Monitor Plot",
249         aoaDirectory + "/aoa_data.csv");
250     exportMonitorPlotData(simulation, "Mean Lift Monitor Plot",
251         meanLiftDirectory + "/Mean_Lift_data.csv");
252 }
253
254 private void exportMonitorPlotData(Simulation simulation, String
255     plotName, String filePath) {
256     // Get the specified monitor plot by name
257     MonitorPlot monitorPlot = (MonitorPlot) simulation.
258         getPlotManager().getPlot(plotName);
259
260     if (monitorPlot != null) {
261         // Optionally configure axis bounds (adjust values as
262             needed)
263         Cartesian2DAxisManager axisManager = (
264             Cartesian2DAxisManager) monitorPlot.getAxisManager();
265         axisManager.setAxesBounds(new Vector(Arrays.<AxisManager.
266             AxisBounds>asList(
267             new AxisManager.AxisBounds("Left Axis", 0.0, false,
268                 1.0, false),
269             new AxisManager.AxisBounds("Bottom Axis", 0.0, false,
270                 1.0, false)
271         )));
272
273         // Export data to the specified file path
274         monitorPlot.export(resolvePath(filePath), ",");
275         simulation.println(plotName + " data exported successfully
276             to: " + filePath);
277     } else {
278         simulation.println("Plot not found: " + plotName);
279     }
280 }
281
282 private void saveScenes(Simulation simulation, String
283     sceneName, String filePath, double deltaTime, double
284     stopTime) {
285     // Get the specified scene by name
286     Scene scene = simulation.getSceneManager().getScene(sceneName
287         );
288     if (scene != null) {
289         SceneUpdate sceneUpdate = scene.getSceneUpdate();
290     }
291 }
```



```

276
277 // Enable animation saving
278 sceneUpdate.setSaveAnimation(true);
279
280 // Set update mode to Delta Time
281 sceneUpdate.getUpdateModeOption().setSelected(
282     StarUpdateModeOption.Type.DELTATIME);
283
284 // Configure Delta Time settings
285 DeltaTimeUpdateFrequency deltaTimeUpdateFrequency =
286     sceneUpdate.getDeltaTimeUpdateFrequency();
287 Units timeUnits = ((Units) simulation.getUnitsManager().
288     getObject("s"));
289 deltaTimeUpdateFrequency.setDeltaTime(String.valueOf(
290     deltaTime), timeUnits); // Convert deltaTime to String
291
292 // Configure Stop Event based on range quantity
293 deltaTimeUpdateFrequency.setStopEnabled(true);
294 deltaTimeUpdateFrequency.setStopTime(stopTime, timeUnits)
295     ;
296
297 // Set animation file path and file name prefix
298 String directoryPath = filePath.substring(0, filePath.
299     lastIndexOf("/"));
300 String fileNamePrefix = filePath.substring(filePath.
301     lastIndexOf("/") + 1);
302
303 sceneUpdate.setAnimationFilePath(directoryPath); // Set
304     directory path
305 sceneUpdate.setAnimationFilenameBase(fileNamePrefix); //
306     Set file name prefix
307
308 simulation.println(sceneName + " animation settings
309     configured successfully. Files will be saved in: " +
310     directoryPath);
311 } else {
312     simulation.println("Scene not found: " + sceneName);
313 }
314 }
315
316 private void savePlots(Simulation simulation) {
317     String plotsDir = simulation.getSessionDir() + "/Plots
318         Directory";
319     createDirectory(plotsDir);
320
321     String[] plotNames = {
322         "Lift Force Plot",
323         "Maximum Co",
324         "Reports Plot",
325         "AoA Monitor Plot",
326         "Mean Lift Monitor Plot"
327     };
328
329     for (String plotName : plotNames) {
330         StarPlot plot = (StarPlot) simulation.getPlotManager().

```

```
320         getObject(plotName);
321         if (plot != null) {
322             plot.encode(plotsDir + "/" + plotName.replaceAll("
323                 ", "_") + ".png", "PNG", 1920, 1080); // Set
324                 resolution here
325         } else {
326             simulation.println("Plot not found: " + plotName);
327         }
328     }
329
330     private void createDirectory(String path) {
331         File dir = new File(path);
332         if (!dir.exists()) {
333             dir.mkdirs();
334         }
335     }
336 }
```

A.9 Java Macro for Hydrofoil Control of Ride Height and Trim Angle

This section analyzes the Java macro designed for STAR-CCM+ to control the ride height and trim angle of a vessel using a PID-based control system. The macro adjusts the angles of attack (AoA) of front and rear hydrofoils based on real-time feedback.

The macro is designed to be adaptable based on the specific control objectives. If the goal is to achieve a target lift instead of maintaining ride height and trim, the target value and corresponding error terms should be modified accordingly, and the control input should be applied to the relevant hydrofoil(s).

For cases where only a single hydrofoil is used, parameters and calculations related to multiple hydrofoils should be removed to simplify the control logic. Conversely, when extending the control to an additional hydrofoil (such as introducing a third hydrofoil), the necessary code lines should be added to account for the additional control surface. The number of hydrofoils and their respective roles in the control strategy should be properly reflected in the macro to ensure effective adjustment of the AoA for stability and performance.

This modular approach allows the macro to be extended or simplified based on the user's simulation needs and vessel configuration.

A.9.1 Functionality Overview

The macro performs the following tasks:

1. Initializes parameters including PID gains, AoA limits, and sampling intervals.
2. Configures simulation parameters and data directories.

3. Monitors ride height and trim angle errors.
4. Applies PID control to adjust hydrofoil AoA.
5. Logs data and generates visualizations.

A.9.2 Function Descriptions

The macro consists of several key functions:

1. **Main Function:** `execute()`
 - Initializes simulation objects and reports.
 - Calls `controlHydrofoil()` to execute the PID control loop.
 - Saves plots and exports simulation data.
2. `controlHydrofoil(Simulation, ReportMonitor, ReportMonitor, String, double)`
 - Runs a loop until the simulation reaches 15s.
 - Reads the ride height and trim angle from reports.
 - Computes errors relative to target values.
 - Calls `adjustAoA()` to update hydrofoil angles.
 - Logs data in a CSV file.
3. `setGlobalParameters(Simulation, Map<String, ParameterConfig>)`
 - Defines and sets global simulation parameters such as:
 - Time step
 - Mass properties
 - Iteration limits
4. `adaptPIDGains(double error)`
 - Dynamically adjusts PID gains based on the error magnitude using:

$$K_p, K_i, K_d = f(\text{error}) \quad (\text{A.1})$$

5. `adjustAoA(...)` **and** `calculatePIDControl(...)`
 - Implements the PID control logic:
 - Computes proportional, integral, and derivative terms.
 - Limits integral windup and constrains AoA changes.
 - Updates AoA in STAR-CCM+ user field function.
6. `recordData(...)` **and** `setupCsv(...)`
 - Handles data logging:
 - Saves time, ride height, AoA, errors, and PID parameters.
 - Exports data in CSV format.

7. createDirectory(String dirPath)
 - Creates required directories for storing output files.
8. saveScenes(...)
 - Generates scene animations for:
 - Velocity field visualization.
 - Density field visualization.

Listing A.9: Java Code

```
1 package macro;
2 import star.common.*;
3 import star.base.neo.*;
4 import java.util.*;
5 import java.io.*;
6 import star.base.report.*;
7 import star.sixdof.*;
8 import star.vis.*;
9 import star.flow.*;
10
11 public class PID02 extends StarMacro {
12     private static final double TARGET_RIDE_HEIGHT_1 = 0.0;
13         // Target ride height (meters)
14     private static final double TARGET_TRIM_ANGLE = 0.0;
15     private static final double INITIAL_SAMPLING_INTERVAL = 0.001;
16         // Sampling interval (seconds)
17     private static final double MAX_INTEGRAL_SUM = 10.0;
18         // Anti-windup limit
19     private static final double AOA_MIN = -8;
20         // Minimum AoA (degrees)
21     private static final double AOA_MAX = 10.0;
22         // Maximum AoA (degrees)
23     private static final double MAX_DELTA_AOA = 0.5;
24         // Max AoA change per step (degrees)
25
26     private double kp = 5 , minkp = 0, maxkp = 5;           // Default
27         proportional gain
28     private double ki = 0.01, minki = 0, maxki = 0.05; // Default
29         integral gain
30     private double kd = 0.000015, minkd = 0, maxkd = 2;    //
31         Default derivative gain
32
33     private double kp_trim = 5;                             // Default
34         proportional gain
35     private double ki_trim = 0.05;                          // Default
36         integral gain 0.05
37     private double kd_trim = 0.000001;                      //
38         Default derivative gain
39
40     private double samplingInterval = INITIAL_SAMPLING_INTERVAL;
41     private double integralSumFront = 0.0;
42     private double integralSumRear = 0.0;
43
44     private double previousErrorRideHeight1 = 0.0;
45     private double previousErrorTrim = 0.0;
```

```

34
35 private double currentAoAFront = 0.0;
36 private double currentAoARear = 0.0;
37
38 private double absoluteErrorSumRideHeight1 = 0.0;
39 private double absoluteErrorTrim = 0.0;
40
41 private double lastRecordedTime = 0.0;
42
43 //private static final double alpha = 1;
44
45 @Override
46 public void execute() {
47     Simulation simulation = getActiveSimulation();
48     TranslationReport rideHeightReport1 = ((TranslationReport)
49         simulation.getReportManager().getReport("Ride Height"));
50     AngleReport trimAngleReport = ((AngleReport) simulation.
51         getReportManager().getReport("Trim Angle"));
52
53     ReportMonitor rideHeightMonitor1 = ((ReportMonitor)
54         simulation.getMonitorManager().getMonitor("Ride Height
55         Monitor"));
56     ReportMonitor trimAngleMonitor = ((ReportMonitor)
57         simulation.getMonitorManager().getMonitor("Trim Angle
58         Monitor"));
59
60     // Set global parameters
61     Map<String, ParameterConfig> globalParameters = new HashMap
62     <>();
63     globalParameters.put("Implicit Unsteady Time-Step", new
64         ParameterConfig(0.001, "s"));
65     globalParameters.put("System Body Mass", new
66         ParameterConfig(10, "kg"));
67     globalParameters.put("Inner Iterations", new
68         ParameterConfig(10, ""));
69
70     setGlobalParameters(simulation, globalParameters);
71     setSystemReleaseTime(simulation);
72     double ReleaseTime = getSystemReleaseTime(simulation);
73     String outputDir = simulation.getSessionDir() + "/"
74         + "hydrofoil_control_results";
75     String velocityScenesDir = outputDir + "/VelocityScenes";
76     String densityScenesDir = outputDir + "/DensityScenes";
77     String densityZoomScenesDir = outputDir + "/"
78         + "DensityZoomScenes";
79     String pressureScenesDir = outputDir + "/PressureScenes";
80
81     createDirectory(outputDir);
82     createDirectory(velocityScenesDir);
83     createDirectory(densityScenesDir);
84     createDirectory(densityZoomScenesDir);
85     createDirectory(pressureScenesDir);
86
87     String csvFilePath = outputDir + "/hydrofoil_control.csv";
88

```

```

78     // Save Density Scene animation
79     saveScenes(simulation, "Density", densityScenesDir + "/"
80         DensityScene_", 0.01, 15.0);
81
82     // Save Velocity Scene animation
83     saveScenes(simulation, "Velocity", velocityScenesDir + "/"
84         VelocityScene_", 0.01, 15.0);
85
86     saveScenes(simulation, "Density_Zoom", densityZoomScenesDir + "
87         /DensityZoomScene_", 0.01, 15.0);
88
89     saveScenes(simulation, "Pressure_Scene", pressureScenesDir
90         + "/PressureScene_", 0.01, 15.0);
91
92     setupCsv(csvFilePath);
93     simulation.getSolution().clearSolution();
94
95     controlHydrofoil(simulation, rideHeightMonitor1,
96         trimAngleMonitor, csvFilePath, ReleaseTime);
97     exportSimulationData(outputDir);
98     savePlots(simulation);
99 }
100
101 private void controlHydrofoil(Simulation simulation,
102     ReportMonitor rideHeightMonitor1, ReportMonitor
103     trimAngleMonitor, String csvFilePath, double ReleaseTime) {
104     UserFieldFunction aoaFrontFunction = (UserFieldFunction)
105         simulation.getFieldFunctionManager().getFunction("AoA1")
106     ;
107     UserFieldFunction aoaRearFunction = (UserFieldFunction)
108         simulation.getFieldFunctionManager().getFunction("AoA2")
109     ;
110     aoaFrontFunction.setDefinition("0*<1 deg>");
111     aoaRearFunction.setDefinition("0*<1 deg>");
112
113     while (simulation.getSolution().getPhysicalTime() <= 15.0)
114     {
115         double currentTime = simulation.getSolution().
116             getPhysicalTime();
117
118         if (currentTime >= ReleaseTime)
119         {
120             if (currentTime - lastRecordedTime >= samplingInterval)
121             {
122                 double currentRideHeight1 = ((TranslationReport)
123                     rideHeightMonitor1.getReport()).getValue();
124                 double currentTrimAngle = ((AngleReport)
125                     trimAngleMonitor.getReport()).getValue();
126
127                 double errorRideHeight1 = TARGET_RIDE_HEIGHT_1 -
128                     currentRideHeight1;
129                 double errorTrim = TARGET_TRIM_ANGLE -
130                     currentTrimAngle;
131
132                 adaptPIDGains(errorRideHeight1);
133                 // Update AoA for the front hydrofoil

```

```

116         currentAoAFront = adjustAoA(
117             errorRideHeight1,          // Error for ride
118                 height adjustment (front)
119             simulation,
120             aoaFrontFunction,          // Front AoA field
121                 function
122             kp,                        // PID gains for
123                 front
124             ki,
125             kd,
126             integralSumFront,          // Integral sum for
127                 front
128             previousErrorRideHeight1, // Previous error
129                 for front
130             currentAoAFront,          // Current AoA for
131                 front
132             AOA_MIN,                  // Minimum AoA
133             AOA_MAX                    // Maximum AoA
134         );
135
136         // Update AoA for the rear hydrofoil
137         currentAoARear = adjustAoA(
138             errorTrim,                // Error for trim
139                 adjustment (rear)
140             simulation,
141             aoaRearFunction,          // Rear AoA field
142                 function
143             kp_trim,                  // PID gains for
144                 rear
145             ki_trim,
146             kd_trim,
147             integralSumRear,          // Integral sum for
148                 rear
149             previousErrorTrim,        // Previous error
150                 for rear
151             currentAoARear,          // Current AoA for
152                 rear
153             AOA_MIN,                  // Minimum AoA
154             AOA_MAX                    // Maximum AoA
155         );
156
157         absoluteErrorSumRideHeight1 += Math.abs(
158             errorRideHeight1);
159         absoluteErrorTrim += Math.abs(errorTrim);
160
161         // Record data
162         recordData(csvFilePath, currentTime,
163             currentRideHeight1, currentAoAFront, kp, ki, kd,
164             errorRideHeight1, currentTrimAngle,
165             currentAoARear, kp_trim, ki_trim,
166             kd_trim, errorTrim);
167
168         lastRecordedTime = currentTime;
169     }
170     simulation.getSimulationIterator().step(1);

```

```
156         }
157         else {
158             // Skip control actions during ReleaseTime
159             simulation.println("ReleaseTime not yet fulfilled.
160                               Waiting...");
161             simulation.getSimulationIterator().step(1); //
162                 Increment time step until ramp time is reached
163         }
164     }
165
166     private static class ParameterConfig
167     {
168         private final double value;
169         private final String units;
170
171         public ParameterConfig(double value, String units)
172         {
173             this.value = value;
174             this.units = units;
175         }
176
177         public double getValue()
178         {
179             return value;
180         }
181
182         public String getUnits() { return units; }
183     }
184
185     private void setGlobalParameters(Simulation simulation, Map<
186         String, ParameterConfig> parameters)
187     {
188         GlobalParameterManager globalParameterManager = simulation
189             .get(GlobalParameterManager.class);
190
191         for (Map.Entry<String, ParameterConfig> entry : parameters
192             .entrySet()) {
193             String parameterName = entry.getKey();
194             ParameterConfig config = entry.getValue();
195
196             ScalarGlobalParameter parameter = (
197                 ScalarGlobalParameter) globalParameterManager.
198                 getObject(parameterName);
199             if (parameter != null) {
200                 Units units = (Units) simulation.getUnitsManager()
201                     .getObject(config.getUnits());
202                 parameter.getQuantity().setValueAndUnits(config.
203                     getValue(), units);
204                 simulation.println(parameterName + " set to " +
205                     config.getValue() + " " + config.getUnits());
206             } else {
207                 simulation.println("Global parameter not found: "
208                     + parameterName);
209             }
210         }
211     }
212 }
```



```

201     }
202 }
203
204 private void setSystemReleaseTime(Simulation simulation) {
205     // Get Implicit Unsteady Time-Step parameter value
206     ScalarGlobalParameter implicitTimeStep = (
207         ScalarGlobalParameter) simulation.get(
208         GlobalParameterManager.class).getObject("Implicit Unsteady
209         Time-Step");
210
211     if (implicitTimeStep != null) {
212         double timeStepValue = implicitTimeStep.getQuantity().
213             getRawValue();
214
215         // Set System Release Time based on Implicit Unsteady
216         Time-Step
217         ScalarGlobalParameter systemReleaseTime = (
218             ScalarGlobalParameter) simulation.get(
219             GlobalParameterManager.class)
220             .getObject("System Release Time");
221
222         if (systemReleaseTime != null) {
223             // Dynamically set the value of System Release Time
224             systemReleaseTime.getQuantity().setValue(
225                 timeStepValue * 1000);
226             simulation.println("System Release Time set as: " +
227                 timeStepValue * 1000);
228         } else {
229             simulation.println("System Release Time parameter not
230                 found.");
231         }
232     } else {
233         simulation.println("Implicit Unsteady Time-Step parameter
234             not found.");
235     }
236 }
237
238 private double getSystemReleaseTime(Simulation simulation) {
239     ScalarGlobalParameter systemReleaseTime = (
240         ScalarGlobalParameter) simulation.get(
241         GlobalParameterManager.class).getObject("System Release
242         Time");
243
244     if (systemReleaseTime != null) {
245         return systemReleaseTime.getQuantity().getRawValue();
246     } else {
247         simulation.println("System Release Time parameter not
248             found.");
249         return 0.0; // Default value if not found
250     }
251 }
252
253 // Utility method to clamp values within a specified range
254 private double clamp(double value, double min, double max) {

```

```

    return Math.max(min, Math.min(max, value));}

242
243 private void adaptPIDGains(double error)
244 {
245     double Error = Math.abs(error);
246     // Adjust gains based on error magnitude
247     if (Error < 0.1 * TARGET_RIDE_HEIGHT_1) {
248         kp = clamp(2.5, minkp, maxkp); // Clamp gains to stay
249         // within limits
250         ki = clamp(0.005, minki, maxki);
251         kd = clamp(0.000005, mind, maxd);
252     } else if (Error < 0.05 * TARGET_RIDE_HEIGHT_1)
253     {
254         kp = clamp(1, minkp, maxkp); // Clamp gains to stay
255         // within limits
256         ki = clamp(0.0005, minki, maxki);
257         kd = clamp(0.0000005, mind, maxd);
258     }
259     else if (Error < 0.01 * TARGET_RIDE_HEIGHT_1)
260     {
261         kp = clamp(0.001, minkp, maxkp); // Clamp gains to stay
262         // within limits
263         ki = clamp(0.00005, minki, maxki);
264         kd = clamp(0.00000005, mind, maxd);
265     }
266 }

267 private double adjustAoA
268 (
269     double error,
270     Simulation simulation,
271     UserFieldFunction aoaFunction,
272     double kp,
273     double ki,
274     double kd,
275     double integralSum,
276     double previousError,
277     double currentAoA,
278     double aoaMin,
279     double aoaMax) {
280     // PID control for hydrofoil (ride height adjustment or other
281     // adjustment)
282     double correction = calculatePIDControl(error, kp, ki, kd,
283     integralSum, previousError);
284
285     // Update AoA based on the PID correction
286     currentAoA += correction;
287     currentAoA = Math.max(Math.min(currentAoA, aoaMax), aoaMin); //
288     // Ensure AoA is within bounds
289
290     // Set the new AoA in the field function
291     aoaFunction.setDefinition(String.format("%f*<1 deg>",
292     currentAoA));
293
294     // Return the updated AoA for the caller to update the
295     // respective state

```

```

289     return currentAoA;
290 }
291
292
293 private double calculatePIDControl(double error, double kp,
    double ki, double kd, double integralSum, double
    previousError) {
294     double proportional = kp * error;
295     integralSum += error * samplingInterval;
296     integralSum = Math.max(Math.min(integralSum,
        MAX_INTEGRAL_SUM), -MAX_INTEGRAL_SUM);
297     double integral = ki * integralSum;
298
299     double derivative = (error - previousError) /
        samplingInterval;
300     double derivativeTerm = kd * derivative;
301     previousError = error;
302     double correction = proportional + integral +
        derivativeTerm;
303     return Math.max(Math.min(correction, MAX_DELTA_AOA), -
        MAX_DELTA_AOA);
304 }
305
306 private void recordData(String csvFilePath, double currentTime,
    double currentRideHeight1, double currentAoAfront,
307     double kp, double ki, double kd, double
        errorRideHeight1, double
        currentTrimAngle,
308     double currentAoARear, double kp_trim,
        double ki_trim, double kd_trim, double
        errorTrim) {
309     try (BufferedWriter writer = new BufferedWriter(new
        FileWriter(csvFilePath, true))) {
310         writer.write(String.format("%f,%f,%f,%f,%f,%f,%f,%f,%f,%f",
            ,%f,%f,%f%n",
311             currentTime, currentRideHeight1, currentAoAfront, kp,
            ki, kd, errorRideHeight1, currentTrimAngle,
312             currentAoARear, kp_trim, ki_trim, kd_trim, errorTrim)
            );
313     } catch (IOException e) {
314         getActiveSimulation().println("Error writing to CSV: " +
            e.getMessage());
315     }
316 }
317
318 private void setupCsv(String csvFilePath) {
319     try (BufferedWriter writer = new BufferedWriter(new
        FileWriter(csvFilePath))) {
320         writer.write("Time,Ride_Height,AoA_Front,Kp,Ki,Kd,
            Error_Ride_Height,Trim_Angle,AoA_Rear,Kp_Trim,
            Ki_Trim,Kd_Trim,Error_Trim\n");
321     } catch (IOException e) {
322         getActiveSimulation().println("Error creating CSV file:
            " + e.getMessage());
323     }
324 }

```

```
325
326     private void createDirectory(String dirPath) {
327         File directory = new File(dirPath);
328         if (!directory.exists()) {
329             directory.mkdirs();
330         }
331     }
332
333     private void saveScenes(Simulation simulation, String sceneName
334         , String filePath, double deltaTime, double stopTime) {
335         // Get the specified scene by name
336         Scene scene = simulation.getSceneManager().getScene(sceneName
337             );
338         if (scene != null) {
339             SceneUpdate sceneUpdate = scene.getSceneUpdate();
340
341             // Enable animation saving
342             sceneUpdate.setSaveAnimation(true);
343
344             // Set update mode to Delta Time
345             sceneUpdate.getUpdateModeOption().setSelected(
346                 StarUpdateModeOption.Type.DELTATIME);
347
348             // Configure Delta Time settings
349             DeltaTimeUpdateFrequency deltaTimeUpdateFrequency =
350                 sceneUpdate.getDeltaTimeUpdateFrequency();
351             Units timeUnits = ((Units) simulation.getUnitsManager().
352                 getObject("s"));
353             deltaTimeUpdateFrequency.setDeltaTime(String.valueOf(
354                 deltaTime), timeUnits); // Convert deltaTime to String
355
356             // Configure Stop Event based on range quantity
357             deltaTimeUpdateFrequency.setStopEnabled(true);
358             deltaTimeUpdateFrequency.setStopTime(stopTime, timeUnits)
359                 ;
360
361             // Set animation file path and file name prefix
362             String directoryPath = filePath.substring(0, filePath.
363                 lastIndexOf("/"));
364             String fileNamePrefix = filePath.substring(filePath.
365                 lastIndexOf("/") + 1);
366
367             sceneUpdate.setAnimationFilePath(directoryPath); // Set
368                 directory path
369             sceneUpdate.setAnimationFilenameBase(fileNamePrefix); //
370                 Set file name prefix
371
372             simulation.println(sceneName + " animation settings
373                 configured successfully. Files will be saved in: " +
374                 directoryPath);
375         } else {
376             simulation.println("Scene not found: " + sceneName);
377         }
378     }
379
380     private void exportSimulationData(String baseDirectory) {
```

```

368     Simulation simulation = getActiveSimulation();
369
370     // Create directories for Lift and Angle of Attack data
371     String plotsdata_CSV_Export = baseDirectory + "/AllData";
372     createDirectory(plotsdata_CSV_Export);
373     // Export Lift and AoA data
374     exportMonitorPlotData(simulation, "Lift Plot",
375         plotsdata_CSV_Export + "/lift_data.csv");
376     exportMonitorPlotData(simulation, "AoA Plot",
377         plotsdata_CSV_Export + "/aoa_data.csv");
378     exportMonitorPlotData(simulation, "Lift Summation Plot",
379         plotsdata_CSV_Export + "/lift_Summation.csv");
380     exportMonitorPlotData(simulation, "Ride Height Plot",
381         plotsdata_CSV_Export + "/ride_Height.csv");
382     exportMonitorPlotData(simulation, "Trim Angle Plot",
383         plotsdata_CSV_Export + "/Trim_Angle.csv");
384     exportMonitorPlotData(simulation, "Mean Trim Angle Plot",
385         plotsdata_CSV_Export + "/Mean_Trim_Angle.csv");
386     exportMonitorPlotData(simulation, "Mean Ride Height Plot",
387         plotsdata_CSV_Export + "/Mean_Ride_Height.csv");
388 }
389
390 private void exportMonitorPlotData(Simulation simulation, String
391     plotName, String filePath) {
392     // Get the specified monitor plot by name
393     MonitorPlot monitorPlot = (MonitorPlot) simulation.
394         getPlotManager().getPlot(plotName);
395
396     if (monitorPlot != null) {
397         // Optionally configure axis bounds (adjust values as
398             needed)
399         Cartesian2DAxisManager axisManager = (
400             Cartesian2DAxisManager) monitorPlot.getAxisManager();
401         axisManager.setAxesBounds(new Vector(Arrays.<AxisManager.
402             AxisBounds>asList(
403             new AxisManager.AxisBounds("Left Axis", 0.0, false,
404                 1.0, false),
405             new AxisManager.AxisBounds("Bottom Axis", 0.0, false,
406                 1.0, false)
407         )));
408
409         // Export data to the specified file path
410         monitorPlot.export(resolvePath(filePath), ",");
411         simulation.println(plotName + " data exported
412             successfully to: " + filePath);
413     } else {
414         simulation.println("Plot not found: " + plotName);
415     }
416 }
417
418 private void savePlots(Simulation simulation) {
419     String plotsDir = simulation.getSessionDir() + "/Plots
420         Directory";
421     createDirectory(plotsDir);
422
423     String[] plotNames = {

```

```
408         "Lift Plot",
409         "AoA Plot",
410         "Lift Summation Plot",
411         "Maximum Co",
412         "Ride Height Plot",
413         "Trim Angle Plot",
414         "Mean Trim Angle Plot",
415         "Mean Ride Height Plot"
416
417     };
418
419     for (String plotName : plotNames) {
420         StarPlot plot = (StarPlot) simulation.getPlotManager().
            getObject(plotName);
421         if (plot != null) {
422             plot.encode(plotsDir + "/" + plotName.replaceAll("
", "_") + ".png", "PNG", 1920, 1080);
423         } else {
424             simulation.println("Plot not found: " + plotName);
425         }
426     }
427 }
428 }
```

B

Appendix 2

B.1 Steady-State 2D Setup Raw Lift Force Plot

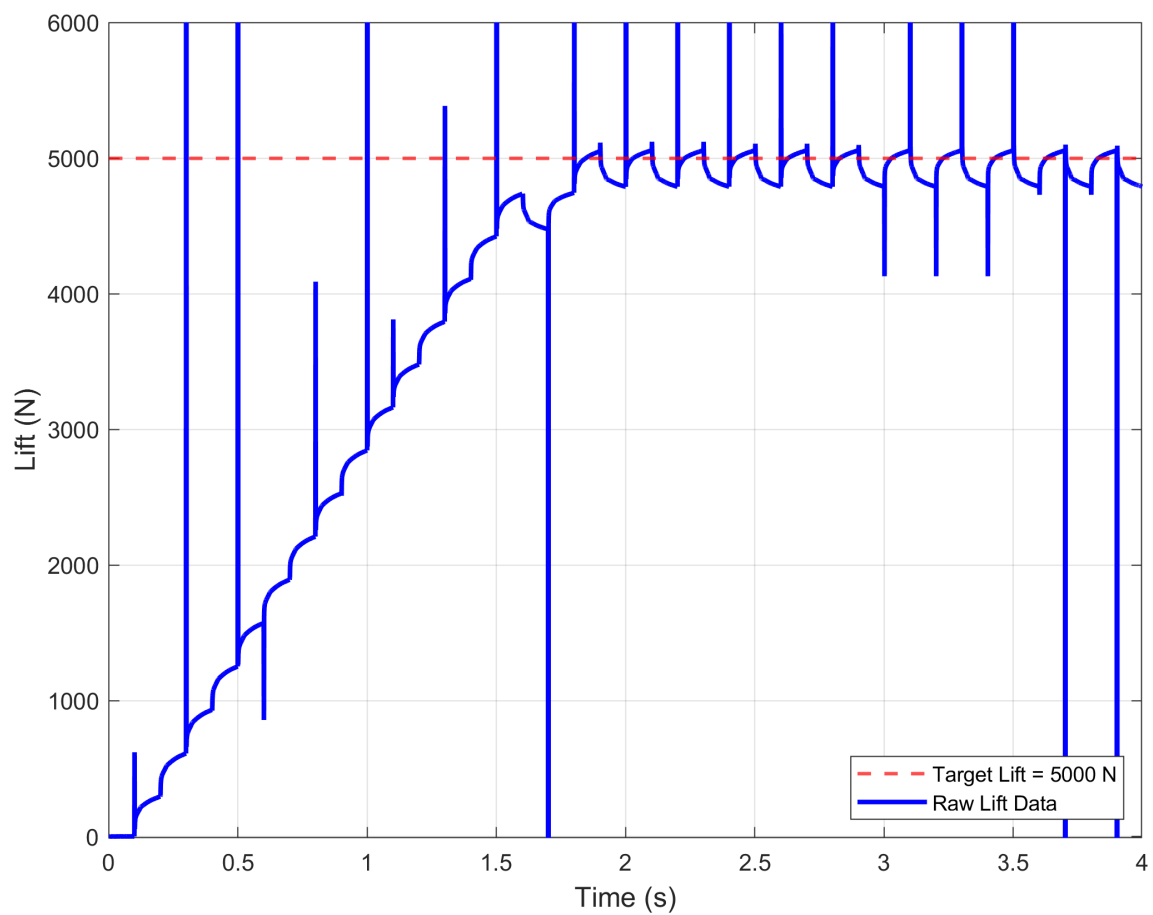


Figure B.1: Raw Lift Force vs Time for Steady-State 2D Setup [??]

B.2 Unsteady-State 2D Setup Raw Lift Force Plot

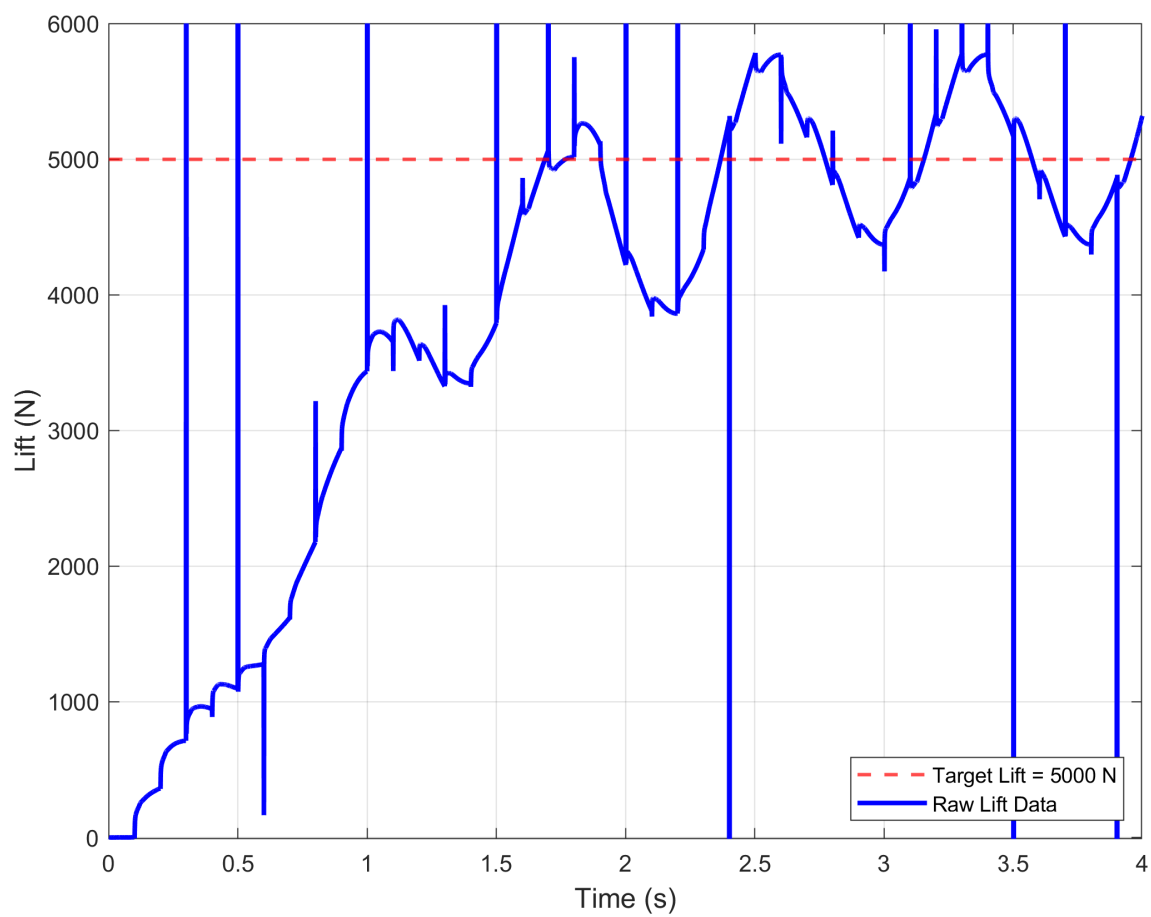


Figure B.2: Raw Lift Force vs Time for Unsteady-State 2D Setup [??]

B.3 Raw Lift Plot for Dual Hydrofoil Control

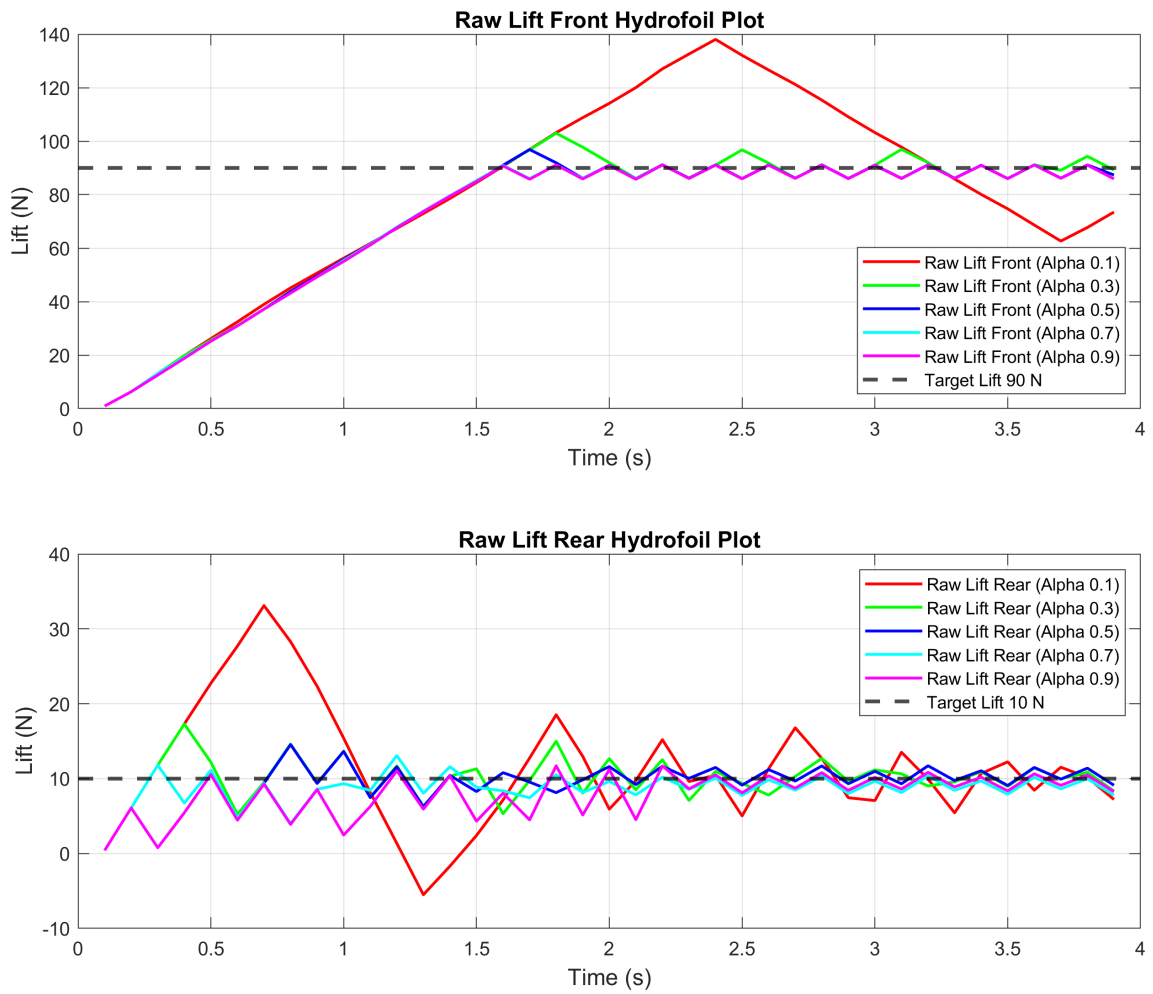


Figure B.3: Raw Lift vs Time for Dual Hydrofoil Control [4.2.3.2]

B.4 Plots for Hydrofoil Depth 0.5CL 4.2.3.3

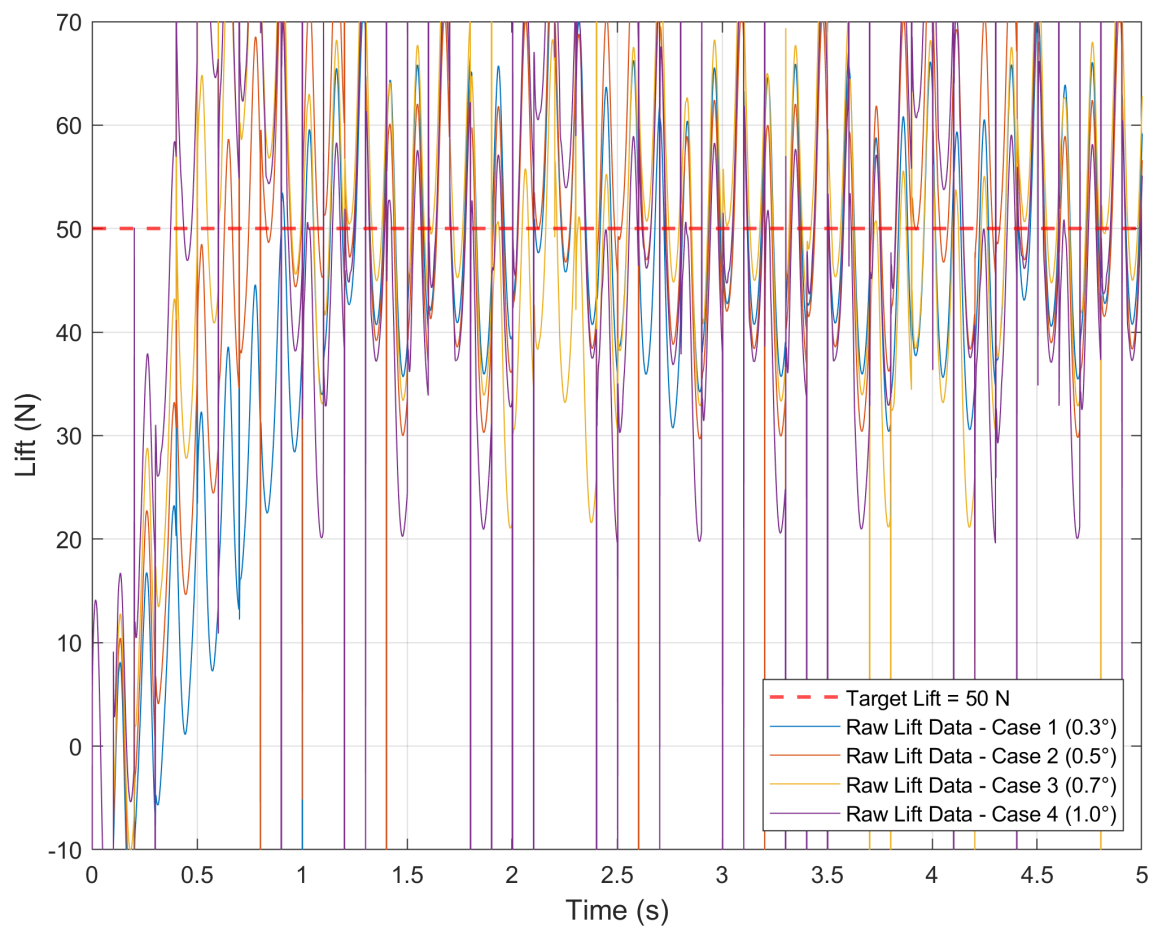


Figure B.4: Raw Lift vs Time

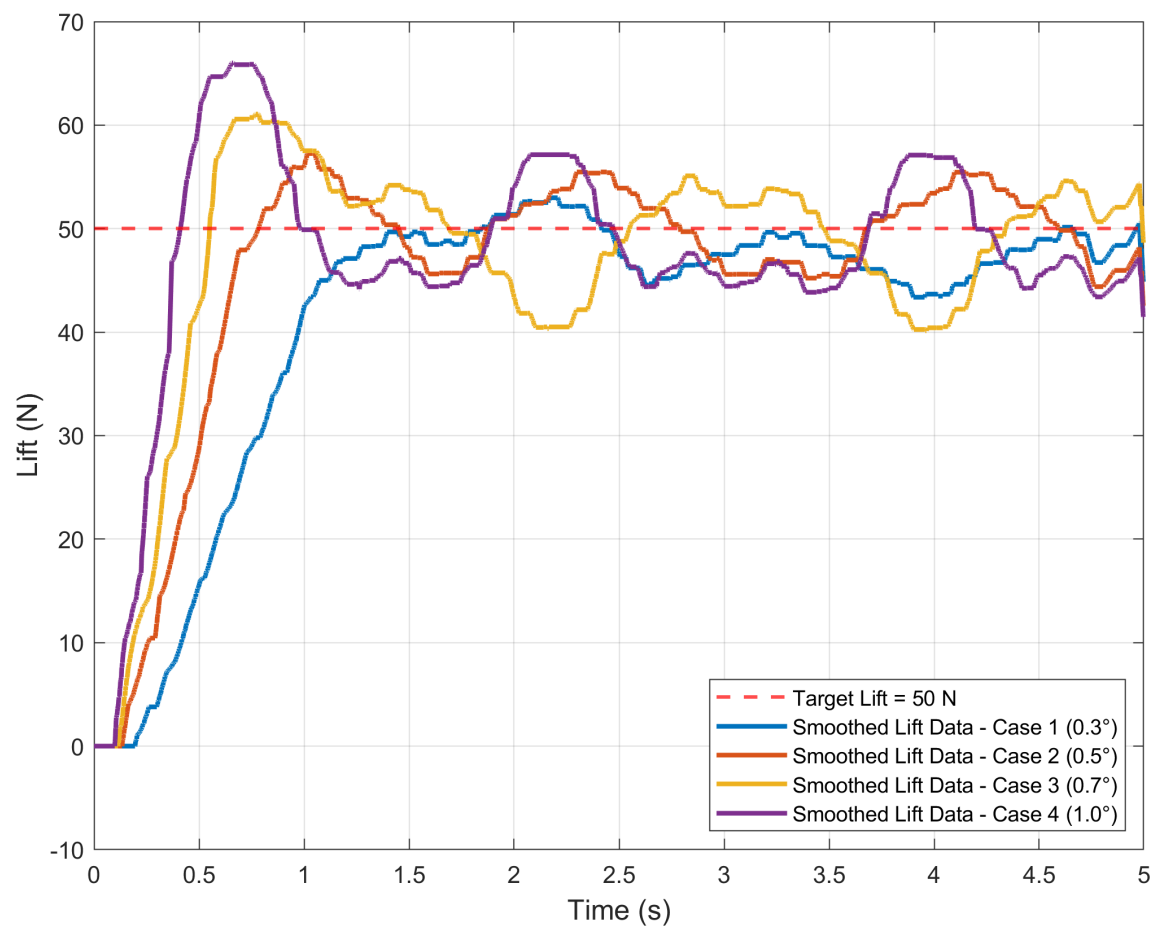


Figure B.5: Smoothed Lift vs Time

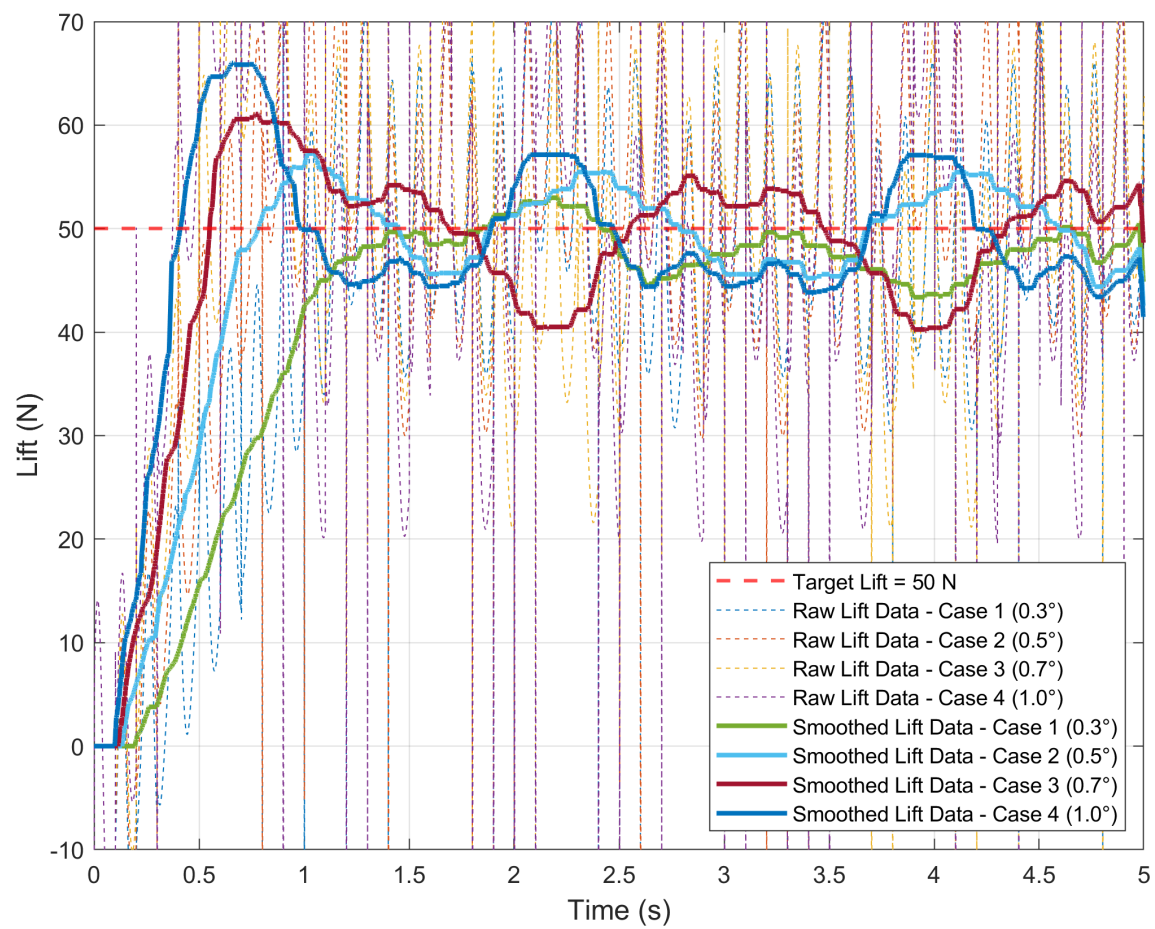


Figure B.6: Combined Lift vs Time

B.5 Plots for Hydrofoil Depth 1.5CL 4.2.3.3

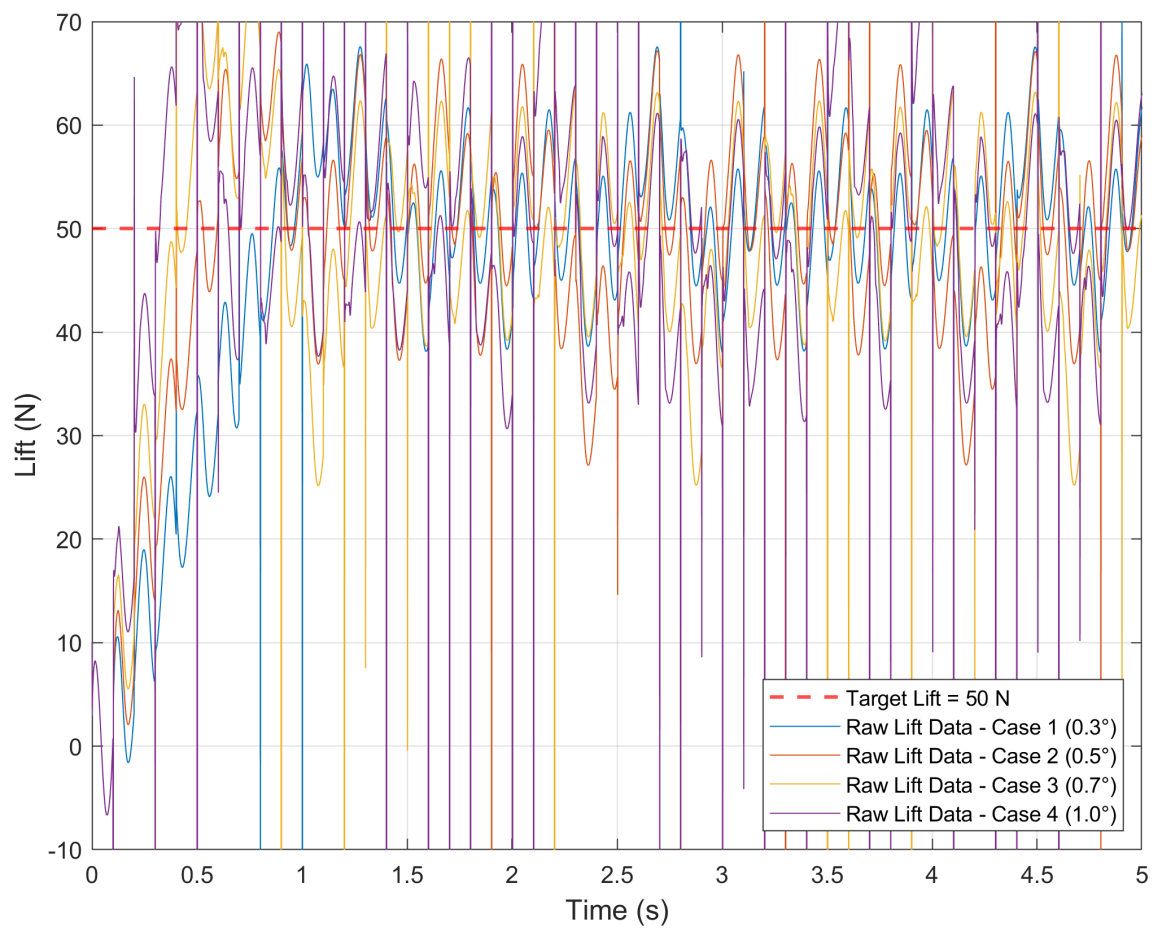


Figure B.7: Raw Lift vs Time

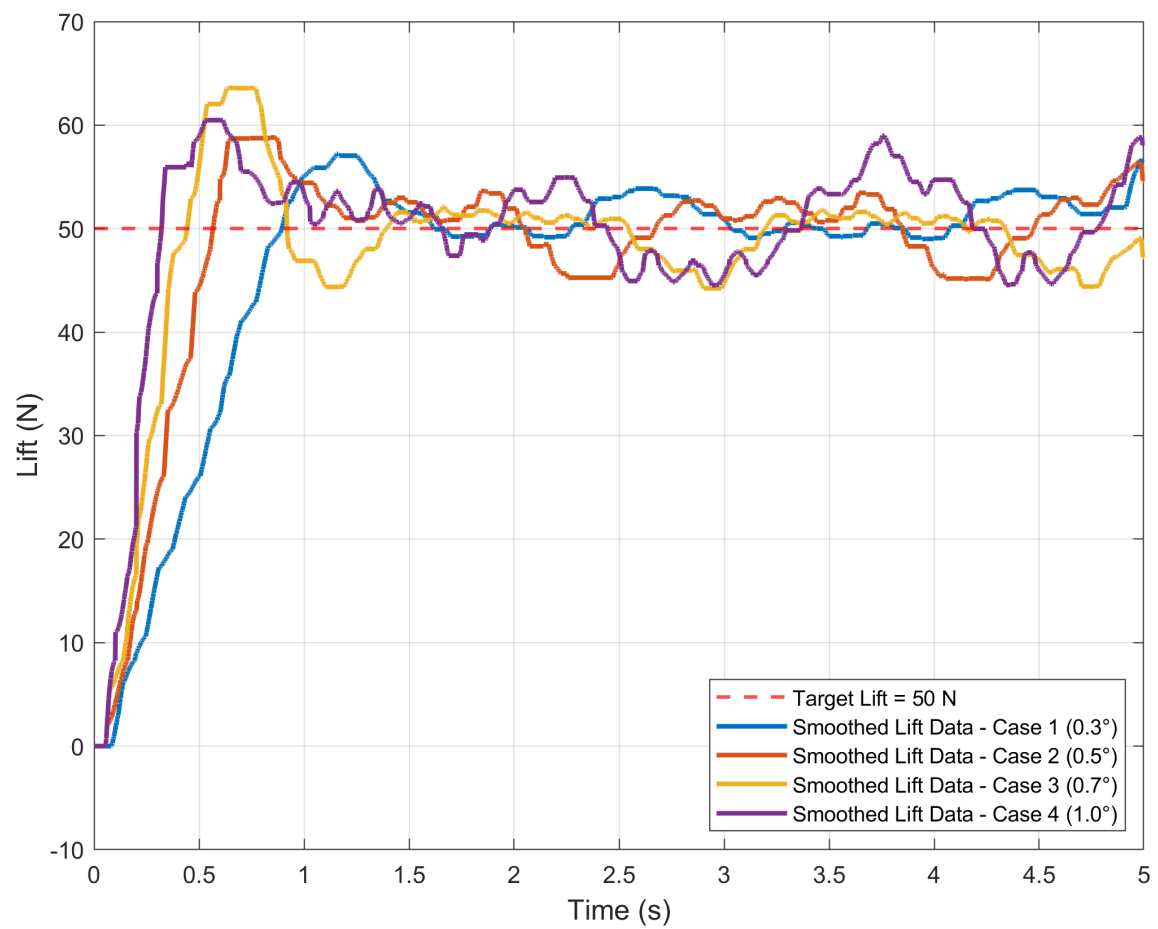
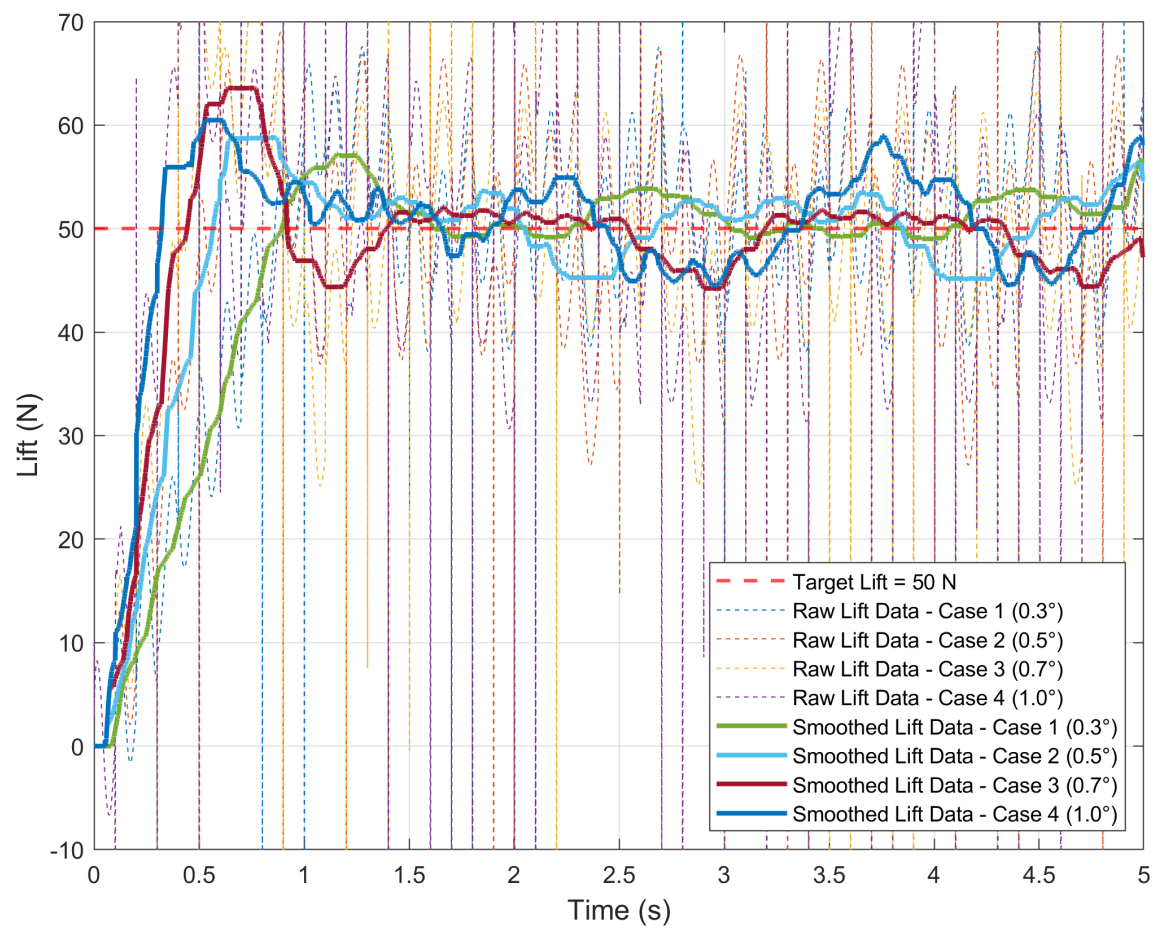


Figure B.8: Smoothed Lift vs Time

**Figure B.9: Combined Lift vs Time**

B.6 Plots for Hydrofoil Depth 3.0CL 4.2.3.3

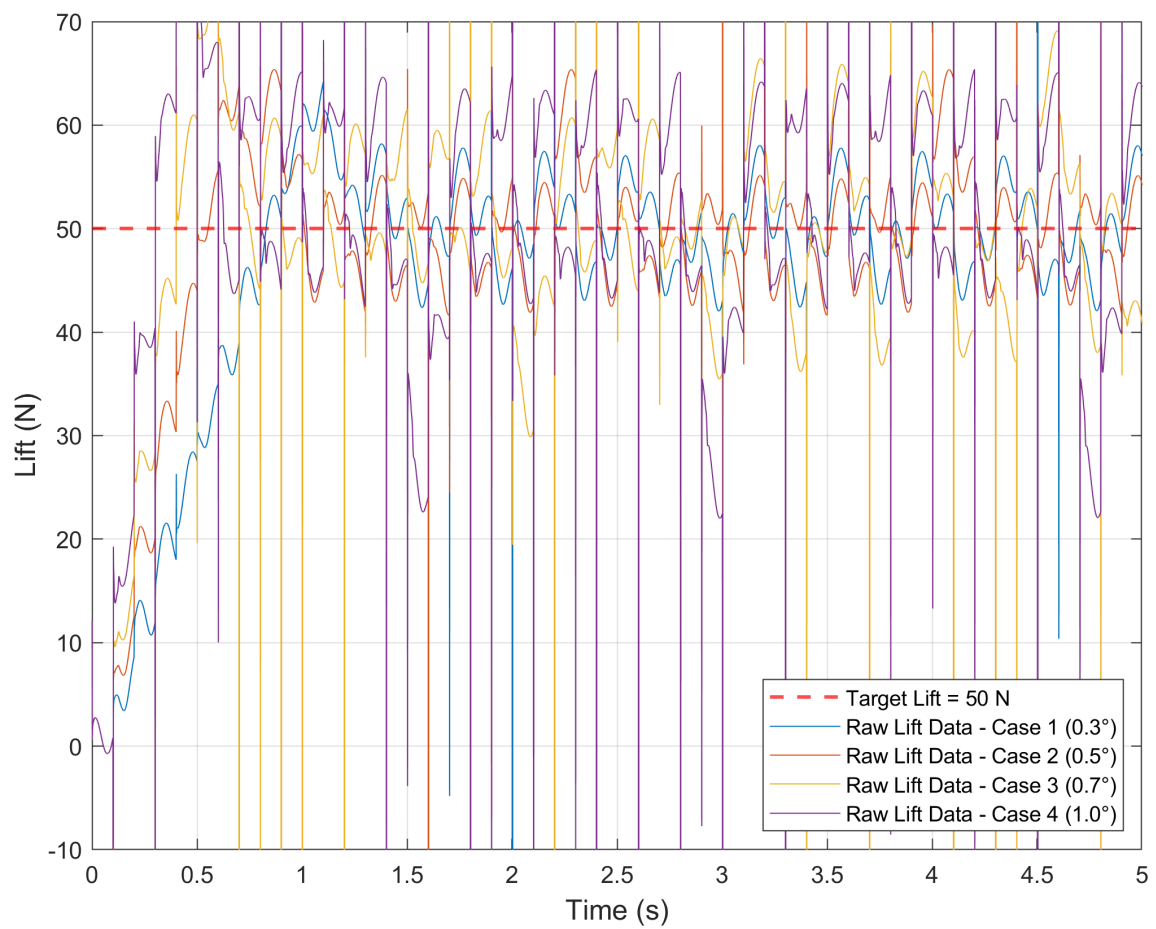


Figure B.10: Raw Lift vs Time

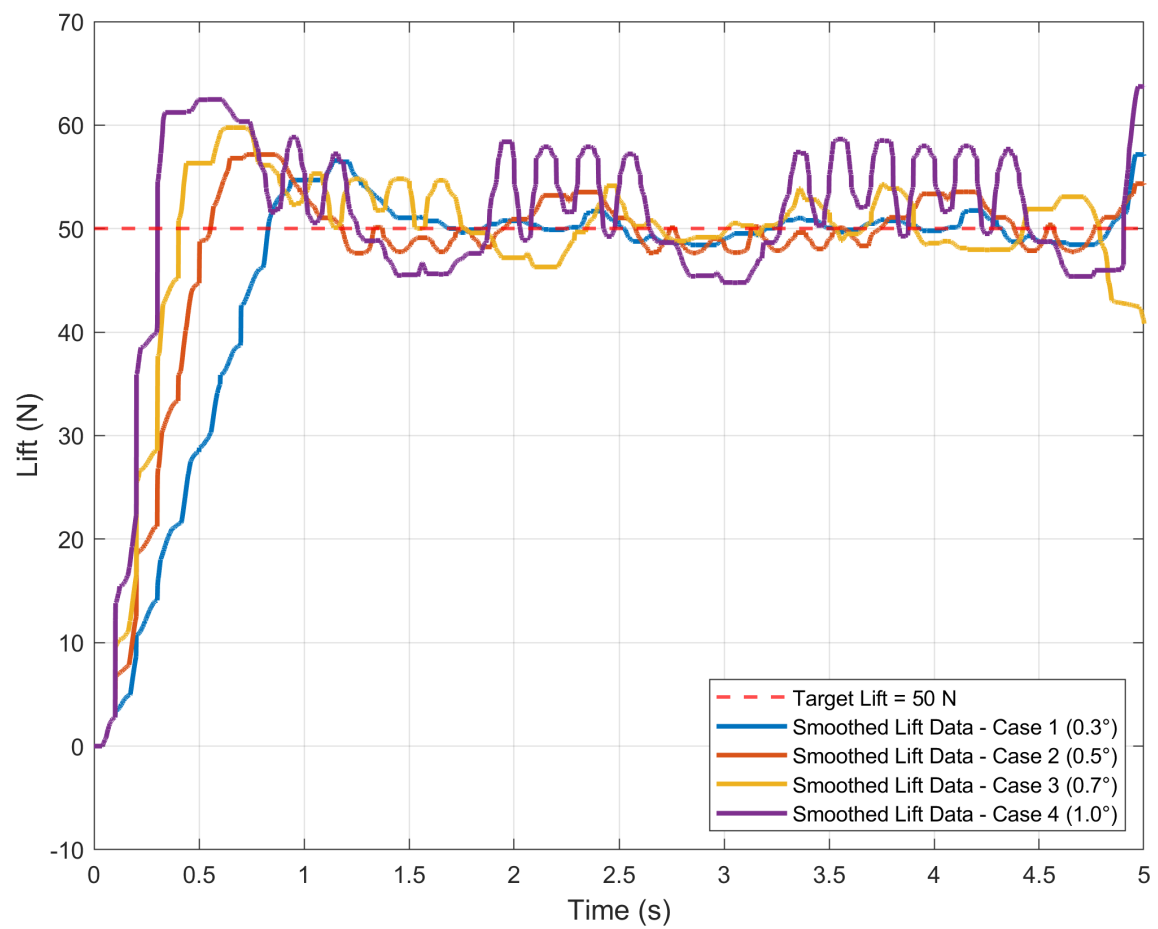


Figure B.11: Smoothed Lift vs Time

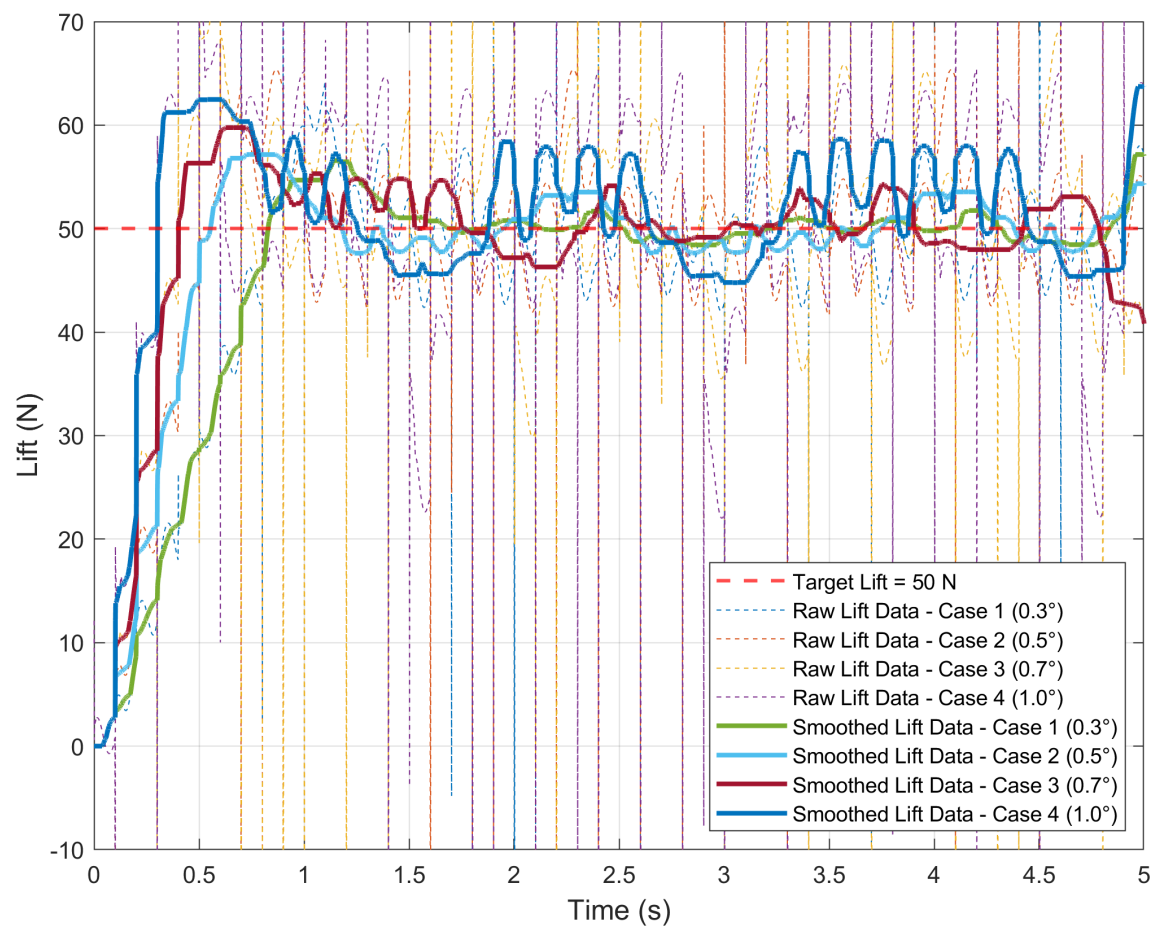


Figure B.12: Combined Lift vs Time

C

Project Timeline

ID	Task Description	Start	Finish	Duration
1	Understanding the objectives of the project and run test cases to check PID control	09/09/2024	20/09/2024	10 day
2	Literature Review	18/09/2024	09/10/2024	16 day
3	Observation of parameter changes (of control system) in the existing model	30/09/2024	18/10/2024	15 day
4	Steady state runs, convergence tests	21/10/2024	25/10/2024	5 day
5	Redefined control system (new control approach, Simulink model setup and testing)	25/10/2024	11/11/2024	12 day
6	Introduction of Noise and noise filters in Simulink	12/11/2024	18/11/2024	5 day
7	Filters and Unsteady Lift coefficient and AoA behaviour simulation	19/11/2024	25/11/2024	5 day
8	Report structuring and introduction to HPC systems	26/11/2024	03/12/2024	6 day
9	Semi-2D modelling and Multiphase integration	04/12/2024	18/12/2024	11 day
10	Modified objectives and implementing new control approach for semi-2D model	19/12/2024	10/01/2025	17 day
11	Dual foil control setup	13/01/2025	20/01/2025	6 day
12	Semi-2D ride height control implementation	21/01/2025	31/01/2025	9 day
13	Depth control for single Hydrofoil	30/01/2025	10/02/2025	8 day
14	Improving semi-2D case for effective control for ride height and pitch	07/02/2025	17/02/2025	7 day
15	Develop 3D model	17/02/2025	26/02/2025	8 day
17	Semi-2D results compilations	03/03/2025	17/03/2025	11 day
16	Report and documentation	17/12/2024	17/03/2025	65 day
18	Final documentation and overview	17/03/2025	21/03/2025	5 day



CHALMERS
UNIVERSITY OF TECHNOLOGY